

CNN2 pigmentation classifier: alignment method & mask-channel comparison

Compares three CNN2 pigmentation-binary checkpoints, all trained on the exact same dataset/architecture (up to the input-row count) and hyperparameters, differing only in how the input tensor is built:

- **bcftools_chain** (`alignment_mapping="bcftools_chain"`, `feature_mode="signals_only"`): a cohort-wide "expanded axis" built once per gene across the whole selected sample set, wide enough to fit every INDEL seen in the cohort. No INDEL/SNP mask channels, only the aligned AlphaGenome signal (`configs/predictors/genotype_based/pigmentation/pigmentation_binary.yaml`).
- **reference_realign** (`alignment_mapping="reference_realign"`, `feature_mode="signals_only"`): each individual realigned independently, straight onto true reference-genome coordinates -- insertions collapse via `max()` onto their anchor position, deletions zero-fill, no cohort-wide axis at all (`configs/predictors/genotype_based/pigmentation/pigmentation_binary_reference_realign.yaml`).
- **all_masks** (`alignment_mapping="bcftools_chain"`, `feature_mode="signals_and_masks"`, both `indel_include_valid_mask` and `indel_include_snp_mask` enabled): same `bcftools_chain` alignment as the first model, but the CNN also sees 4 explicit INDEL/SNP location mask channels per gene per haplotype (insertion, deletion, valid, SNP) on top of the aligned signal -- 10 channels/gene instead of 6, so Stage1's vertical kernel/stride is widened from 6 to 10 to keep collapsing one full gene block into a single row (`configs/predictors/genotype_based/pigmentation/pigmentation_binary_all_masks.yaml`).

The first two isolate the effect of how AlphaGenome predictions get mapped onto a common axis; `all_masks` instead isolates the effect of what the CNN is allowed to see (raw signal only vs. signal + explicit variant-location channels), holding the alignment method fixed at `bcftools_chain`. `bcftools_chain` vs `reference_realign` were smoke-tested against each other directly before training (see `tests/test_reference_realign_mapper.py` and the ad hoc real-data check that found Pearson $r=1.0000$ between the two methods' tensors for real samples, with divergence limited to a handful of positions exactly at INDEL loci) -- this notebook is about comparing the trained models, not re-litigating the alignment math itself.

Sections:

1. Load all three configs
2. Load data + all three trained CNN2 checkpoints
3. Predictions on the held-out test split (accuracy / confusion matrix, side by side)
4. DeepLIFT attributions (per-class mean, reusing `analysis.interpretability.DeepLIFT`)
5. Visualization of each model's mean attribution map

6. Attribution comparison across the three models (the one genuinely new piece of code here -- everything else reuses existing interpretability infra)
7. Top attribution windows, and whether they shift between models
8. DeepLIFT attribution vs. GTF exon structure -- does the model attend to exons?

In [1]:

```
%matplotlib inline
import json
import os
import sys
from pathlib import Path

REPO_ROOT = Path("/home/breno/I2CA/genomics")
if str(REPO_ROOT / "src") not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / "src"))
os.chdir(REPO_ROOT)

import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.metrics import confusion_matrix, classification_report

from genomics.predictors.genotype_based.config import load_config, generate_experiment_name, get_experiment_runs_dir
from genomics.predictors.genotype_based.data.pipeline import prepare_data
from genomics.predictors.genotype_based.models import CNN2AncestryPredictor
from genomics.predictors.genotype_based.analysis.interpretability import (
    DeepLIFT,
    summarize_deeplift_by_track,
    find_top_deeplift_windows,
    build_haplotype_track_layout,
)
from genomics.predictors.genotype_based.analysis.run_deeplift import _plot_mean_deeplift

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda

Section 1 -- Load all three configs

Each experiment directory is derived from `generate_experiment_name`, which encodes both the `alignment_mapping` tag and (since the `all_masks` config was added) the `feature_mode/mask-flags` tag, so all three configs resolve to distinct, non-colliding on-disk directories automatically.

In [2]:

```
CONFIG_PATHS = {
    "bcftools_chain": REPO_ROOT / "configs/predictors/genotype_based/pigmentation/pigmentation_binary.yaml",
    "reference_realign": REPO_ROOT / "configs/predictors/genotype_based/pigmentation/pigmentation_binary_reference_realign.yaml",
    "all_masks": REPO_ROOT / "configs/predictors/genotype_based/pigmentation/pigmentation_binary_all_masks.yaml",
}

configs = {name: load_config(path) for name, path in CONFIG_PATHS.items()}

EXPERIMENT_DIRS = {
    name: get_experiment_runs_dir(config) / generate_experiment_name(config)
    for name, config in configs.items()
}

for name, config in configs.items():
    di = config.dataset_input
    print(f"{name}: alignment_mapping={di.alignment_mapping!r} feature_mode={di.feature_mode!r}")
    print(f"    experiment_dir={EXPERIMENT_DIRS[name]}")
    print(f"    checkpoint exists: {(EXPERIMENT_DIRS[name] / 'models' / 'best_accuracy.pt').exists()}")
```

```
bcftools_chain: alignment_mapping='bcftools_chain' feature_mode='signals_only'
    experiment_dir=results/genotype_based_predictor/runs/
cnn2_pigmentation_rna_seq_H1+H2_haplotype_channels_32768_log_s1k6x32f16_s2f32_s3f64_gpavg_fc256_L100-40_relu_0.5_adam
    checkpoint exists: True
reference_realign: alignment_mapping='reference_realign' feature_mode='signals_only'
    experiment_dir=results/genotype_based_predictor/runs/
cnn2_pigmentation_rna_seq_H1+H2_haplotype_channels_reference_realign_32768_log_s1k6x32f16_s2f32_s3f64_gpavg_fc256_L100-40_relu_0.5_adam
    checkpoint exists: True
all_masks: alignment_mapping='bcftools_chain' feature_mode='signals_and_masks'
    experiment_dir=results/genotype_based_predictor/runs/cnn2_pigmentation_rna_seq_H1+H2_haplotype_channels_signals_and_masks-valid-
snp_32768_log_s1k10x32f16_s2f32_s3f64_gpavg_fc256_L100-40_relu_0.5_adam
    checkpoint exists: True
```

Section 2 -- Load data + all three trained CNN2 checkpoints

`prepare_data` transparently reuses each config's already-materialized processed-tensor cache (each combination of `alignment_mapping` / `feature_mode` / `mask` flags lands in a distinct cache directory automatically, since they're all part of the cache's view hash -- no manual cache-path bookkeeping needed). Note `all_masks` has a wider input tensor (110 rows/haplotype vs. 66 for the other two, since it adds 4 mask channels per gene) and a different Stage1 kernel, so `build_model` picks up the right shape from each config independently.

In [3]:

```
def build_model(config, dataset):
    input_shape = dataset.get_input_shape()
    num_classes = dataset.get_num_classes()
    if config.model.type.upper() != "CNN2":
        raise ValueError(f"This notebook only compares CNN2 checkpoints, got {config.model.type}")
    return CNN2AncestryPredictor(config, input_shape, num_classes)

def load_checkpoint(model, checkpoint_path):
    checkpoint = torch.load(checkpoint_path, map_location=device)
    state_dict = checkpoint.get("model_state_dict", checkpoint)
    model.load_state_dict(state_dict)
    model.eval()
    return model

datasets = {}
loaders = {}
models = {}

for name, config in configs.items():
    experiment_dir = EXPERIMENT_DIRS[name]
    full_ds, train_loader, val_loader, test_loader = prepare_data(config, experiment_dir)
    datasets[name] = full_ds
    loaders[name] = {"train": train_loader, "val": val_loader, "test": test_loader}

    model = build_model(config, full_ds).to(device)
    checkpoint_path = experiment_dir / "models" / "best_accuracy.pt"
    models[name] = load_checkpoint(model, checkpoint_path)
    n_params = sum(p.numel() for p in model.parameters())
    print(f"{name}: loaded {checkpoint_path.name} ({n_params:,} params), test set size = {len(test_loader.dataset)}")

class_names = datasets["bcftools_chain"].get_class_names()
print(f"class_names={class_names}")

# prepare_data's dataset-report step (genomics.core.metrics.save_classification_plots)
# forces the matplotlib backend to Agg (matplotlib.use("Agg", force=True)) as a side effect,
# which silently breaks plt.show() in every cell below. Re-enable the inline backend now that
# data loading is done.
%matplotlib inline
```

✓ Cache válido e compatível

Cache encontrado: rna_seq_H1+H2_32768_ds1_log_shuf_ont3_view568c1eff121a

Cache: 2026-07-08T20:07:18.644142 | 1072 amostras

Preparando train_data.pt...

✓ 761 samples (sharded preload)

Preparando val_data.pt...

✓ 149 samples (sharded preload)

Preparando test_data.pt...

✓ 162 samples (sharded preload)

✓ Train:761 Val:149 Test:162

Dataset report salvo:
results/cache/genotype_based_predictor/pigmentation/datasets/rna_seq_H1+H2_32768_ds1_log_shuf_ont3_view568c1eff121a/report

Modelo CNN2 criado:

- Input shape efetivo: 132 x 32768 (1 canal)

- Genes selecionados: **11** genes → **132** linhas
- Stage **1**: **16** filters, **kernel**=(**6**, **32**), **stride**=(**6**, **32**) → **16** x **22** x **1024**
- Stage **2**: **32** filters, **kernel**=(**1**, **5**), **stride**=(**1**, **2**), **padding**=(**0**, **2**) → **32** x **22** x **512**
- Stage **3**: **64** filters, **kernel**=(**1**, **5**), **stride**=(**1**, **2**), **padding**=(**0**, **2**) → **64** x **22** x **256**
- Global Pool (avg): **kernel**=(**1**, **256**) → **64** x **22** x **1**
- FC: **1408** → **256** → **2**
- Params: **377,202**

bcftools_chain: loaded best_accuracy.pt (377,202 params), test set size = 162

✓ Cache válido e compatível

Cache encontrado: rna_seq_H1+H2_32768_ds1_log_shuf_ont3_viewcedc94e55802

Cache: **2026-07-29T14:41:16.300986** | **1072** amostras

Preparando train_data.pt...

✓ **761** samples (sharded preload)

Preparando val_data.pt...

✓ **149** samples (sharded preload)

Preparando test_data.pt...

✓ **162** samples (sharded preload)

✓ Train:**761** Val:**149** Test:**162**

Dataset report salvo:

results/cache/genotype_based_predictor/pigmentation/datasets/rna_seq_H1+H2_32768_ds1_log_shuf_ont3_viewcedc94e55802/report

Modelo CNN2 criado:

- Input shape efetivo: **132** x **32768** (**1** canal)
- Genes selecionados: **11** genes → **132** linhas
- Stage **1**: **16** filters, **kernel**=(**6**, **32**), **stride**=(**6**, **32**) → **16** x **22** x **1024**
- Stage **2**: **32** filters, **kernel**=(**1**, **5**), **stride**=(**1**, **2**), **padding**=(**0**, **2**) → **32** x **22** x **512**
- Stage **3**: **64** filters, **kernel**=(**1**, **5**), **stride**=(**1**, **2**), **padding**=(**0**, **2**) → **64** x **22** x **256**
- Global Pool (avg): **kernel**=(**1**, **256**) → **64** x **22** x **1**
- FC: **1408** → **256** → **2**

- Params: **377,202**

reference_realign: loaded best_accuracy.pt (377,202 params), test set size = 162

✓ Cache válido e compatível

Cache encontrado: rna_seq_H1+H2_32768_ds1_log_shuf_ont3_viewca26f2f3b461

Cache: **2026-08-01T21:15:44.994638 | 1072 amostras**

Preparando train_data.pt...

✓ **761** samples (sharded preload)

Preparando val_data.pt...

✓ **149** samples (sharded preload)

Preparando test_data.pt...

✓ **162** samples (sharded preload)

✓ Train:**761** Val:**149** Test:**162**

Dataset report salvo:

results/cache/genotype_based_predictor/pigmentation/datasets/rna_seq_H1+H2_32768_ds1_log_shuf_ont3_viewca26f2f3b461/report

Modelo CNN2 criado:

- Input shape efetivo: 220 x 32768 (1 canal)
- Genes selecionados: 11 genes → 220 linhas
- Stage 1: 16 filters, kernel=(10, 32), stride=(10, 32) → 16 x 22 x 1024
- Stage 2: 32 filters, kernel=(1, 5), stride=(1, 2), padding=(0, 2) → 32 x 22 x 512
- Stage 3: 64 filters, kernel=(1, 5), stride=(1, 2), padding=(0, 2) → 64 x 22 x 256
- Global Pool (avg): kernel=(1, 256) → 64 x 22 x 1
- FC: 1408 → 256 → 2
- Params: 379,250

```
all_masks: loaded best_accuracy.pt (379,250 params), test set size = 162
class_names=['strong pigmentation', 'weak pigmentation']
```

Section 3 -- Predictions on the held-out test split

Both models are evaluated on their own config's test split (same `random_seed` / `family_split_mode` in both configs, so the underlying individuals are the same -- only the input tensors differ). Accuracy here should be directly comparable to each run's own `val_best_accuracy_results.json` as a sanity cross-check.

In [4]:

```
def collect_predictions(model, loader):
    y_true, y_pred = [], []
    with torch.no_grad():
        for batch in loader:
            features, targets = batch[0], batch[1]
            logits = model(features.to(device))
            preds = logits.argmax(dim=1).cpu().numpy()
            y_true.extend(targets.numpy().tolist())
            y_pred.extend(preds.tolist())
    return np.array(y_true), np.array(y_pred)

predictions = {}
for name in configs:
    y_true, y_pred = collect_predictions(models[name], loaders[name]["test"])
    predictions[name] = {"y_true": y_true, "y_pred": y_pred}
    acc = (y_true == y_pred).mean()
    print(f"=== {name} (test, n={len(y_true)}) === accuracy={acc:.4f}")
    print(classification_report(y_true, y_pred, target_names=class_names))
```

```

=== bcftools_chain (test, n=162) === accuracy=0.8704
      precision    recall  f1-score   support

strong pigmentation      0.89      0.93      0.91      115
weak pigmentation       0.81      0.72      0.76       47

      accuracy
macro avg      0.85      0.83      0.84      162
weighted avg   0.87      0.87      0.87      162

=== reference_realign (test, n=162) === accuracy=0.8889
      precision    recall  f1-score   support

strong pigmentation      0.92      0.92      0.92      115
weak pigmentation       0.81      0.81      0.81       47

      accuracy
macro avg      0.87      0.87      0.87      162
weighted avg   0.89      0.89      0.89      162

=== all_masks (test, n=162) === accuracy=1.0000
      precision    recall  f1-score   support

strong pigmentation      1.00      1.00      1.00      115
weak pigmentation       1.00      1.00      1.00       47

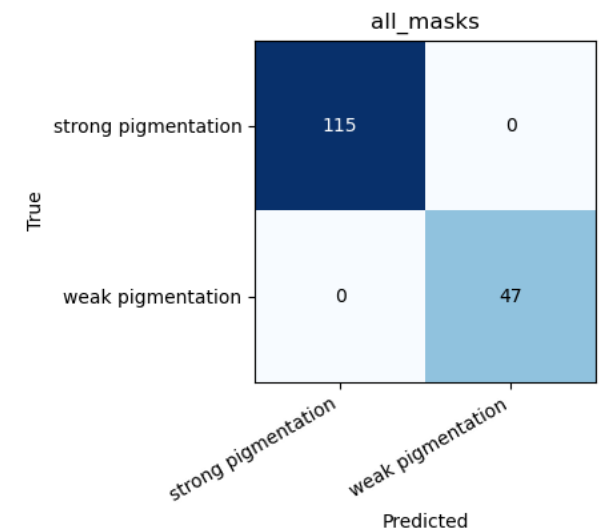
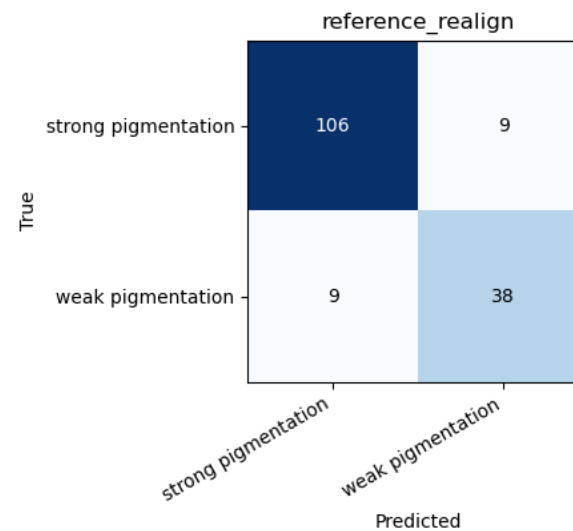
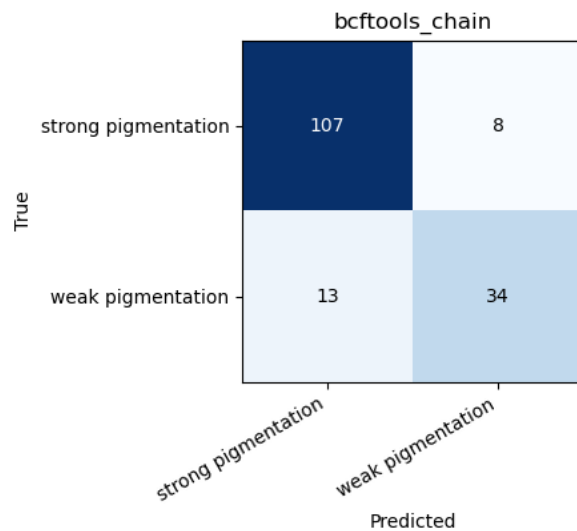
      accuracy
macro avg      1.00      1.00      1.00      162
weighted avg   1.00      1.00      1.00      162

```

In [5]:

```
n_models = len(configs)
fig, axes = plt.subplots(1, n_models, figsize=(5.5 * n_models, 4.5))
for ax, name in zip(axes, configs):
    cm = confusion_matrix(predictions[name]["y_true"], predictions[name]["y_pred"])
    im = ax.imshow(cm, cmap="Blues")
    ax.set_title(name)
    ax.set_xticks(range(len(class_names)))
    ax.set_xticklabels(class_names, rotation=30, ha="right")
    ax.set_yticks(range(len(class_names)))
    ax.set_yticklabels(class_names)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, str(cm[i, j]), ha="center", va="center",
                    color="white" if cm[i, j] > cm.max() / 2 else "black")
fig.suptitle("Confusion matrix: bcftools_chain vs reference_realign vs all_masks (test split)")
fig.tight_layout()
plt.show()
```

Confusion matrix: bcftools_chain vs reference_realign vs all_masks (test split)



Section 4 -- DeepLIFT attributions (per-class mean)

Reuses `DeepLIFT.generate_class_mean` directly (the same call `analysis/run_deeplift.py`'s CLI makes) -- it averages DeepLIFT attributions over every test-split sample of a given true class, in one call per (alignment method, class). `baseline_type="zeros"` is used throughout: since every feature is log-normalized before the model sees it, an all-zeros input corresponds to the per-position training-set mean (same convention `06_interpretability.ipynb` uses for the AlphaGenome-notebooks project -- kept consistent here for a comparable interpretation, even though these are separate model families).

In [6]:

```
deeplift_results = {} # name -> {class_idx: {"class_name":..., "mean_attr":..., "mean_input":..., "n":...}}

for name in configs:
    model = models[name]
    test_dataset = loaders[name]["test"].dataset
    deeplift = DeepLIFT(model)

    per_class = {}
    for class_idx, class_name in enumerate(class_names):
        mean_attr, mean_input, n = deeplift.generate_class_mean(
            class_idx, dataset=test_dataset, baseline_type="zeros",
        )
        if n == 0:
            continue
        per_class[class_idx] = {
            "class_name": class_name,
            "mean_attr": mean_attr,
            "mean_input": mean_input,
            "n": n,
        }
    print(f"{name} / {class_name}: DeepLIFT computed over {n} test samples")
    deeplift_results[name] = per_class
```

```
/home/breno/miniforge3/envs/genomics/lib/python3.10/site-packages/torch/autograd/graph.py:869: UserWarning: Attempting to run cuBLAS, but there was
no current CUDA context! Attempting to set the primary context... (Triggered internally at /pytorch/aten/src/ATen/cuda/CublasHandlePool.cpp:335.)
  return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
```

```
bcftools_chain / strong pigmentation: DeepLIFT computed over 115 test samples
bcftools_chain / weak pigmentation: DeepLIFT computed over 47 test samples
reference_realign / strong pigmentation: DeepLIFT computed over 115 test samples
reference_realign / weak pigmentation: DeepLIFT computed over 47 test samples
all_masks / strong pigmentation: DeepLIFT computed over 115 test samples
all_masks / weak pigmentation: DeepLIFT computed over 47 test samples
```

Section 5 -- Visualization of each model's mean attribution map

Reuses `analysis.run_deeplift._plot_mean_deeplift` directly (a plain-data function, no CLI-args coupling) -- one figure per (alignment method, class).

In [7]:

```
FIG_DIR = REPO_ROOT / "notebooks" / ".cache" / "genotype_cnn_alignment_deeplift"
FIG_DIR.mkdir(parents=True, exist_ok=True)

for name in configs:
    config = configs[name]
    for class_idx, payload in deeplift_results[name].items():
        top_windows = find_top_deeplift_windows(payload["mean_attr"], config, window_size=512, top_k=20)
        out_path = FIG_DIR / f"deeplift_mean_{name}_{payload['class_name'].replace(' ', '_')}.png"
        _plot_mean_deeplift(
            mean_input=payload["mean_input"],
            mean_attr=payload["mean_attr"],
            top_windows=top_windows,
            config=config,
            class_label=payload["class_name"],
            split="test",
            num_samples=payload["n"],
            output_path=out_path,
        )
    print(f"saved {out_path}")
```

```
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_bcftools_chain_strong_pigmentation.png
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_bcftools_chain_weak_pigmentation.png
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_reference_realign_strong_pigmentation.png
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_reference_realign_weak_pigmentation.png
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_all_masks_strong_pigmentation.png
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/deeplift_mean_all_masks_weak_pigmentation.png
```

Section 6 -- Attribution comparison across the three models

The one genuinely new piece of visualization in this notebook: merge each model's per-gene mean absolute attribution (`summarize_deeplift_by_track`, aggregated per gene/haplotype) and compare side by side across all three models. A gene where the models diverge a lot is a candidate for having its CNN2 decision driven either by INDEL-adjacent positions (where `bcftools_chain` and `reference_realign` genuinely disagree -- recall the pre-training smoke test: the two alignment methods matched almost exactly except at a handful of INDEL loci per gene) or by the extra explicit mask channels changing what the model attends to (`all_masks` vs. `bcftools_chain`, same alignment, different input features).

In [8]:

```
def gene_level_attribution(name, class_idx):
    config = configs[name]
    payload = deeplift_results[name][class_idx]
    track_rows = summarize_deeplift_by_track(payload["mean_attr"], config)
    df = pd.DataFrame(track_rows)
    gene_df = df.groupby(["haplotype", "gene_name"], as_index=False)["mean_abs_attr"].mean()
    gene_df["model"] = name
    return gene_df

TARGET_CLASS_IDX = class_names.index("strong pigmentation") if "strong pigmentation" in class_names else 0
MODEL_NAMES = list(configs)

comparison_df = pd.concat(
    [gene_level_attribution(name, TARGET_CLASS_IDX) for name in MODEL_NAMES],
    ignore_index=True,
)

pivot_df = comparison_df.pivot_table(
    index=["gene_name", "haplotype"], columns="model", values="mean_abs_attr"
).reset_index()
pivot_df["range"] = pivot_df[MODEL_NAMES].max(axis=1) - pivot_df[MODEL_NAMES].min(axis=1)
pivot_df = pivot_df.sort_values("range", ascending=False)
pivot_df
```

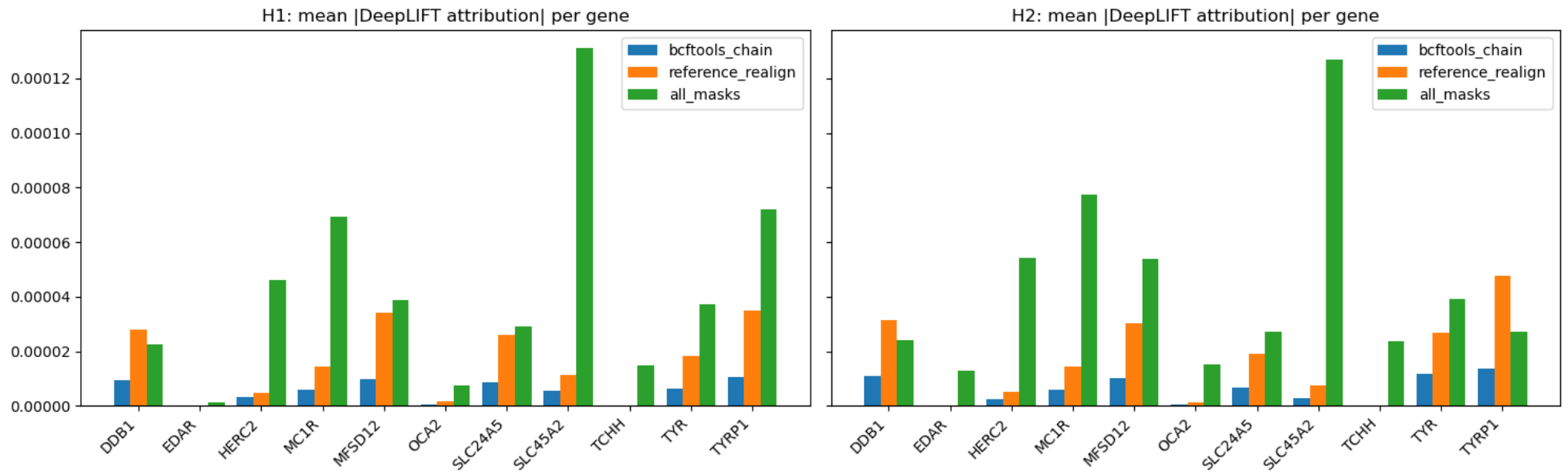
Out[8]:

model	gene_name	haplotype	all_masks	bcftools_chain	reference_realign	range
14	SLC45A2	H1	0.000131	5.523389e-06	1.148215e-05	0.000126
15	SLC45A2	H2	0.000127	2.983853e-06	7.516967e-06	0.000124
7	MC1R	H2	0.000077	5.781510e-06	1.442037e-05	0.000072
6	MC1R	H1	0.000069	5.863222e-06	1.430658e-05	0.000063
20	TYRP1	H1	0.000072	1.059086e-05	3.494780e-05	0.000061
5	HERC2	H2	0.000054	2.411434e-06	4.944187e-06	0.000052
9	MFS12	H2	0.000054	1.001218e-05	3.039536e-05	0.000044
4	HERC2	H1	0.000046	3.212988e-06	4.869104e-06	0.000043
21	TYRP1	H2	0.000027	1.366896e-05	4.768427e-05	0.000034
18	TYR	H1	0.000037	6.339295e-06	1.821053e-05	0.000031
8	MFS12	H1	0.000039	9.734000e-06	3.409645e-05	0.000029
19	TYR	H2	0.000039	1.152055e-05	2.685927e-05	0.000028
17	TCHH	H2	0.000024	3.189308e-09	9.885108e-09	0.000024
1	DDB1	H2	0.000024	1.080841e-05	3.144519e-05	0.000021
13	SLC24A5	H2	0.000027	6.604372e-06	1.913883e-05	0.000021
12	SLC24A5	H1	0.000029	8.654333e-06	2.590164e-05	0.000020
0	DDB1	H1	0.000023	9.208069e-06	2.775394e-05	0.000019
16	TCHH	H1	0.000015	5.653990e-09	2.036248e-08	0.000015
11	OCA2	H2	0.000015	4.790935e-07	1.388857e-06	0.000015
3	EDAR	H2	0.000013	1.928943e-08	2.789177e-08	0.000013
10	OCA2	H1	0.000007	3.290472e-07	1.573431e-06	0.000007
2	EDAR	H1	0.000001	1.748153e-08	2.730712e-08	0.000001

In [9]:

```
genes_order = sorted(pivot_df["gene_name"].unique())
fig, axes = plt.subplots(1, 2, figsize=(15, 5), sharey=True)
n_models = len(MODEL_NAMES)
width = 0.8 / n_models
for ax, hap in zip(axes, ["H1", "H2"]):
    sub = pivot_df[pivot_df["haplotype"] == hap].set_index("gene_name").reindex(genes_order)
    x = np.arange(len(genes_order))
    for i, name in enumerate(MODEL_NAMES):
        offset = (i - (n_models - 1) / 2) * width
        ax.bar(x + offset, sub[name], width, label=name)
    ax.set_xticks(x)
    ax.set_xticklabels(genes_order, rotation=45, ha="right")
    ax.set_title(f"{hap}: mean |DeepLIFT attribution| per gene")
    ax.legend()
fig.suptitle(f"DeepLIFT attribution by gene -- class={class_names[TARGET_CLASS_IDX]}")
fig.tight_layout()
plt.show()
```

DeepLIFT attribution by gene -- class=strong pigmentation



Section 7 -- Top attribution windows: do they shift between alignment methods?

Lists each model's highest-attribution 512bp windows and flags genes where the two alignment methods' top windows land in meaningfully different places -- most informative for INDEL-bearing loci like `HERC2 / OCA2`, where the two realignment strategies are expected to genuinely differ.

In [10]:

```
top_windows = {}
for name in configs:
    config = configs[name]
    payload = deeplift_results[name][TARGET_CLASS_IDX]
    windows = find_top_deeplift_windows(payload["mean_attr"], config, window_size=512, top_k=15)
    top_windows[name] = pd.DataFrame(windows)[["gene_name", "haplotype", "window_center", "mean_abs_attr"]]
    print(f"=== {name}: top windows, class={class_names[TARGET_CLASS_IDX]} ===")
    display(top_windows[name])
```

=== bcftools_chain: top windows, class=strong pigmentation ===

	gene_name	haplotype	window_center	mean_abs_attr
0	MFSD12	H2	17456	0.000301
1	MFSD12	H2	17455	0.000301
2	MFSD12	H2	17457	0.000301
3	MFSD12	H2	17458	0.000301
4	MFSD12	H2	17454	0.000301
5	MFSD12	H2	17459	0.000301
6	MFSD12	H2	17453	0.000301
7	MFSD12	H2	17460	0.000301
8	MFSD12	H2	17452	0.000301
9	MFSD12	H2	17461	0.000301
10	MFSD12	H2	17462	0.000301
11	MFSD12	H2	17451	0.000300
12	MFSD12	H2	17463	0.000300
13	MFSD12	H2	17464	0.000300
14	MFSD12	H2	17465	0.000300

=== reference_realign: top windows, class=strong pigmentation ===

	gene_name	haplotype	window_center	mean_abs_attr
0	MFSD12	H1	17468	0.001378
1	MFSD12	H1	17467	0.001378
2	MFSD12	H1	17466	0.001378
3	MFSD12	H1	17469	0.001378
4	MFSD12	H1	17465	0.001378
5	MFSD12	H1	17470	0.001378
6	MFSD12	H1	17464	0.001378
7	MFSD12	H1	17463	0.001377
8	MFSD12	H1	17462	0.001377
9	MFSD12	H1	17461	0.001377
10	MFSD12	H1	17460	0.001377
11	MFSD12	H1	17459	0.001377

	gene_name	haplotype	window_center	mean_abs_attr
12	MFSD12	H1	17471	0.001377
13	MFSD12	H1	17458	0.001377
14	MFSD12	H1	17457	0.001377

=== all_masks: top windows, class=strong pigmentation ===

	gene_name	haplotype	window_center	mean_abs_attr
0	SLC45A2	H1	7519	0.001146
1	SLC45A2	H1	7520	0.001145
2	SLC45A2	H1	7522	0.001144
3	SLC45A2	H1	7521	0.001143
4	SLC45A2	H1	7523	0.001143
5	SLC45A2	H1	7528	0.001142
6	SLC45A2	H1	7524	0.001142
7	SLC45A2	H1	7518	0.001142
8	SLC45A2	H1	7526	0.001142
9	SLC45A2	H1	7527	0.001142
10	SLC45A2	H1	7525	0.001142
11	SLC45A2	H1	7529	0.001141
12	SLC45A2	H1	7517	0.001139
13	SLC45A2	H1	7530	0.001139
14	SLC45A2	H1	7516	0.001138

Section 8 -- DeepLIFT attribution vs. GTF exon structure

Does the model actually weight exon positions more than intronic/UTR positions, or is it just picking up on some other pattern (e.g. coverage-level shifts unrelated to exon/intron structure)? This overlays each model's mean **signed** DeepLIFT attribution curve (signal tracks only -- mask channels are `all_masks`-specific and not comparable across models, so they're excluded here) against the gene's GENCODE v46 exon structure for the same class used in Sections 6-7 (`class_names[TARGET_CLASS_IDX]`).

Sign convention. The curve is `attrs.mean(dim=(haplotype, signal_track))` with no `.abs()` applied at any stage -- it's the same signed, cross-sample-averaged `mean_attr` tensor from Section 4 (see that section's markdown for the Rescale-rule derivation), just reduced over haplotypes/tracks instead of taking a magnitude first. A region sitting **above** the zero line consistently pushes the model's output toward `class_names[TARGET_CLASS_IDX]` ; **below** zero pushes away from it; near zero means the model's output for this class isn't sensitive to that position (either because the input there is near-baseline/zero coverage, or because the model genuinely doesn't weight it). Because sign survived the per-sample averaging in Section 4, a non-trivial value here (positive or negative) reflects a consistent effect across the class population, not per-sample noise.

Coordinate mapping and its limits. `reference_realign`'s window is built by realigning each individual directly onto true reference-genome coordinates (see `ReferenceRealignMapper._center_crop_bounds`), so position `i` in its cropped window maps exactly onto a known reference position -- no approximation. `bcftools_chain` and `all_masks` instead sit on a cohort-wide "expanded axis" that's stretched wherever any selected individual carries an INSERTION, so their window positions drift from true reference coordinates by a few bp per INDEL locus. Both alignments start from the identical reference window per gene, though, and the pre-training smoke test found the two methods match almost exactly except at a handful of INDEL-adjacent positions -- so plotting all three models on the same nominal reference-coordinate x-axis (computed once, from `reference_realign`'s exact formula) is a reasonable, if approximate for two of the three models, way to eyeball exon overlap. Treat `bcftools_chain/all_masks` exon alignment as "close" rather than "exact" -- `reference_realign` is the one model where this overlay is pixel-exact by construction.

Reuses the `mean_attr/mean_input` tensors already computed in Section 4 (no new DeepLIFT passes) and `build_haplotype_track_layout` (the same per-gene/track row map `summarize_deeplift_by_track` uses internally) to pick out just the signal-track rows for one gene, averaged over both haplotypes (but not averaged across tracks -- each of the 3 ontology terms x 2 strands gets its own pair of panels, since collapsing them together would hide track-specific patterns), then lightly smoothed (centered moving average, matching the `window_size=512` granularity Section 7 already uses) for legibility -- raw per-base values are too noisy to eyeball directly.

Each track gets two stacked panels: the actual (normalized, log1p-transformed) RNA-seq signal fed to the model on top, and its signed DeepLIFT attribution directly below -- so it's possible to see, for example, whether an attribution peak lines up with real coverage or with a near-zero input region (relevant to the "AlphaGenome's RNA-seq input is itself exon-biased" caveat discussed earlier: intronic positions have little real coverage to begin with, so DeepLIFT has little to attribute there regardless of what the model "chooses" to weight).

Exon overlay only on the sense strand. A gene's exons are annotated on one specific strand (from GENCODE); overlaying them on the antisense-strand panels too would imply an exon/intron structure that doesn't exist for that strand's transcription. So the green exon shading (and the exon-

location row at the bottom of each column) is drawn only under the column matching the gene's own annotated strand -- the other column is left unshaded.

In [11]:

```
import requests

from alphagenome.data import genome, gene_annotation
from alphagenome.data import transcript as transcript_utils

GTF_CACHE_DIR = REPO_ROOT / "notebooks" / ".cache" / "annotations"
GTF_URL = (
    "https://storage.googleapis.com/alphagenome/reference/gencode/"
    "hg38/gencode.v46.annotation.gtf.gz.feather"
)

def load_gtf(cache_dir: Path) -> pd.DataFrame:
    # Same cache location/URL as notebooks/utils/annotations.py's load_gtf, so this reuses
    # that notebook's ~tens-of-MB download if it already ran -- reimplemented inline (rather
    # than importing notebooks.utils) to keep this notebook's only dependency on the rest of
    # the repo scoped to genomics.*, matching every other cell here.
    cache_dir.mkdir(parents=True, exist_ok=True)
    cache_path = cache_dir / "gencode.v46.annotation.gtf.gz.feather"
    if not cache_path.exists():
        r = requests.get(GTF_URL, timeout=120)
        r.raise_for_status()
        cache_path.write_bytes(r.content)
    return pd.read_feather(cache_path)

gtf = load_gtf(GTF_CACHE_DIR)
gtf_manes = gene_annotation.filter_to_manes_select_transcript(gene_annotation.filter_protein_coding(gtf))
gtf_protein_coding = gene_annotation.filter_protein_coding(gtf)
transcript_extractor_manes = transcript_utils.TranscriptExtractor(gtf_manes)
transcript_extractor_coding = transcript_utils.TranscriptExtractor(gtf_protein_coding)
gene_id_map = gtf.drop_duplicates("gene_name").set_index("gene_name")["gene_id"]
print(f"GTF loaded: {len(gtf):,} rows -- {gtf_manes['gene_name'].nunique():,} genes with a MANE Select transcript")
```

GTF loaded: 3,467,156 rows -- 19,226 genes with a MANE Select transcript

In [12]:

```
def window_genomic_axis(gene: str, window_center_size: int):
    '''Reference-coordinate window start/length, using the identical centering formula
    ReferenceRealignMapper.center_crop_bounds applies when building reference_realign's
    training tensors -- see Section 8's markdown for why this axis is exact for that model
    and approximate for bcftools_chain/all_masks.'''
    meta_path = DATASET_DIR / "references" / "windows" / gene / "window_metadata.json"
    meta = json.loads(meta_path.read_text())
    chrom = meta["chromosome"]
    start_lbased = int(meta["start"])
    end_lbased = int(meta["end"])
    full_ref_length = end_lbased - start_lbased + 1

    size = min(window_center_size, full_ref_length)
    center_offset = full_ref_length // 2
    crop_start = max(0, center_offset - size // 2)
    crop_end = min(full_ref_length, crop_start + size)
    if crop_end - crop_start < size:
        crop_start = max(0, crop_end - size)

    window_start_0based = (start_lbased - 1) + crop_start # alphagenome genome.Interval is 0-based half-open
    local_length = crop_end - crop_start
    return chrom, window_start_0based, local_length

def gene_exon_local_boxes(gene: str, chrom: str, window_start_0based: int, local_length: int):
    interval = genome.Interval(chrom, window_start_0based, window_start_0based + local_length)
    target_gene_id = gene_id_map.get(gene)

    transcripts = transcript_extractor_mane.extract(interval)
    matched = [t for t in transcripts if t.gene_id == target_gene_id] if target_gene_id else transcripts
    if not matched:
        # Fallback for genes without a MANE Select tag in this GENCODE release: take the
        # protein-coding transcript with the most exons for this gene within the window.
        transcripts = [t for t in transcript_extractor_coding.extract(interval) if t.gene_id == target_gene_id]
        matched = [max(transcripts, key=lambda t: len(t.exons))] if transcripts else []

    boxes, strand = [], None
    for t in matched:
        strand = t.strand
        for exon in t.exons:
            s = max(0, exon.start - window_start_0based)
            e = min(local_length, exon.end - window_start_0based)
            if e > s:
                boxes.append((s, e))
    return sorted(boxes), strand
```

```

def gene_track_curves(tensor: torch.Tensor, config, gene: str) -> dict:
    '''Per-(ontology_curie, strand) curve for one gene from a (haplotype, row, L)-shaped tensor
    (either Section 4's signed mean_attr or its mean_input), averaged over both haplotypes only
    -- {(ontology_curie, strand): 1D curve}, same length as the window. Used both for the signed
    DeepLIFT attribution (no .abs() -- positive = pushes toward the target class, see Section 8
    markdown) and for the raw model-input RNA-seq signal itself.'''
    layout = build_haplotype_track_layout(config)
    rows = [r for r in layout if r["gene_name"] == gene and r["track_type"] == "signal"]
    t = tensor.detach().cpu()
    if t.ndim == 2:
        t = t.unsqueeze(0)
    return {
        (r["ontology_curie"], r["strand"]): t[:, r["row_index"], :].mean(dim=0).numpy()
        for r in rows
    }

def smooth(x: np.ndarray, window: int = 513) -> np.ndarray:
    if window <= 1:
        return x
    kernel = np.ones(window) / window
    return np.convolve(x, kernel, mode="same")

DATASET_DIR = Path(configs["bcftools_chain"].dataset_input.dataset_dir)
MODEL_COLORS = {"bcftools_chain": "tab:blue", "reference_realign": "tab:orange", "all_masks": "tab:red"}
WINDOW_CENTER_SIZES = {name: config.dataset_input.window_center_size for name, config in configs.items()}
assert len(set(WINDOW_CENTER_SIZES.values())) == 1, f"window_center_size differs across models: {WINDOW_CENTER_SIZES}"
print(f"window_center_size shared across all models: {next(iter(WINDOW_CENTER_SIZES.values()))}")

```

window_center_size shared across all models: 32768

In [13]:

```
EXON_FIG_DIR = REPO_ROOT / "notebooks" / ".cache" / "genotype_cnn_alignment_deeplift"
EXON_FIG_DIR.mkdir(parents=True, exist_ok=True)

ONTOLOGY_TERMS_KO = list(configs["bcftools_chain"].dataset_input.ontology_terms)
STRANDS = ["+", "-"]
N_ONT = len(ONTOLOGY_TERMS_KO)

N_ROWS = 2 * N_ONT + 1 # (RNA-seq signal, DeepLIFT attribution) pair per ontology, + exon row

for gene in configs["bcftools_chain"].dataset_input.genes_to_use:
    window_center_size = WINDOW_CENTER_SIZES["bcftools_chain"]
    chrom, window_start_0based, local_length = window_genomic_axis(gene, window_center_size)
    exon_boxes, gene_strand = gene_exon_local_boxes(gene, chrom, window_start_0based, local_length)

    attr_curves_by_model = {
        name: gene_track_curves(deeplift_results[name][TARGET_CLASS_IDX]["mean_attr"], configs[name], gene)
        for name in MODEL_NAMES
    }
    input_curves_by_model = {
        name: gene_track_curves(deeplift_results[name][TARGET_CLASS_IDX]["mean_input"], configs[name], gene)
        for name in MODEL_NAMES
    }

    fig, axes = plt.subplots(
        N_ROWS, 2, figsize=(13, 1.5 * N_ROWS + 1.0), sharex=True,
        gridspec_kw={"height_ratios": [2, 2] * N_ONT + [1]},
    )
    for row in range(N_ROWS - 1):
        axes[row, 0].sharey(axes[row, 1])

    for col, strand in enumerate(STRANDS):
        is_sense = strand == gene_strand
        for i, ontology in enumerate(ONTOLOGY_TERMS_KO):
            signal_row, attr_row = 2 * i, 2 * i + 1

            ax_sig = axes[signal_row, col]
            for name in MODEL_NAMES:
                curve = input_curves_by_model[name].get((ontology, strand))
                if curve is not None:
                    ax_sig.plot(smooth(curve), label=name, color=MODEL_COLORS[name], linewidth=1.0)
            if is_sense:
                for s, e in exon_boxes:
                    ax_sig.axvspan(s, e, color="tab:green", alpha=0.10, zorder=0)
            if col == 0:
                ax_sig.set_ylabel(f"{ontology}\nRNA-seq signal\n(normalized)", fontsize=7)
```

```

if signal_row == 0:
    suffix = " (sense)" if is_sense else " (antisense)"
    ax_sig.set_title(f"{strand} strand{suffix}", fontsize=9)
    if col == 0:
        ax_sig.legend(fontsize=6.5, loc="upper right")

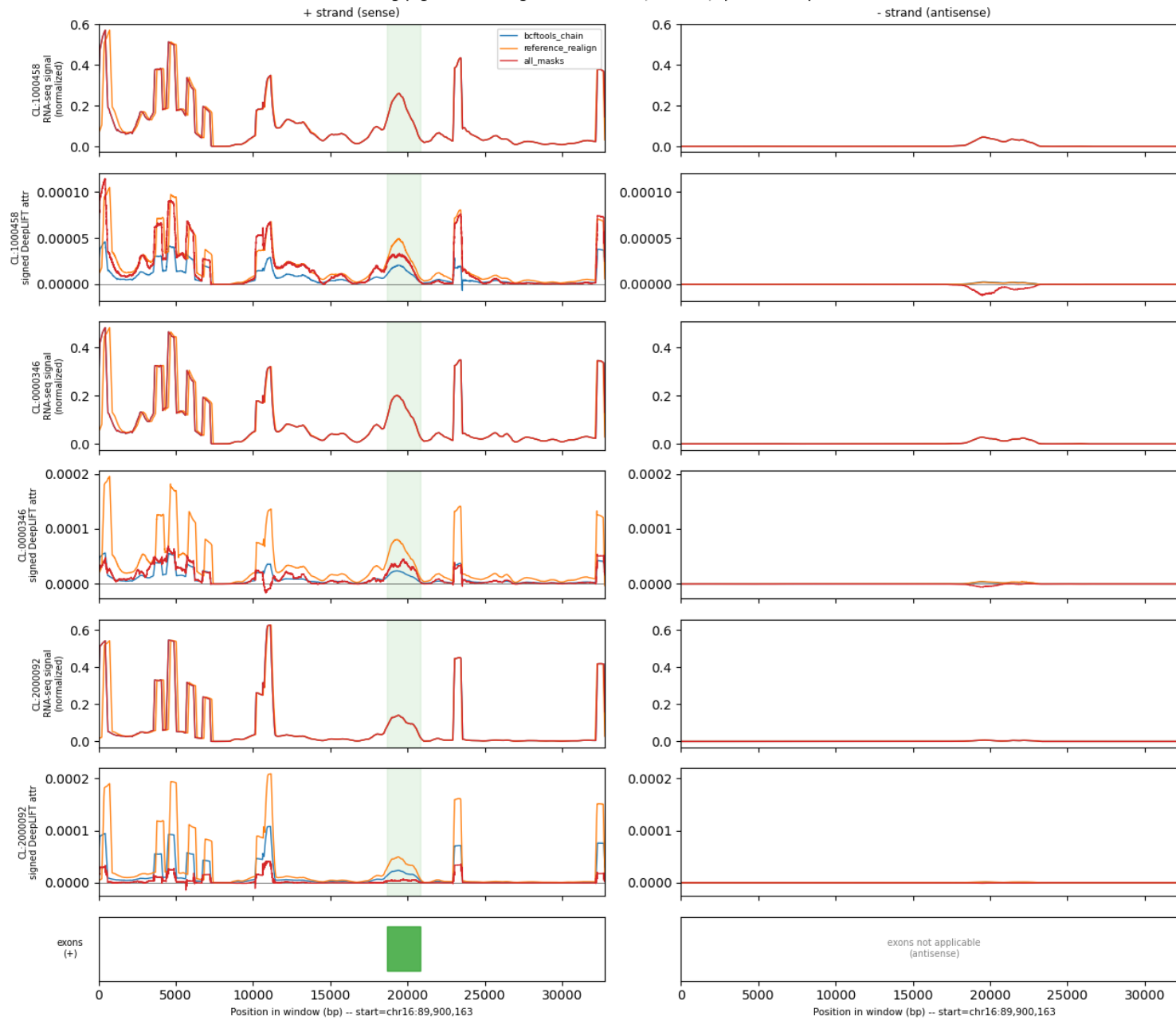
ax_attr = axes[attr_row, col]
for name in MODEL_NAMES:
    curve = attr_curves_by_model[name].get((ontology, strand))
    if curve is not None:
        ax_attr.plot(smooth(curve), color=MODEL_COLORS[name], linewidth=1.0)
ax_attr.axhline(0, color="black", linewidth=0.7, alpha=0.5, zorder=1)
if is_sense:
    for s, e in exon_boxes:
        ax_attr.axvspan(s, e, color="tab:green", alpha=0.10, zorder=0)
if col == 0:
    ax_attr.set_ylabel(f"{ontology}\nsigned DeepLIFT attr", fontsize=7)

ax_exon = axes[N_ROWS - 1, col]
if is_sense:
    for s, e in exon_boxes:
        ax_exon.add_patch(plt.Rectangle((s, 0.15), e - s, 0.7, color="tab:green", alpha=0.8))
    ax_exon.set_ylabel(f"exons\n({strand})", fontsize=7, rotation=0, labelpad=22, va="center")
else:
    ax_exon.text(0.5, 0.5, "exons not applicable\n(antisense)", transform=ax_exon.transAxes,
                ha="center", va="center", fontsize=7, color="gray")
ax_exon.set_xlim(0, local_length)
ax_exon.set_ylim(0, 1)
ax_exon.set_yticks([])
ax_exon.set_xlabel(f"Position in window (bp) -- start={chrom}:{window_start_0based + 1:,}", fontsize=7.5)

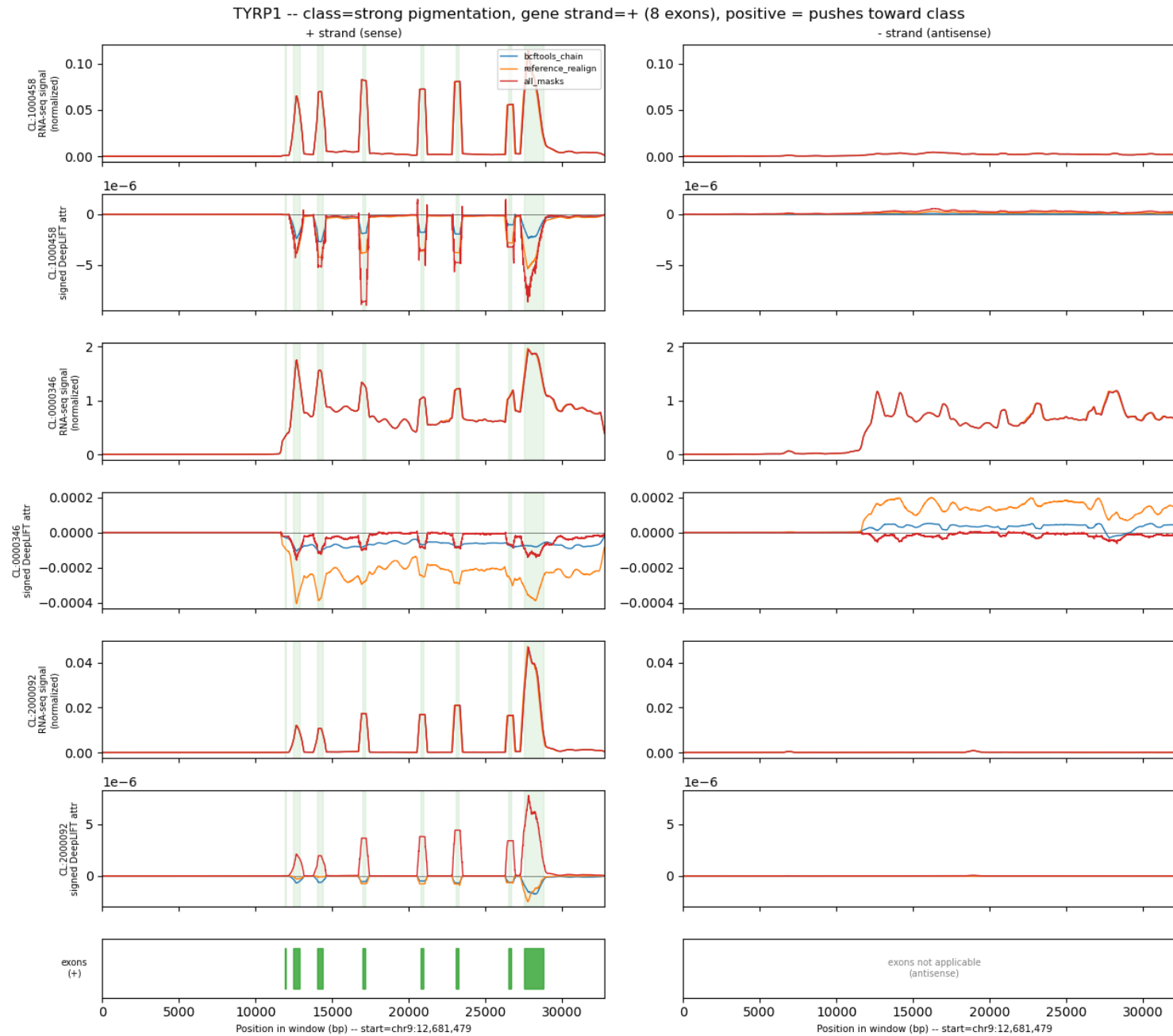
fig.suptitle(
    f"{gene} -- class={class_names[TARGET_CLASS_IDX]}, gene strand={gene_strand} "
    f"({len(exon_boxes)} exons), positive = pushes toward class"
)
fig.tight_layout()
out_path = EXON_FIG_DIR / f"exon_overlap_{gene}.png"
fig.savefig(out_path, dpi=140)
plt.show()
plt.close(fig)
print(f"saved {out_path}")

```

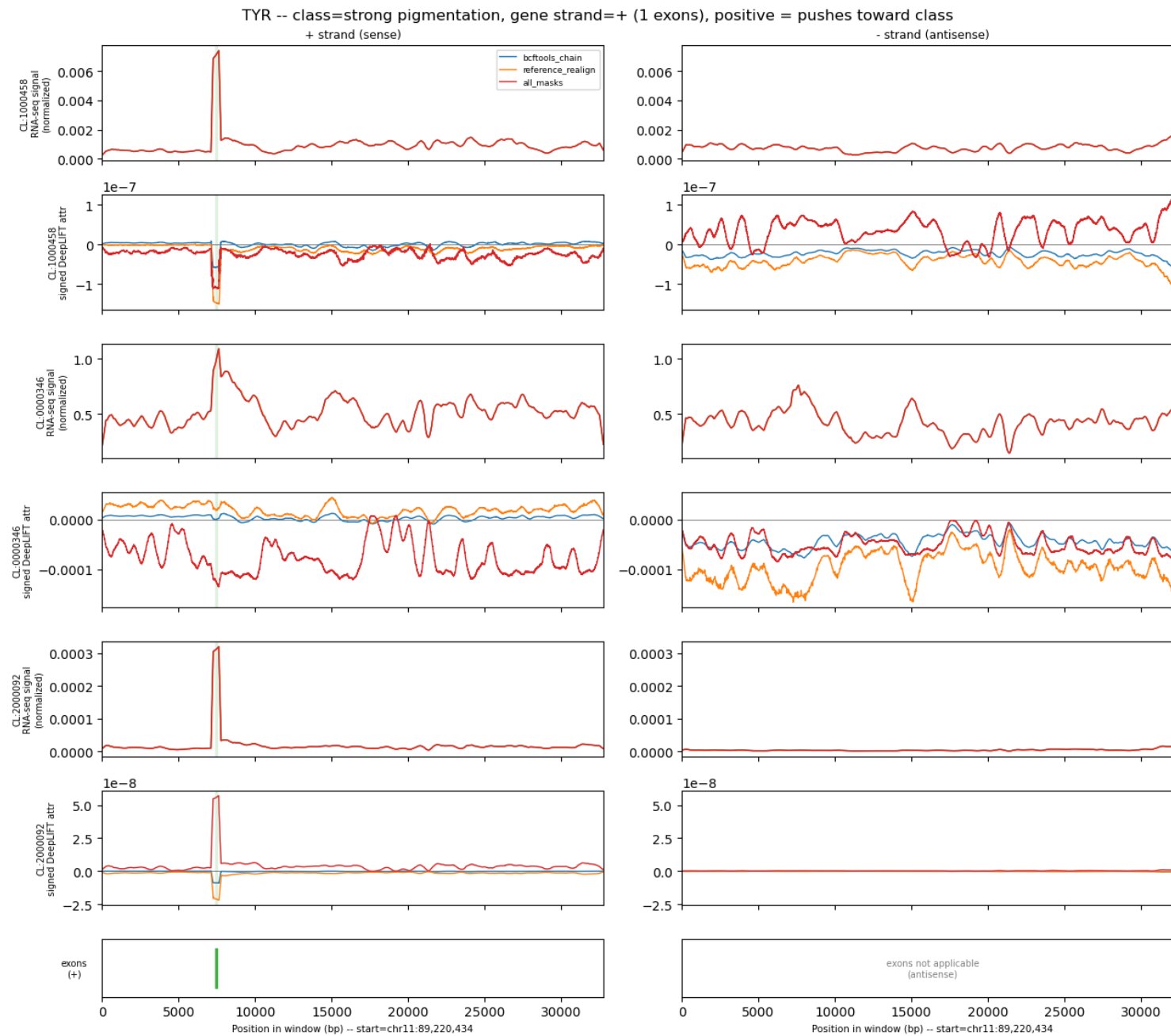
MC1R -- class=strong pigmentation, gene strand=+ (1 exons), positive = pushes toward class



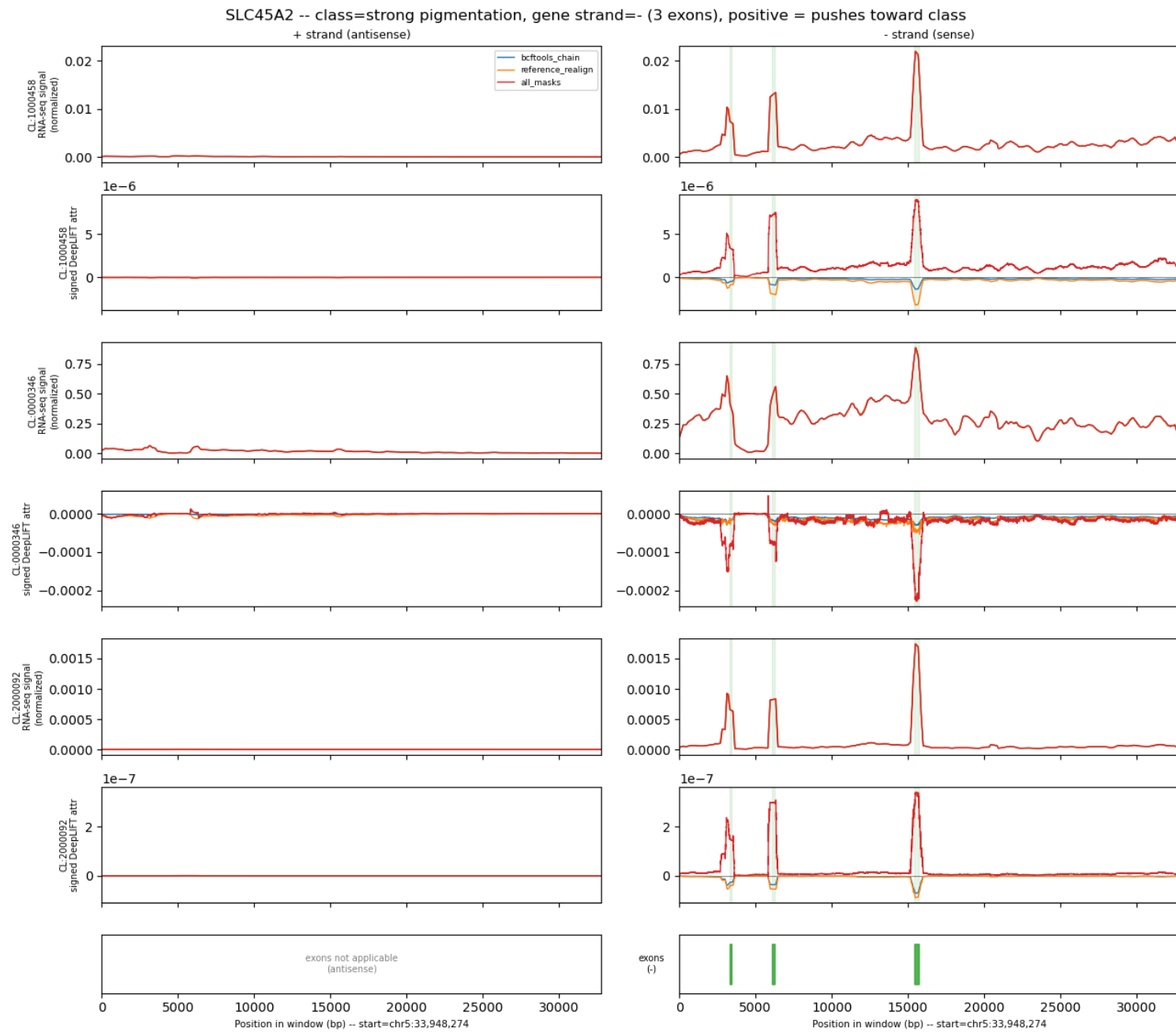
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_MC1R.png



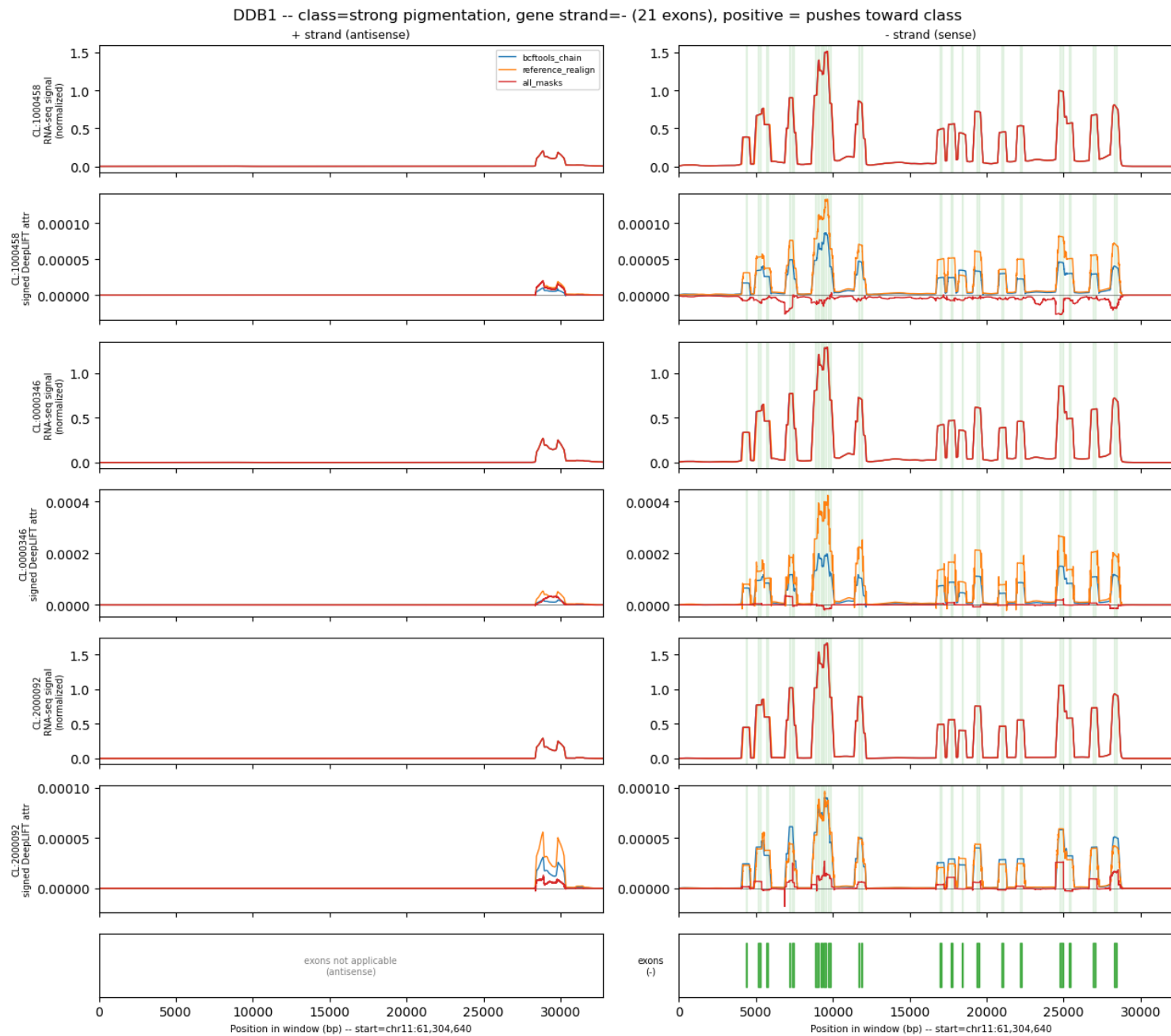
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_TYRP1.png



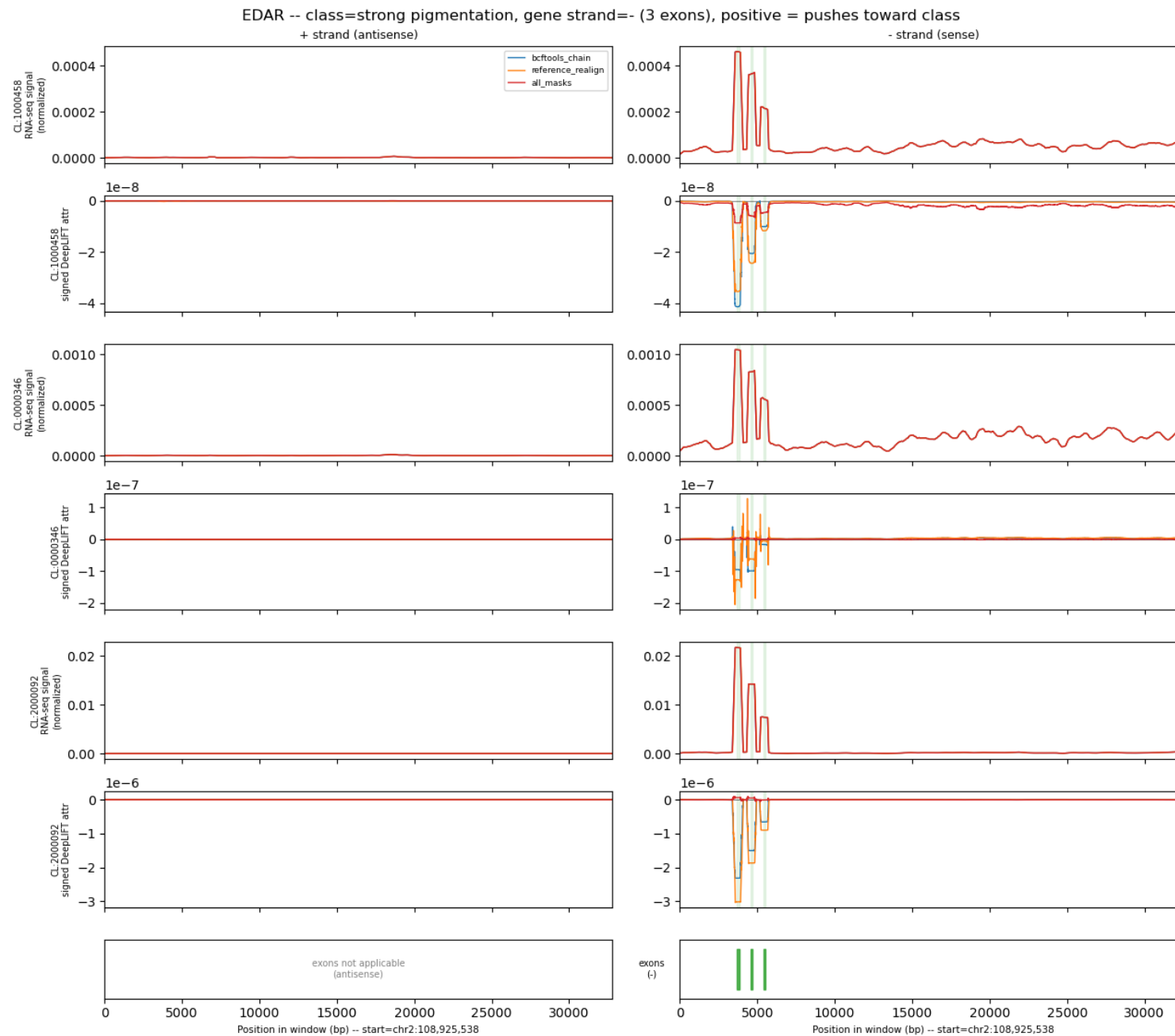
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_TYR.png



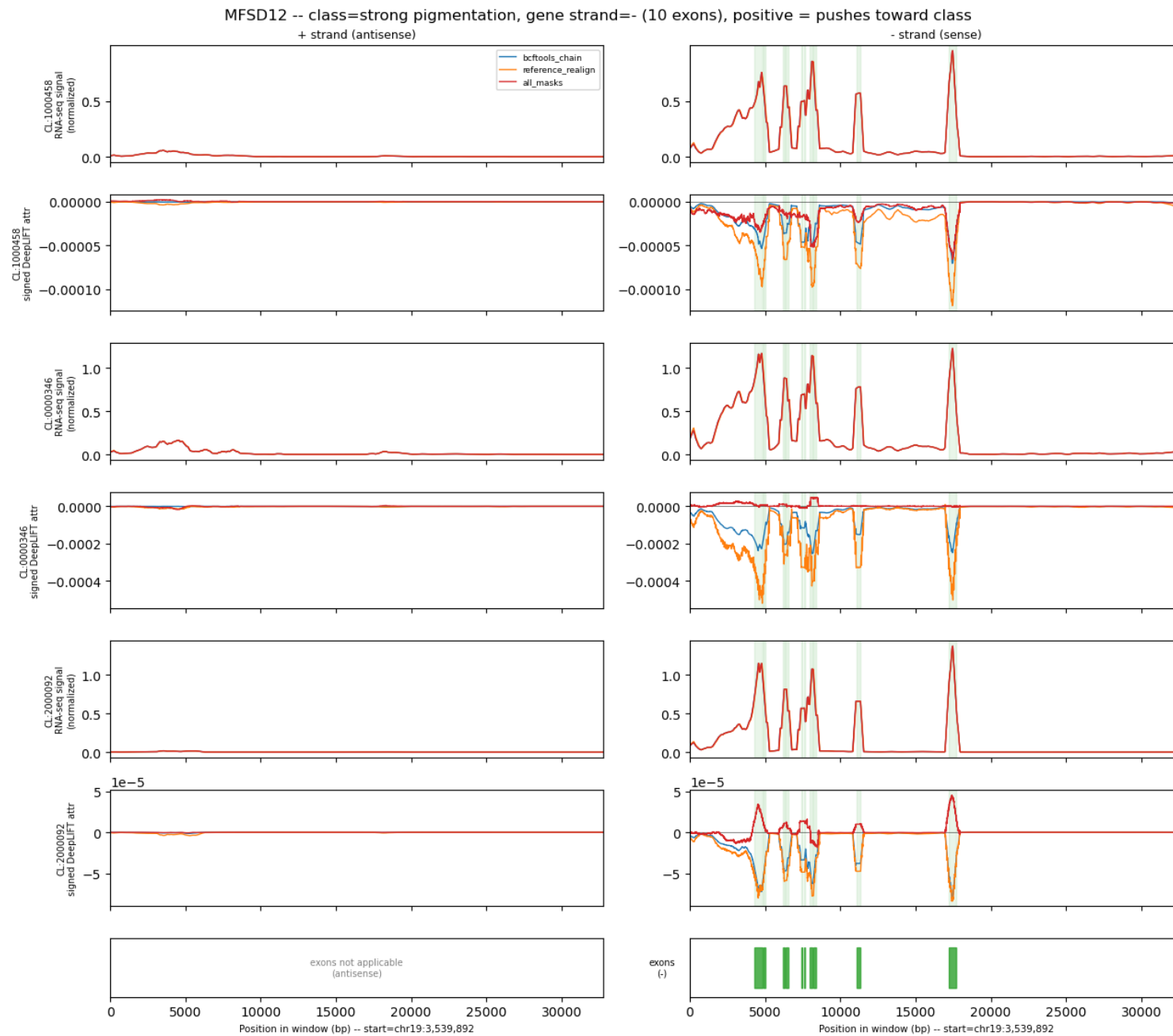
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_SLC45A2.png



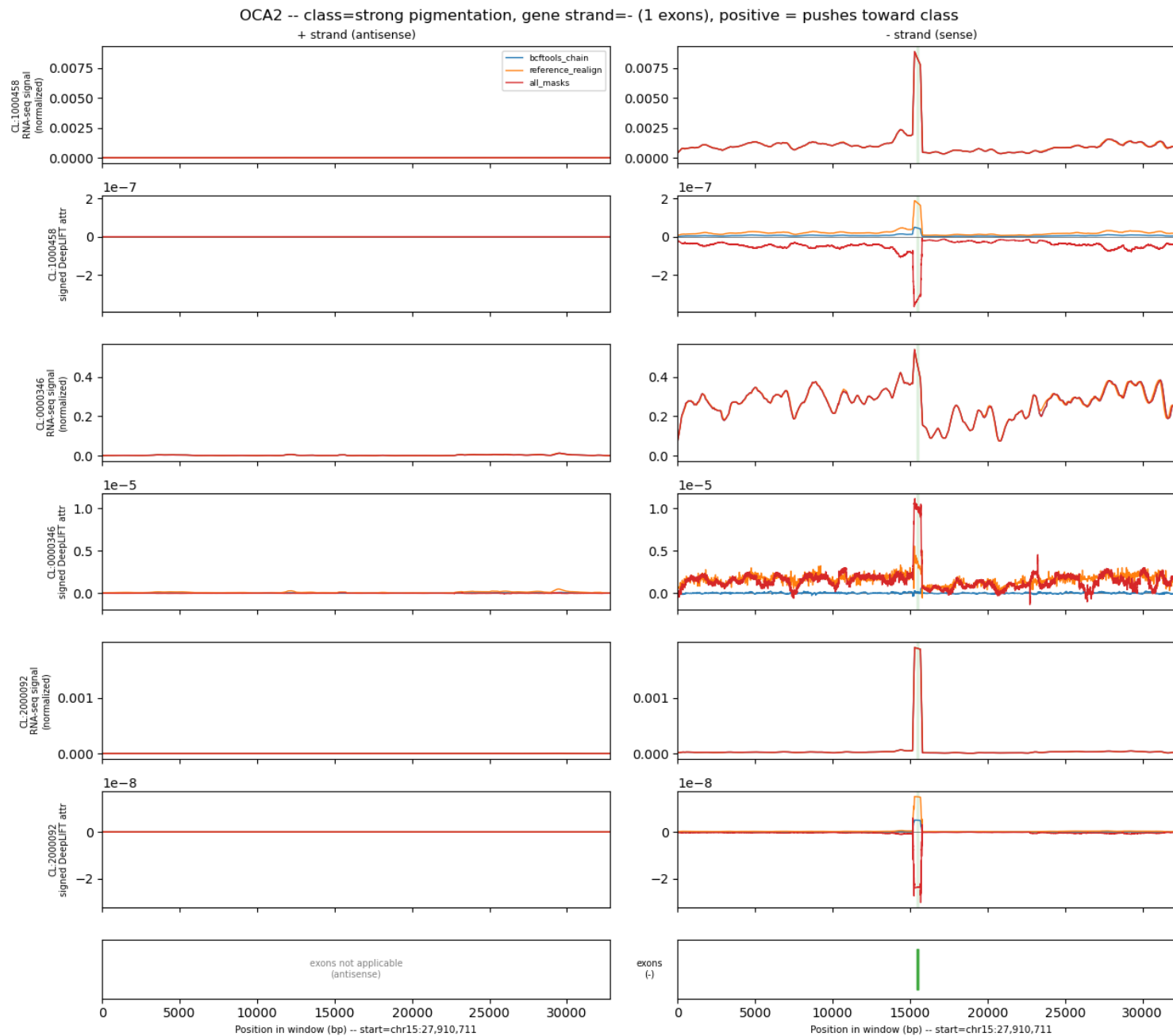
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_DDB1.png



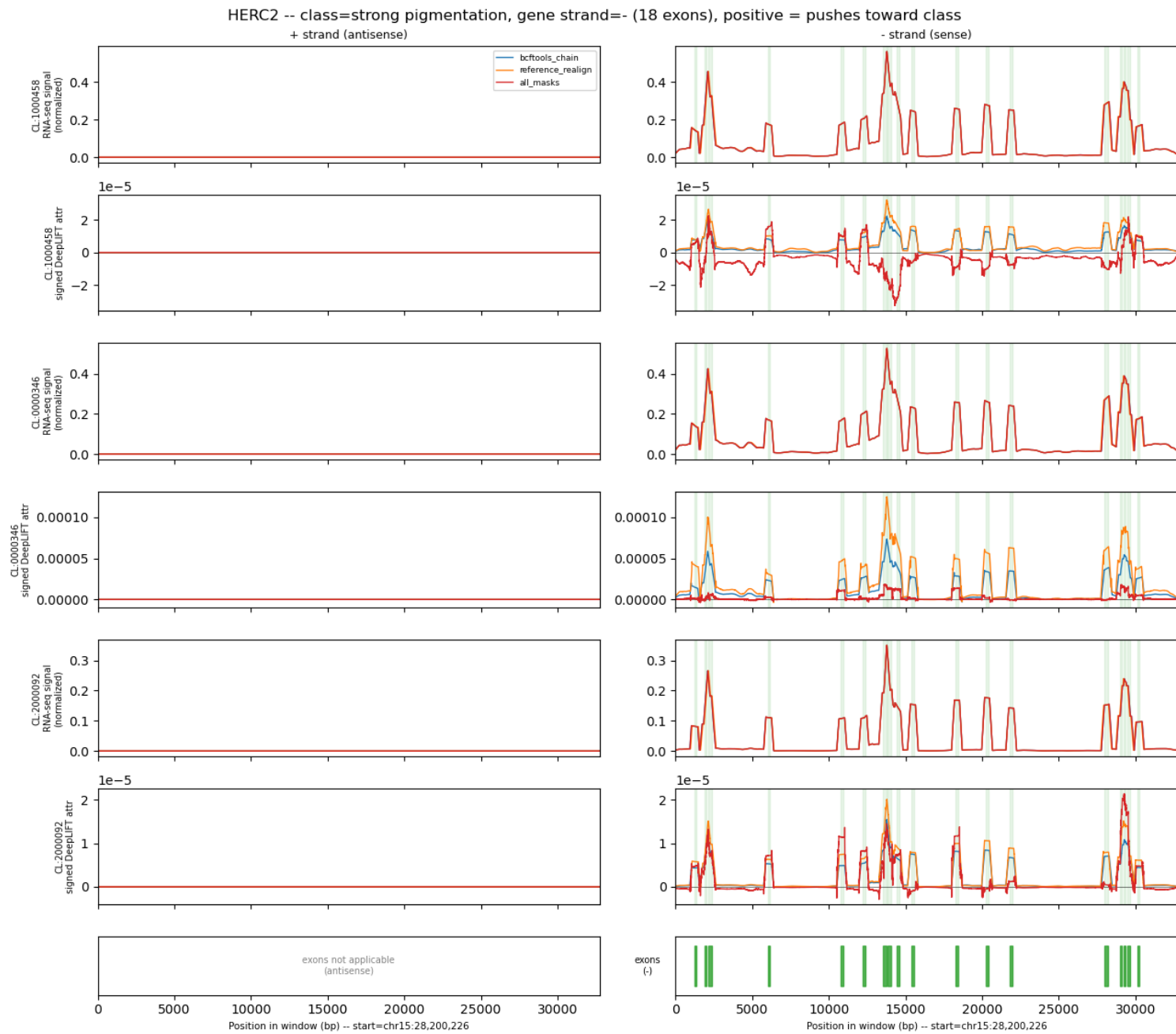
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_EDAR.png



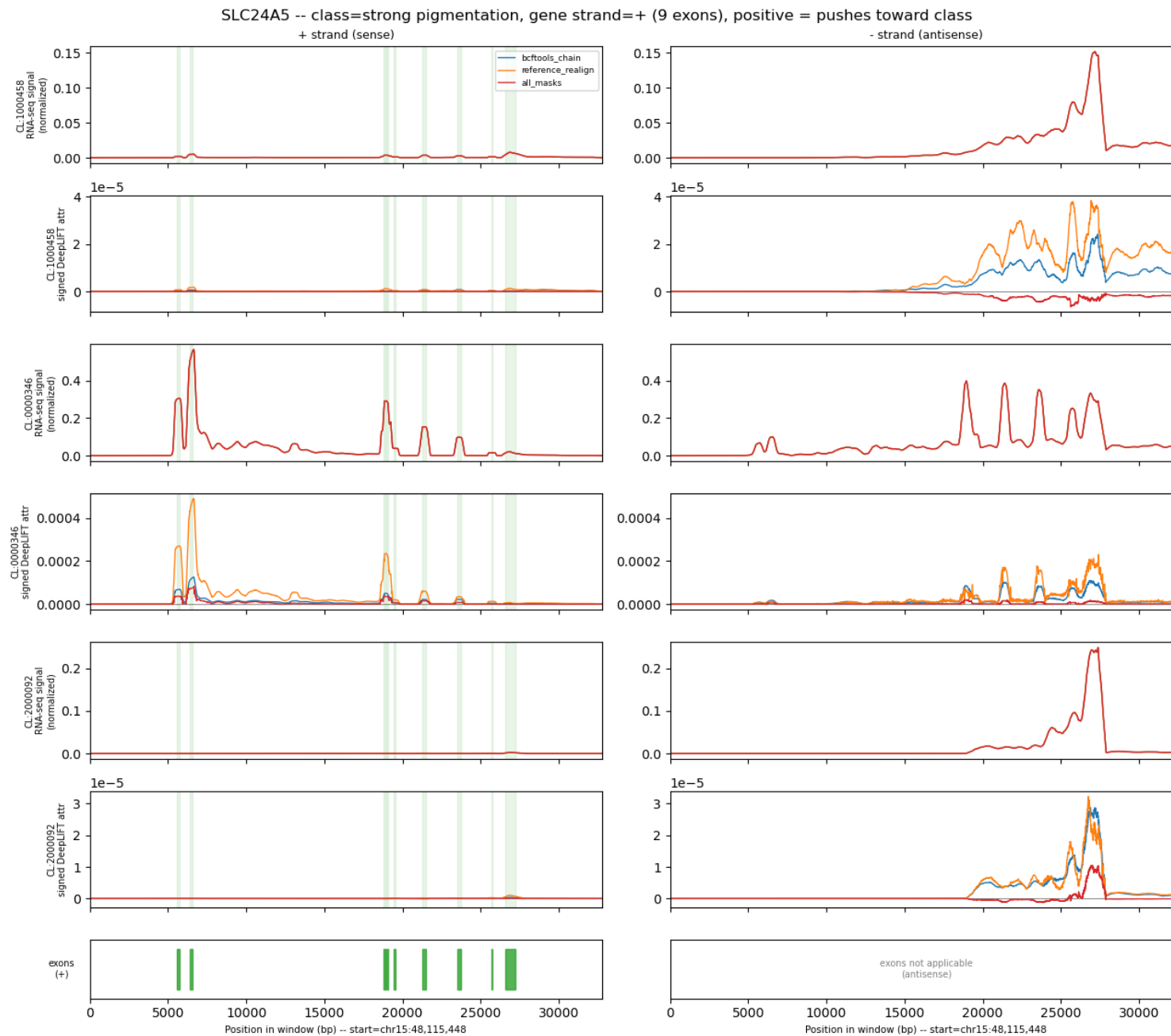
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_MFSD12.png



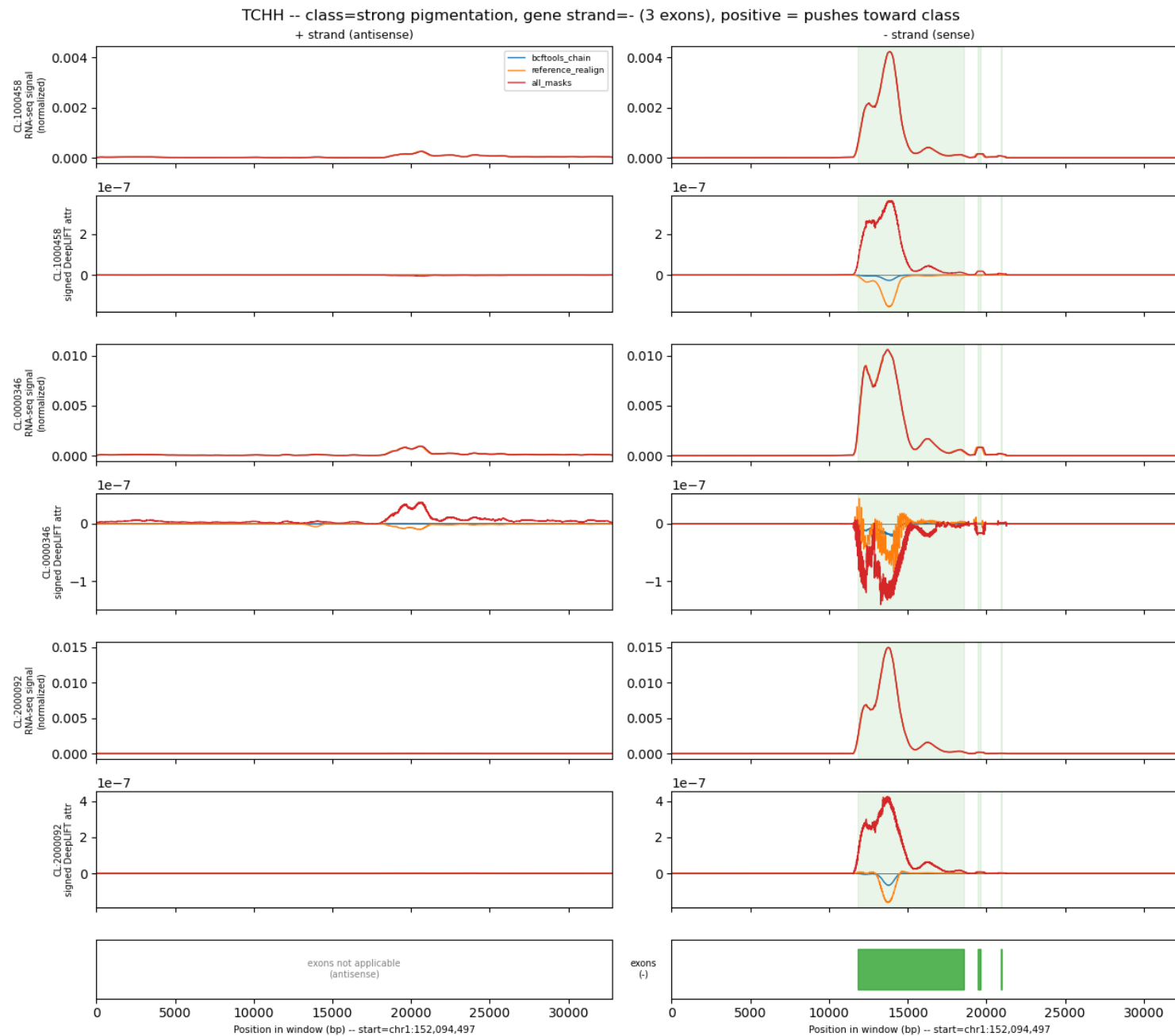
saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_OCA2.png



saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_HERC2.png



saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_SLC24A5.png



saved /home/breno/I2CA/genomics/notebooks/.cache/genotype_cnn_alignment_deeplift/exon_overlap_TCHH.png

Section 9 -- In-silico promoter/enhancer knockout: does perturbing regulation change the prediction?

Sections 4-8 asked where the trained CNN2 attends within each gene. This section asks a more causal question for one real individual: if we actually disrupt a gene's promoter/enhancer with a single-nucleotide edit -- something a base-editing CRISPR experiment could plausibly do in vitro/in vivo -- and let **AlphaGenome itself re-predict** the resulting RNA-seq track (not just edit the track by hand), does the CNN's "strong pigmentation" logit move the way we'd expect?

Scope. Only the `bcftools_chain` model is used here (the original run) -- extending this to all three models would triple the AlphaGenome API cost for a question that isn't about alignment method. All other genes/haplotypes keep this individual's real, unmodified predictions; only the targeted gene's block is replaced.

Three ways to locate the promoter/enhancer:

1. **Biology-database path** -- the gene's GENCODE v46 MANE Select transcription start site (`alphagenome.data.gene_annotation.extract_tss`), i.e. where a molecular biologist would look first (`get_gene_tss` below).
2. **AlphaGenome-functional-signal path (ATAC)** -- an unfiltered-ontology ATAC prediction across the full 524,288 bp reference window, taking the position of peak accessibility across any of AlphaGenome's ~167 profiled biosamples within 5 kb of the TSS (`find_atac_summit` below).
Ontology caveat (already established empirically in `pigmentation_alphagenome_cnn_variant_scoring.ipynb`, Section 3 there): AlphaGenome's ENCODE-derived catalog has **zero** ATAC/DNase/CAGE/ChIP tracks for the pigmentation config's 3 melanocyte/hair-follicle ontology terms -- only RNA_SEQ/splice tracks exist for those. So the accessibility signal used here is a general "is this regulatory in any known cell type" proxy, not melanocyte-specific. Treat it as a plausible candidate location, not a melanocyte-validated one.
3. **AlphaGenome-functional-signal path (CAGE, melanocyte-specific)** -- CAGE (Cap Analysis of Gene Expression) directly measures active transcription initiation, arguably a sharper promoter-finding signal than ATAC's general accessibility proxy. Unlike ATAC/DNase/ChIP, AlphaGenome does profile CAGE for two melanocyte subtypes directly relevant to pigmentation: `CL:0002566` (dark melanocyte) and `CL:0002567` (light melanocyte) -- so this path is melanocyte-specific, not a generic proxy (`find_cage_promoter` below).

The knockout itself. (Revised from an earlier single-base transversion -- that turned out to be too weak a perturbation for most genes: single-nucleotide sensitivity in AlphaGenome is sparse, and a generic TSS/ATAC/CAGE-derived position isn't guaranteed to be the one specific base that matters.) Instead, a `window_size`-bp window (default 100 bp) centered on the located position is **scrambled** -- the existing bases are shuffled in place (`apply_scramble` below), destroying any motif/positional structure in that window (known or unknown) while preserving its exact base composition, so an observed effect isn't just a GC-content artifact. This is a much closer analog to how promoter/enhancer knockouts are actually done experimentally (CRISPR deletion of the core element), while keeping the sequence's total length fixed, as `predict_sequence` requires. Applied **homozygously** (same scramble on both H1 and H2), matching the "both copies carry the variant" convention already used in `pigmentation_alphagenome_cnn_variant_scoring.ipynb`, to simulate a full loss-of-function rather than a heterozygous partial effect.

Mechanics. For the target gene+haplotype: load the individual's own real consensus FASTA (<sample>.<hap>.window.fixed.fa -- the exact sequence AlphaGenome originally scored to produce the cached predictions_H{1,2}/rna_seq.npz this individual's real input tensor was built from), scramble the window, call `client.predict_sequence(...)` on the edited sequence, then feed the resulting track through the same `ProcessedGenomicDataset._process_window_haplotype_channels` method training used (so the individual's own real bcftools_chain alignment entry is reused unchanged -- a pure SNV doesn't shift any downstream coordinate) to get a properly aligned, normalized block, substituted into that individual's otherwise-real input tensor.

In [14]:

```
from dotenv import load_dotenv
from alphagenome.models import dna_client

load_dotenv(Path.home() / ".env")
API_KEY = os.environ.get("ALPHAGENOME_API_KEY")
if not API_KEY:
    raise RuntimeError("ALPHAGENOME_API_KEY not found in the environment or ~/.env.")

ag_client = dna_client.create(api_key=API_KEY)
ORGANISM = dna_client.Organism.HOMO_SAPIENS
KO_CONFIG = configs["bcftools_chain"]
KO_GENES = list(KO_CONFIG.dataset_input.genes_to_use)
KO_ONTOLOGY_TERMS = list(KO_CONFIG.dataset_input.ontology_terms)
KO_MODEL = models["bcftools_chain"]
print("AlphaGenome client ready. Section 9 uses the bcftools_chain model only.")
```

```
/home/breno/miniforge3/envs/genomics/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

AlphaGenome client ready. Section 9 uses the bcftools_chain model only.

In [15]:

```
from genomics.predictors.genotype_based.data.pipeline import (
    _make_runtime_processed_datasets, _resolve_runtime_dataset_dir, _load_split_index,
)
from genomics.predictors.genotype_based.config import get_dataset_cache_dir
from genomics.predictors.genotype_based.data.normalization import apply_normalization

KO_CACHE_DIR = get_dataset_cache_dir(KO_CONFIG)
ko_runtime_dataset_dir = _resolve_runtime_dataset_dir(KO_CONFIG)
# A *real* ProcessedGenomicDataset (not the CachedProcessedDataset Section 2 loaded), needed for
# its low-level _process_window_haplotype_channels + bcftools_chain_mapper/dynamic_indel_aligner
# -- Section 2's dataset is a tensor-cache reader with no per-gene builder methods.
ko_full_ds, _ko_train_ds, _ko_val_ds, _ko_test_ds = _make_runtime_processed_datasets(
    ko_runtime_dataset_dir, KO_CACHE_DIR, KO_CONFIG,
)
ko_normalization_params = ko_full_ds.normalization_params
ko_idx_to_target = ko_full_ds.idx_to_target
ko_strong_idx = [i for i, name in ko_idx_to_target.items() if name == "strong pigmentation"][0]
print(f"idx_to_target={ko_idx_to_target}, strong_idx={ko_strong_idx}")
```

```
/home/breno/miniforge3/envs/genomics/lib/python3.10/site-packages/rich/live.py:260: UserWarning: install
"ipywidgets" for Jupyter support
  warnings.warn('install "ipywidgets" for Jupyter support')
```

[INFO] GenomicLongevityDataset inicializado:

- Dataset: non_longevous_dataset_genes
- Indivíduos: 3202
- Load predictions: True
- Load sequences: False
- Cache sequences: False

```
idx_to_target={0: 'strong pigmentation', 1: 'weak pigmentation'}, strong_idx=0
```

Select the individual

INDIVIDUAL_ID is a plain variable -- change it to analyze a different person. Restricted to the **test split** (consistent with every other section) and to samples whose population maps to "strong pigmentation" in the config's class_map (YRI / ESN / LWK / MSL / GWD).

In [16]:

```
ko_split_index = _load_split_index(KO_CACHE_DIR)
ko_test_sample_ids = ko_split_index["test"]
ko_pedigree = ko_full_ds.dataset_metadata.get("individuals_pedigree", {})
STRONG_PIGMENTATION_POPS = {"YRI", "ESN", "LWK", "MSL", "GWD"}
strong_pigmentation_test_ids = sorted(
    sid for sid in ko_test_sample_ids
    if ko_pedigree.get(sid, {}).get("population") in STRONG_PIGMENTATION_POPS
)
print(f"{len(strong_pigmentation_test_ids)} 'strong pigmentation' individuals in the test split")

INDIVIDUAL_ID = strong_pigmentation_test_ids[0] # <-- change this to analyze a different individual
print(f"INDIVIDUAL_ID = {INDIVIDUAL_ID!r} (population={ko_pedigree[INDIVIDUAL_ID]['population']!r})")
```

```
115 'strong pigmentation' individuals in the test split
INDIVIDUAL_ID = 'HG02461' (population='GWD')
```

Path 1 -- biology-database promoter location (GENCODE TSS)

Reuses Section 8's already-loaded `gtfmane/gtfprotein_coding`.

In [17]:

```
tss_df_mane = gene_annotation.extract_tss(gtf_mane)
tss_df_coding = gene_annotation.extract_tss(gtf_protein_coding)

def get_gene_tss(gene: str):
    '''Returns (chrom, tss_pos_0based, strand) from the MANE Select transcript, falling back to
    any protein-coding transcript's TSS if no MANE Select tag exists for this gene.'''
    rows = tss_df_mane[tss_df_mane["gene_name"] == gene]
    if rows.empty:
        rows = tss_df_coding[tss_df_coding["gene_name"] == gene]
    if rows.empty:
        raise ValueError(f"No TSS found for gene {gene!r}")
    row = rows.iloc[0]
    return row["Chromosome"], int(row["Start"]), row["Strand"]

for _g in KO_GENES:
    _chrom, _tss, _strand = get_gene_tss(_g)
    print(f"{_g:>8}: {_chrom}:{_tss} ({_strand})")
```

```
MC1R: chr16:89918861 (+)
TYRP1: chr9:12693384 (+)
TYR: chr11:89177874 (+)
SLC45A2: chr5:33984693 (-)
DDB1: chr11:61333105 (-)
EDAR: chr2:108989220 (-)
MFSD12: chr19:3557586 (-)
OCA2: chr15:28099315 (-)
HERC2: chr15:28322179 (-)
SLC24A5: chr15:48120989 (+)
TCHH: chr1:152115444 (-)
```

Path 2 -- AlphaGenome-functional-signal promoter/enhancer location (ATAC)

One `predict_interval` call per gene (reference genome, not individual-specific -- the location of a regulatory element is treated as roughly individual-invariant; only the later knockout prediction uses the individual's own sequence), cached to disk since it's the same call regardless of which individual gets analyzed.

In [18]:

```
ATAC_SUMMIT_CACHE_DIR = REPO_ROOT / "notebooks" / ".cache" / "promoter_knockout_predictions"
ATAC_SUMMIT_CACHE_DIR.mkdir(parents=True, exist_ok=True)

def find_atac_summit(gene: str, search_radius: int = 5000) -> dict:
    json_cache_path = ATAC_SUMMIT_CACHE_DIR / f"atac_summit_{gene}.json"
    curve_cache_path = ATAC_SUMMIT_CACHE_DIR / f"atac_summit_{gene}_curve.npy"
    if json_cache_path.exists() and curve_cache_path.exists():
        result = json.loads(json_cache_path.read_text())
        result["max_per_position"] = np.load(curve_cache_path)
        return result

    window_meta = json.loads((DATASET_DIR / "references" / "windows" / gene / "window_metadata.json").read_text())
    chrom = window_meta["chromosome"]
    start_lbased = int(window_meta["start"])
    full_ref_length = int(window_meta["end"]) - start_lbased + 1
    assert full_ref_length == dna_client.SEQUENCE_LENGTH_500KB

    _, tss_pos_0based, _ = get_gene_tss(gene)
    tss_local_idx = tss_pos_0based - (start_lbased - 1)

    interval = genome.Interval(chromosome=chrom, start=start_lbased - 1, end=start_lbased - 1 + full_ref_length)
    atac_output = ag_client.predict_interval(
        interval, organism=ORGANISM, requested_outputs=[dna_client.OutputType.ATAC], ontology_terms=None,
    )
    atac_values = atac_output.atac.values # (524288, n_biosamples), all biosamples (ontology unfiltered)
    lo = max(0, tss_local_idx - search_radius)
    hi = min(full_ref_length, tss_local_idx + search_radius)
    # Max across all biosamples at each position -- the same reduction used to pick the summit,
    # cached here (search-window only, not the full 524,288 bp) so Section 9's location-comparison
    # plot can reuse it without a second AlphaGenome call.
    window_max_per_pos = atac_values[lo:hi, :].max(axis=1)
    summit_local_idx = lo + int(np.argmax(window_max_per_pos))

    result = {
        "gene": gene, "chrom": chrom, "start_lbased": start_lbased,
        "tss_local_idx": tss_local_idx, "summit_local_idx": summit_local_idx,
        "search_lo": lo, "search_hi": hi,
        "peak_value": float(window_max_per_pos.max()),
    }
    json_cache_path.write_text(json.dumps(result))
    np.save(curve_cache_path, window_max_per_pos)
    result["max_per_position"] = window_max_per_pos
    return result
```

```
for _g in KO_GENES:
    _r = find_atac_summit(_g)
    print(f"{_g:>8}: ATAC summit at offset {_r['summit_local_idx'] - _r['tss_local_idx']:+d} bp from TSS "
          f"(peak={_r['peak_value']:.2f})")
```

```
MC1R: ATAC summit at offset +4359 bp from TSS (peak=70.50)
TYRP1: ATAC summit at offset +2322 bp from TSS (peak=6.28)
TYR: ATAC summit at offset -1787 bp from TSS (peak=5.16)
SLC45A2: ATAC summit at offset -4476 bp from TSS (peak=6.00)
DDB1: ATAC summit at offset -5 bp from TSS (peak=91.50)
EDAR: ATAC summit at offset +57 bp from TSS (peak=24.88)
MFSD12: ATAC summit at offset +175 bp from TSS (peak=165.00)
OCA2: ATAC summit at offset -3077 bp from TSS (peak=41.00)
HERC2: ATAC summit at offset -2608 bp from TSS (peak=2.31)
SLC24A5: ATAC summit at offset -1511 bp from TSS (peak=5.91)
TCHH: ATAC summit at offset -4897 bp from TSS (peak=9.44)
```

Location comparison -- where did each method place the promoter/enhancer?

For every gene: the max-across-biosamples ATAC accessibility curve (the same reduction `find_atac_summit` uses to pick its summit) around the TSS, with both candidate knockout positions marked.

In [19]:

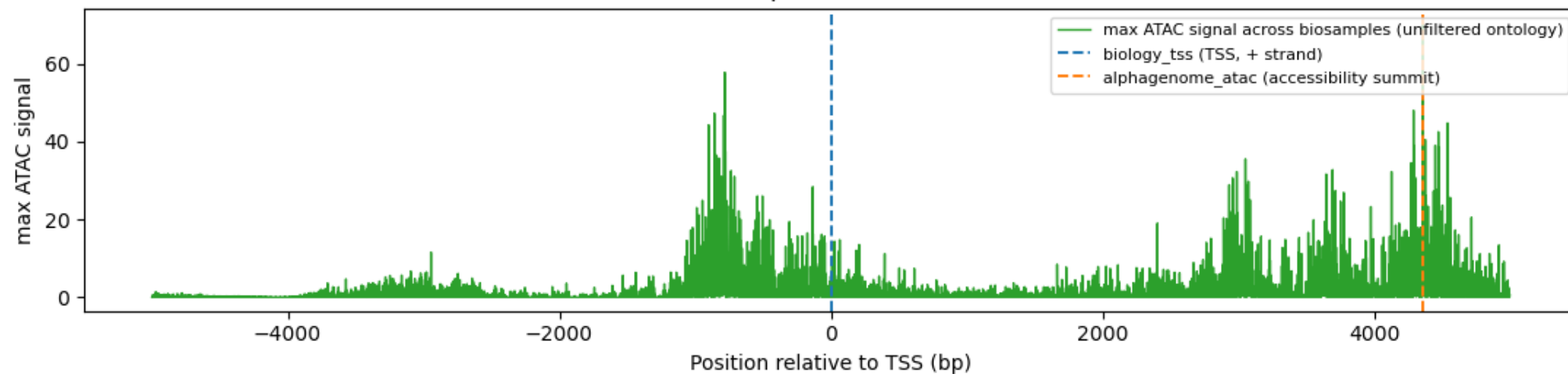
```
KNOCKOUT_FIG_DIR = REPO_ROOT / "notebooks" / ".cache" / "genotype_cnn_alignment_deeplift"
KNOCKOUT_FIG_DIR.mkdir(parents=True, exist_ok=True)

def plot_location_methods(gene: str):
    _, tss_pos_0based, strand = get_gene_tss(gene)
    summit = find_atac_summit(gene)
    tss_local_idx = summit["tss_local_idx"]
    summit_local_idx = summit["summit_local_idx"]
    curve = summit["max_per_position"]
    x = np.arange(summit["search_lo"], summit["search_hi"]) - tss_local_idx

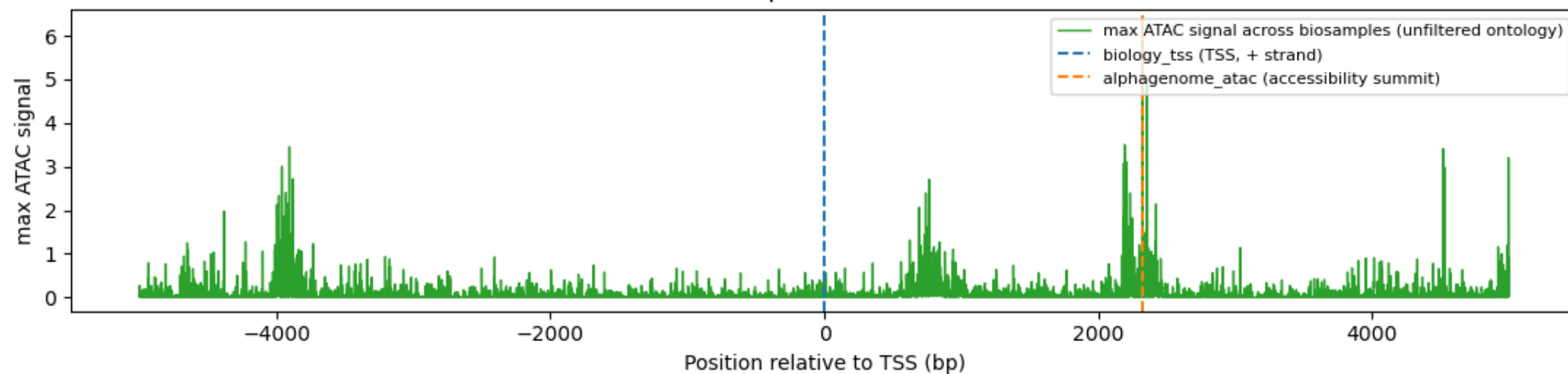
    fig, ax = plt.subplots(figsize=(11, 3))
    ax.plot(x, curve, color="tab:green", linewidth=1.0, label="max ATAC signal across biosamples (unfiltered ontology)")
    ax.axvline(0, color="tab:blue", linestyle="--", linewidth=1.4, label=f"biology_tss (TSS, {strand} strand)")
    ax.axvline(summit_local_idx - tss_local_idx, color="tab:orange", linestyle="--", linewidth=1.4,
               label="alphagenome_atac (accessibility summit)")
    ax.set_xlabel("Position relative to TSS (bp)")
    ax.set_ylabel("max ATAC signal")
    ax.set_title(f"{gene} -- candidate promoter/enhancer locations")
    ax.legend(fontsize=8, loc="upper right")
    fig.tight_layout()
    out_path = KNOCKOUT_FIG_DIR / f"location_comparison_{gene}.png"
    KNOCKOUT_FIG_DIR.mkdir(parents=True, exist_ok=True)
    fig.savefig(out_path, dpi=140)
    plt.show()
    plt.close(fig)

for _g in KO_GENES:
    plot_location_methods(_g)
```

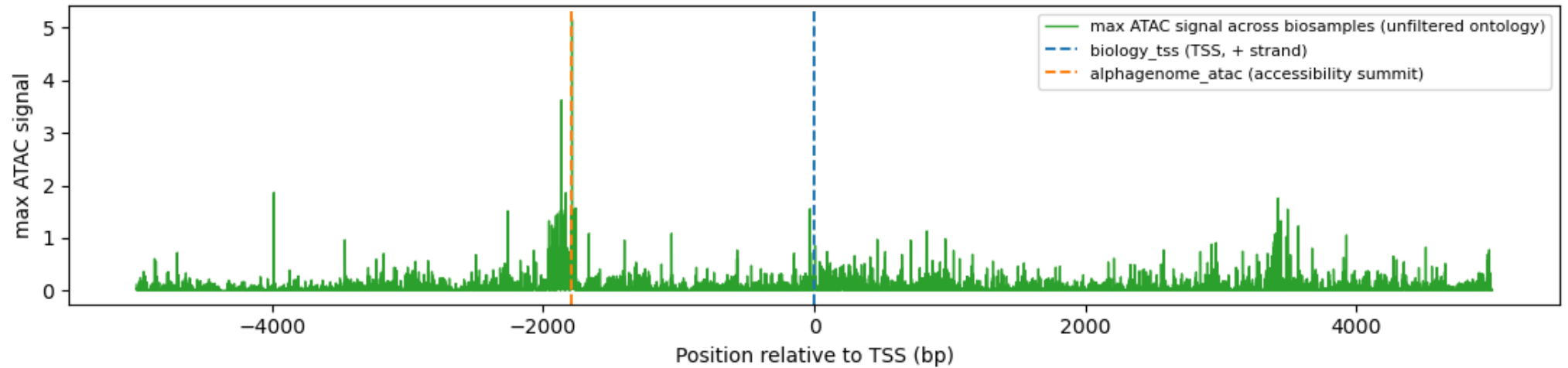
MC1R -- candidate promoter/enhancer locations



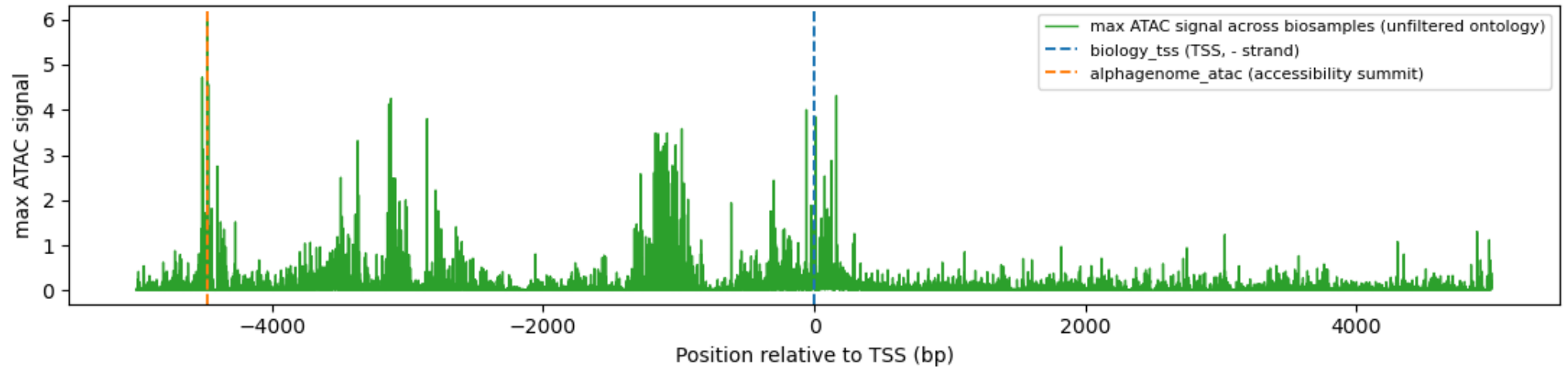
TYRP1 -- candidate promoter/enhancer locations



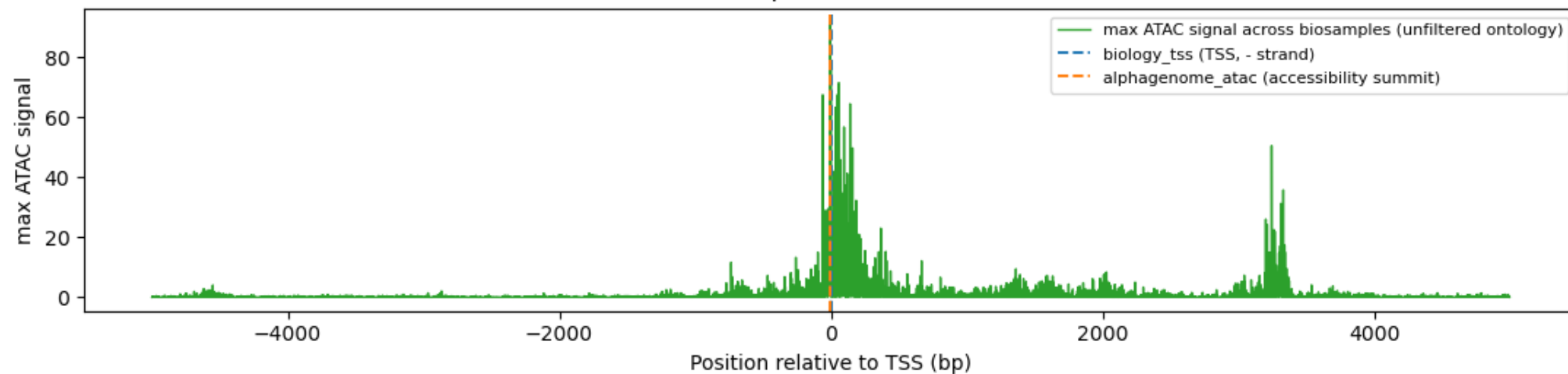
TYR -- candidate promoter/enhancer locations



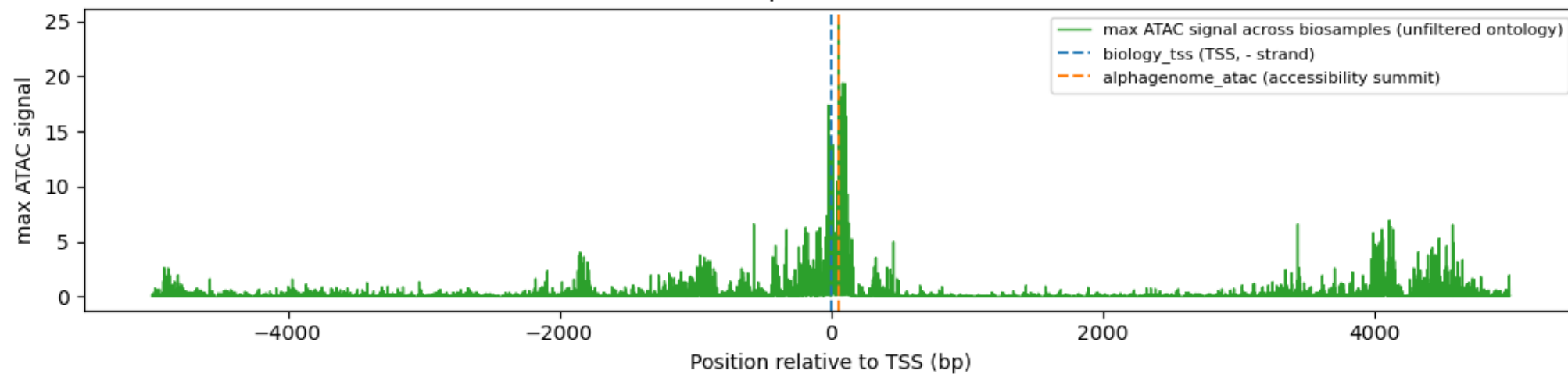
SLC45A2 -- candidate promoter/enhancer locations



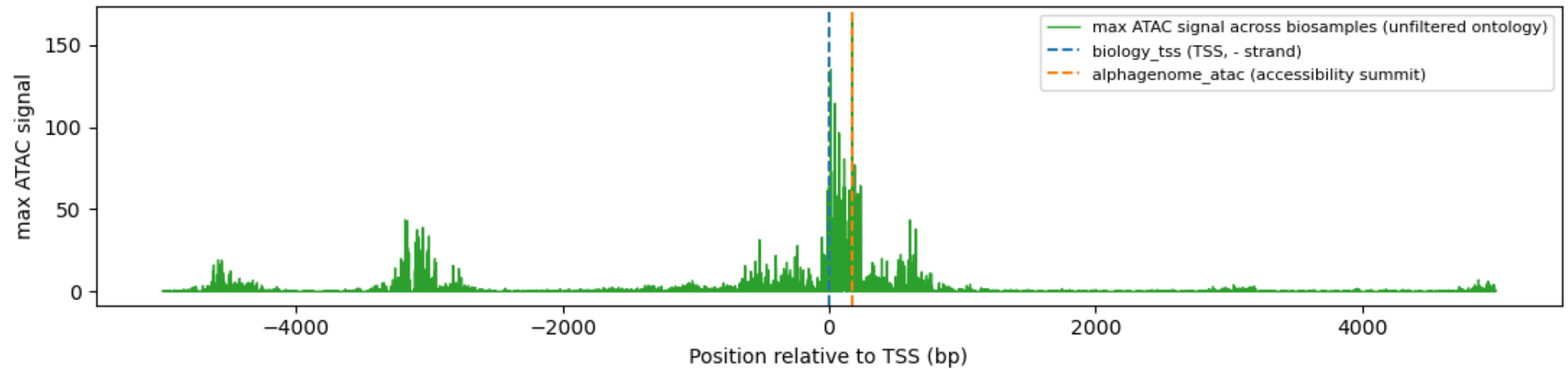
DDB1 -- candidate promoter/enhancer locations



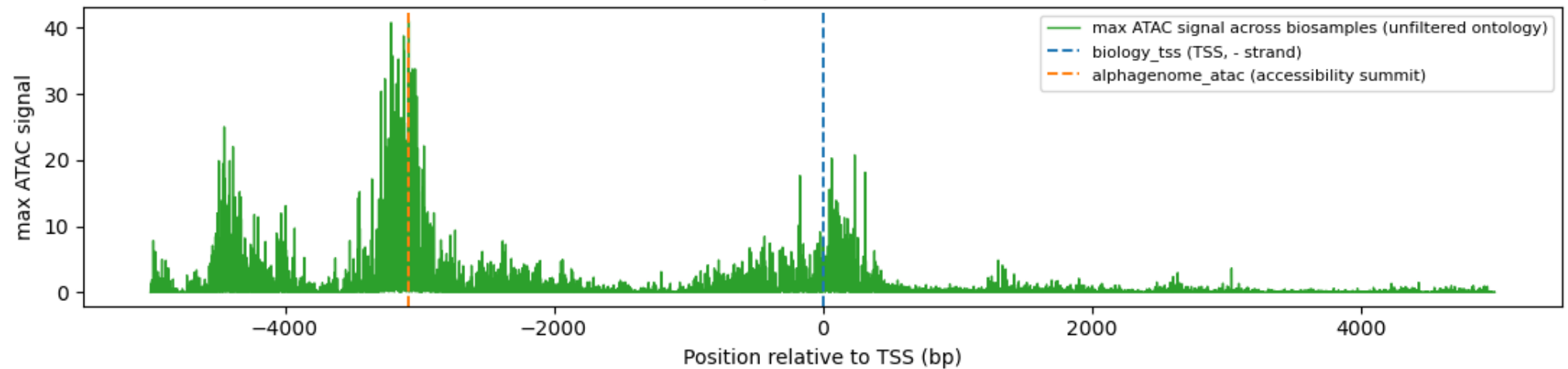
EDAR -- candidate promoter/enhancer locations



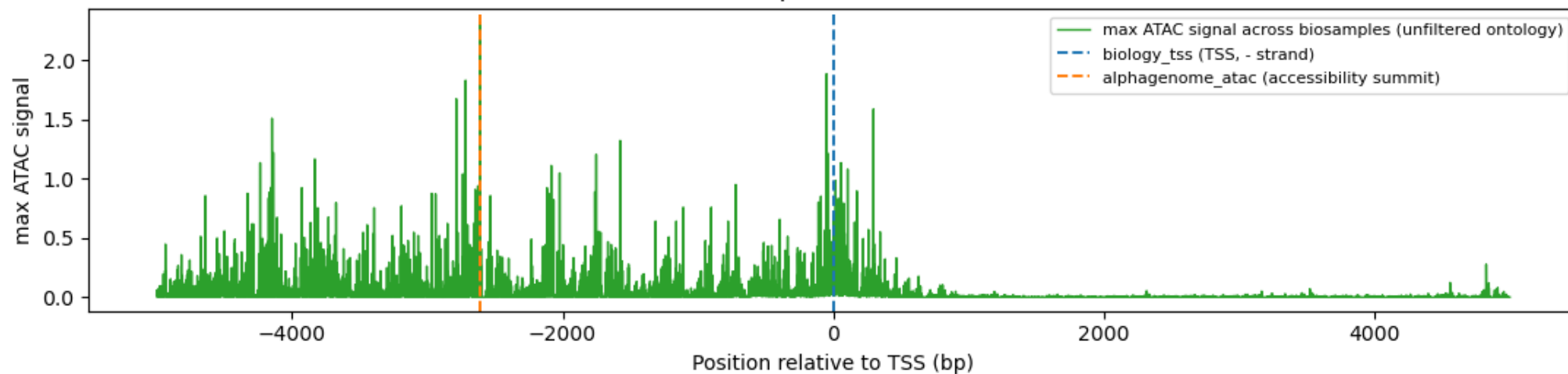
MFSD12 -- candidate promoter/enhancer locations



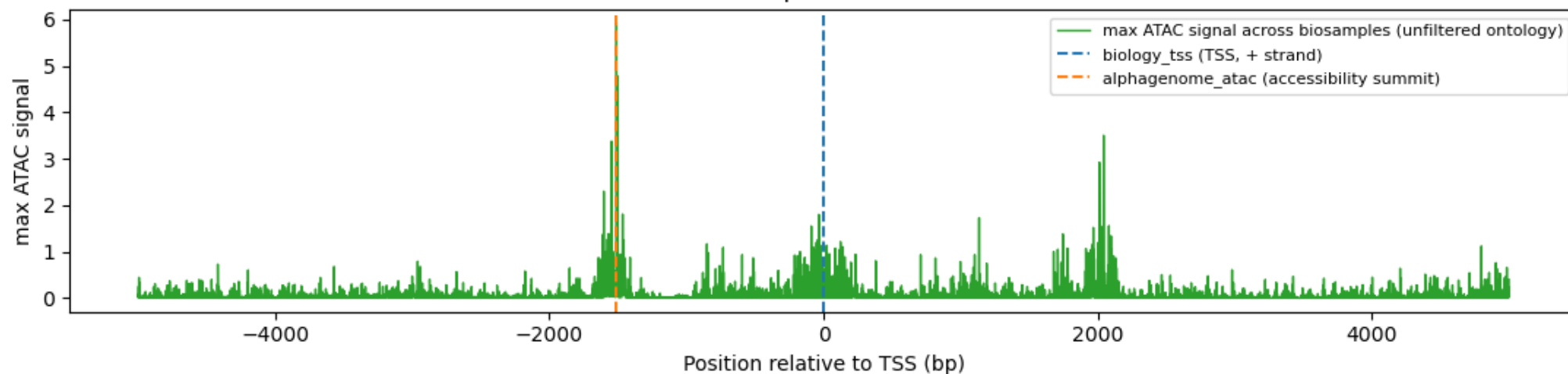
OCA2 -- candidate promoter/enhancer locations

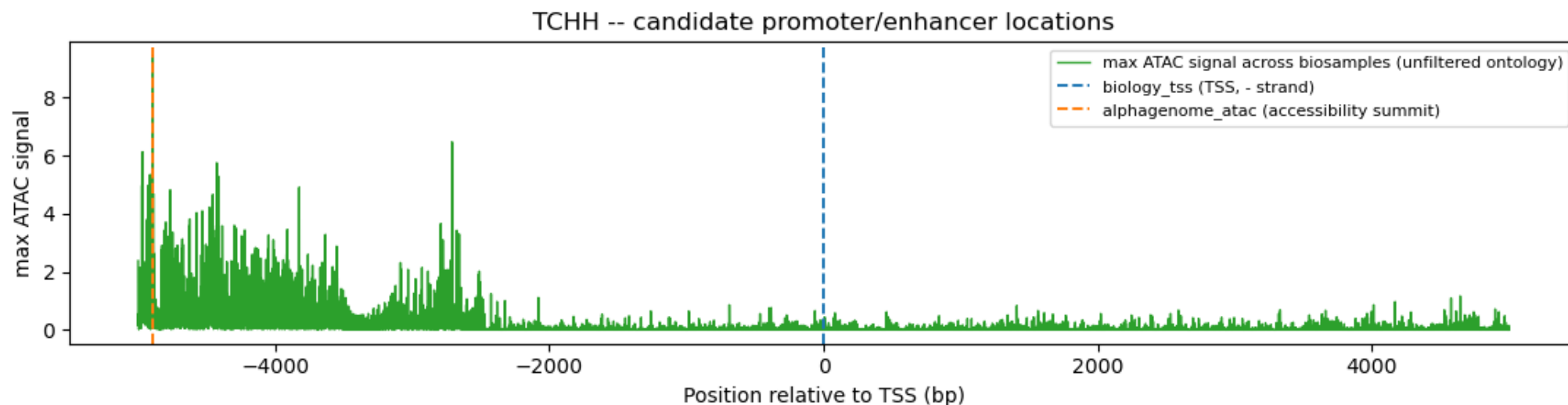


HERC2 -- candidate promoter/enhancer locations



SLC24A5 -- candidate promoter/enhancer locations





Path 3 -- AlphaGenome-functional-signal promoter/enhancer location (CAGE, melanocyte-specific)

One `predict_interval` call per gene, requesting `OutputType.CAGE` filtered to exactly `CL:0002566` (dark melanocyte) and `CL:0002567` (light melanocyte) -- 2 tracks each (+ and - strand). The summit is the position of peak signal across all 4 tracks (both subtypes, both strands) within 5 kb of the TSS, same search radius and reduction as Path 2's ATAC summit, so the two are directly comparable.

In [20]:

```
def find_cage_promoter(gene: str, search_radius: int = 5000) -> dict:
    json_cache_path = ATAC_SUMMIT_CACHE_DIR / f"cage_summit_{gene}.json"
    curve_cache_path = ATAC_SUMMIT_CACHE_DIR / f"cage_summit_{gene}_curves.npz"
    if json_cache_path.exists() and curve_cache_path.exists():
        result = json.loads(json_cache_path.read_text())
        curves = np.load(curve_cache_path)
        result["dark_curve"] = curves["dark"]
        result["light_curve"] = curves["light"]
        return result

    window_meta = json.loads((DATASET_DIR / "references" / "windows" / gene / "window_metadata.json").read_text())
    chrom = window_meta["chromosome"]
    start_lbased = int(window_meta["start"])
    full_ref_length = int(window_meta["end"]) - start_lbased + 1
    assert full_ref_length == dna_client.SEQUENCE_LENGTH_500KB

    _, tss_pos_0based, _ = get_gene_tss(gene)
    tss_local_idx = tss_pos_0based - (start_lbased - 1)

    interval = genome.Interval(chromosome=chrom, start=start_lbased - 1, end=start_lbased - 1 + full_ref_length)
    cage_output = ag_client.predict_interval(
        interval, organism=ORGANISM, requested_outputs=[dna_client.OutputType.CAGE],
        ontology_terms=["CL:0002566", "CL:0002567"],
    )
    cage_values = cage_output.cage.values
    cage_meta = cage_output.cage.metadata
    dark_cols = [i for i, row in cage_meta.iterrows() if row["ontology_curie"] == "CL:0002566"]
    light_cols = [i for i, row in cage_meta.iterrows() if row["ontology_curie"] == "CL:0002567"]

    lo = max(0, tss_local_idx - search_radius)
    hi = min(full_ref_length, tss_local_idx + search_radius)
    # Max across the 2 strand tracks within each subtype -- CAGE is stranded, and either strand's
    # initiation signal is informative for "is this an active promoter here."
    dark_curve = cage_values[lo:hi, dark_cols].max(axis=1)
    light_curve = cage_values[lo:hi, light_cols].max(axis=1)
    combined = np.maximum(dark_curve, light_curve)
    summit_local_idx = lo + int(np.argmax(combined))

    result = {
        "gene": gene, "chrom": chrom, "start_lbased": start_lbased,
        "tss_local_idx": tss_local_idx, "summit_local_idx": summit_local_idx,
        "search_lo": lo, "search_hi": hi,
        "peak_value": float(combined.max()),
    }
    json_cache_path.write_text(json.dumps(result))
```

```

np.savez_compressed(curve_cache_path, dark=dark_curve, light=light_curve)
result["dark_curve"] = dark_curve
result["light_curve"] = light_curve
return result

```

```

for _g in KO_GENES:
    _r = find_cage_promoter(_g)
    print(f"{_g:>8}: CAGE summit at offset {_r['summit_local_idx'] - _r['tss_local_idx']:+d} bp from TSS "
          f"(peak={_r['peak_value']:.2f})")

```

```

MC1R: CAGE summit at offset +4480 bp from TSS (peak=1632.00)
TYRP1: CAGE summit at offset +844 bp from TSS (peak=564.00)
TYR: CAGE summit at offset -4 bp from TSS (peak=784.00)
SLC45A2: CAGE summit at offset -40 bp from TSS (peak=126.00)
DDB1: CAGE summit at offset -36 bp from TSS (peak=1064.00)
EDAR: CAGE summit at offset -82 bp from TSS (peak=2.81)
MFSD12: CAGE summit at offset -3 bp from TSS (peak=1712.00)
OCA2: CAGE summit at offset +1 bp from TSS (peak=162.00)
HERC2: CAGE summit at offset -37 bp from TSS (peak=5.84)
SLC24A5: CAGE summit at offset +28 bp from TSS (peak=227.00)
TCHH: CAGE summit at offset -1 bp from TSS (peak=73.50)

```

Dark vs. light melanocyte CAGE signal -- visualizing the difference

Skin/hair pigmentation is set by a mix of dark (eumelanin-producing) and light (pheomelanin-producing) melanocyte states, so it's worth seeing whether the two subtypes' predicted promoter activity actually differs at each gene, not just their combined peak.

In [21]:

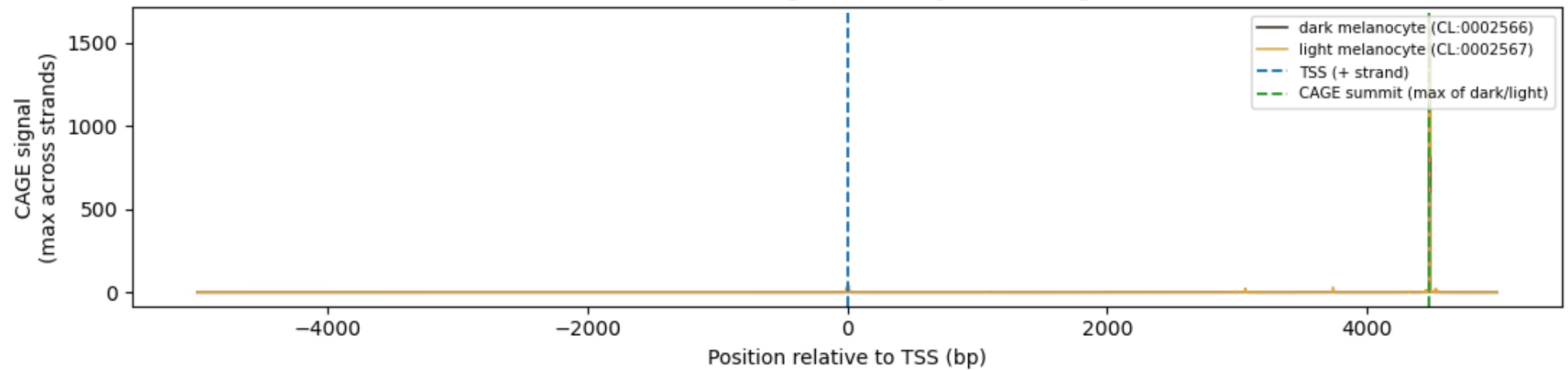
```
CAGE_DARK_COLOR = "#2b2118"
CAGE_LIGHT_COLOR = "#d9a441"

def plot_cage_comparison(gene: str):
    _, tss_pos_0based, strand = get_gene_tss(gene)
    cage = find_cage_promoter(gene)
    tss_local_idx = cage["tss_local_idx"]
    summit_local_idx = cage["summit_local_idx"]
    x = np.arange(cage["search_lo"], cage["search_hi"]) - tss_local_idx

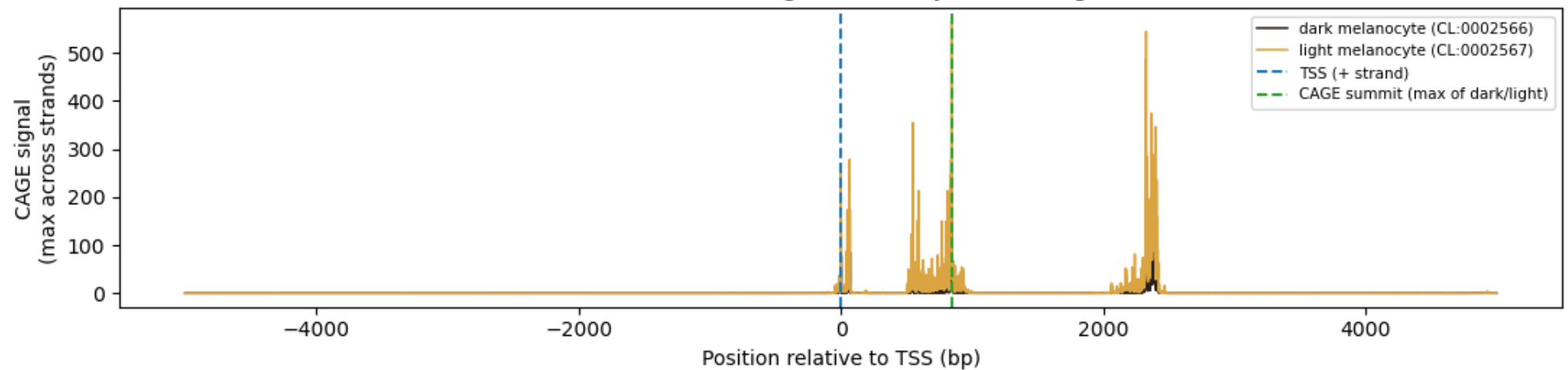
    fig, ax = plt.subplots(figsize=(11, 3))
    ax.plot(x, cage["dark_curve"], color=CAGE_DARK_COLOR, linewidth=1.1, label="dark melanocyte (CL:0002566)")
    ax.plot(x, cage["light_curve"], color=CAGE_LIGHT_COLOR, linewidth=1.1, label="light melanocyte (CL:0002567)")
    ax.axvline(0, color="tab:blue", linestyle="--", linewidth=1.4, label=f"TSS ({strand} strand)")
    ax.axvline(summit_local_idx - tss_local_idx, color="tab:green", linestyle="--", linewidth=1.4,
               label="CAGE summit (max of dark/light)")
    ax.set_xlabel("Position relative to TSS (bp)")
    ax.set_ylabel("CAGE signal\n(max across strands)")
    ax.set_title(f"{gene} -- dark vs. light melanocyte CAGE signal")
    ax.legend(fontsize=7.5, loc="upper right")
    fig.tight_layout()
    out_path = KNOCKOUT_FIG_DIR / f"cage_comparison_{gene}.png"
    fig.savefig(out_path, dpi=140)
    plt.show()
    plt.close(fig)

for _g in KO_GENES:
    plot_cage_comparison(_g)
```

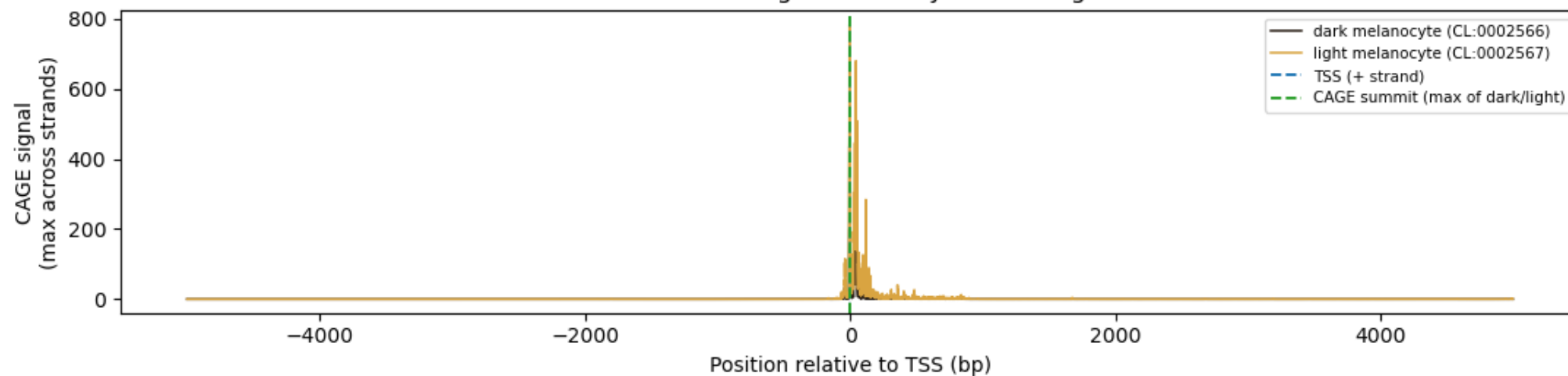
MC1R -- dark vs. light melanocyte CAGE signal



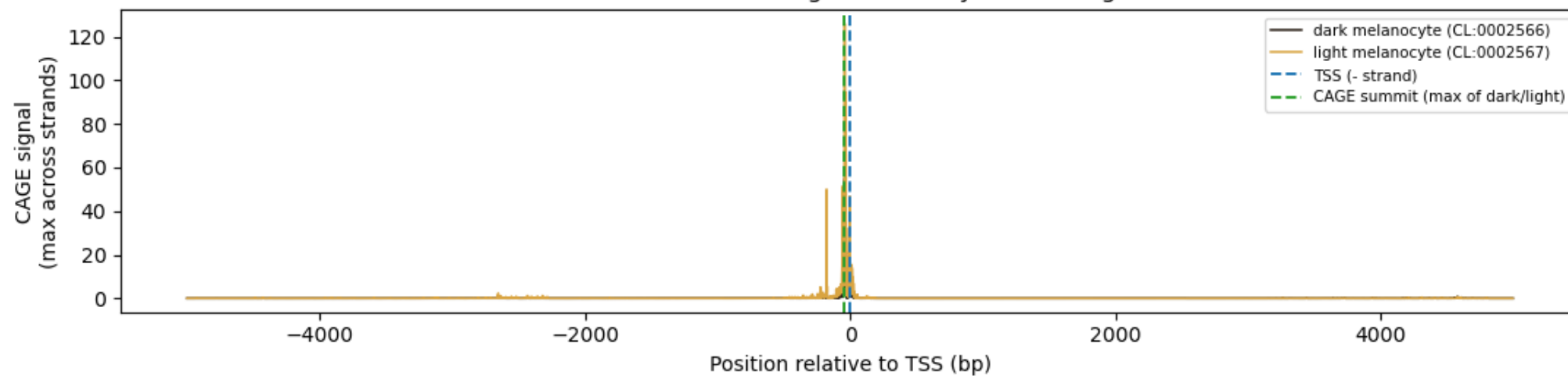
TYRP1 -- dark vs. light melanocyte CAGE signal



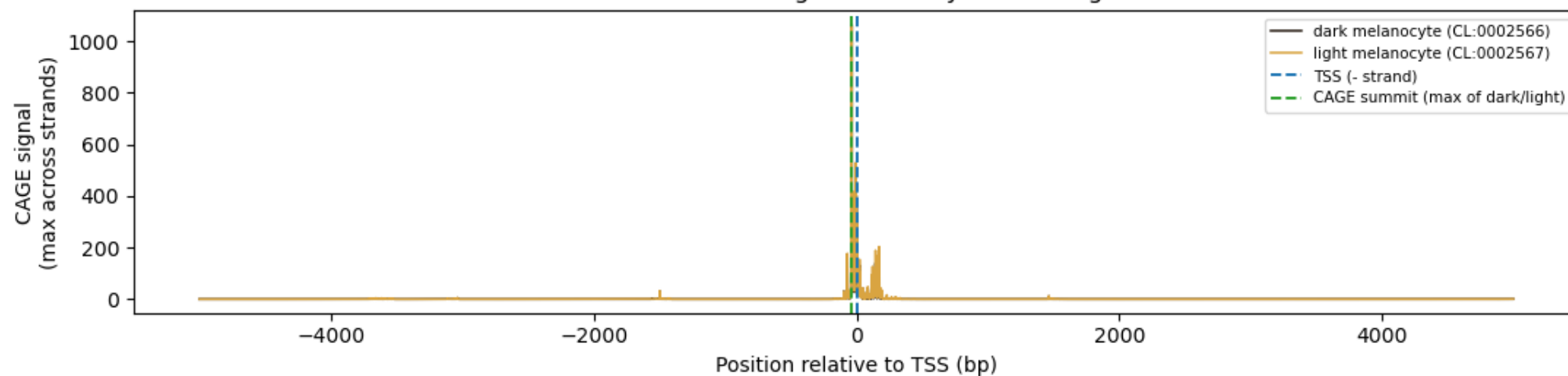
TYR -- dark vs. light melanocyte CAGE signal



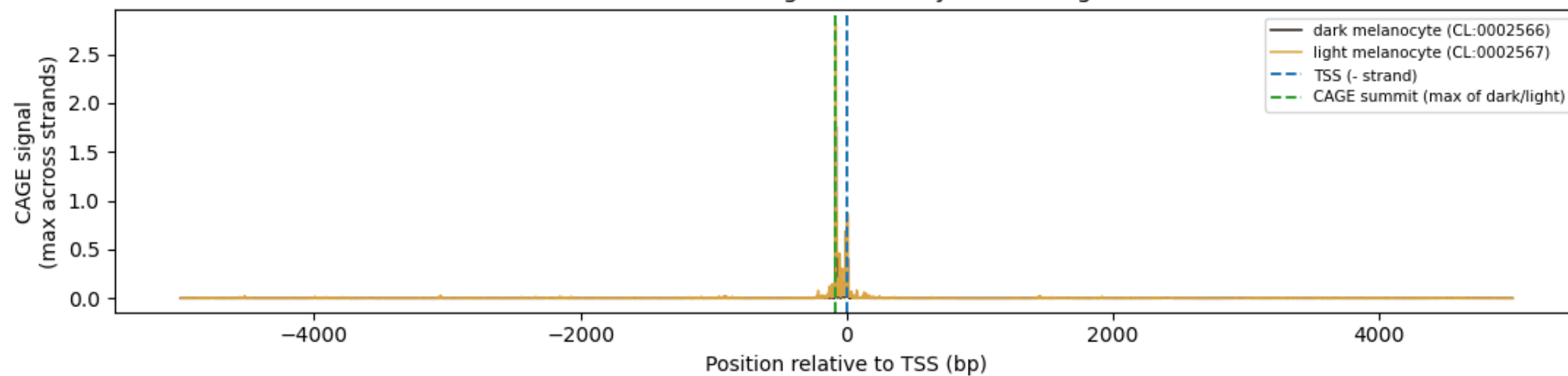
SLC45A2 -- dark vs. light melanocyte CAGE signal



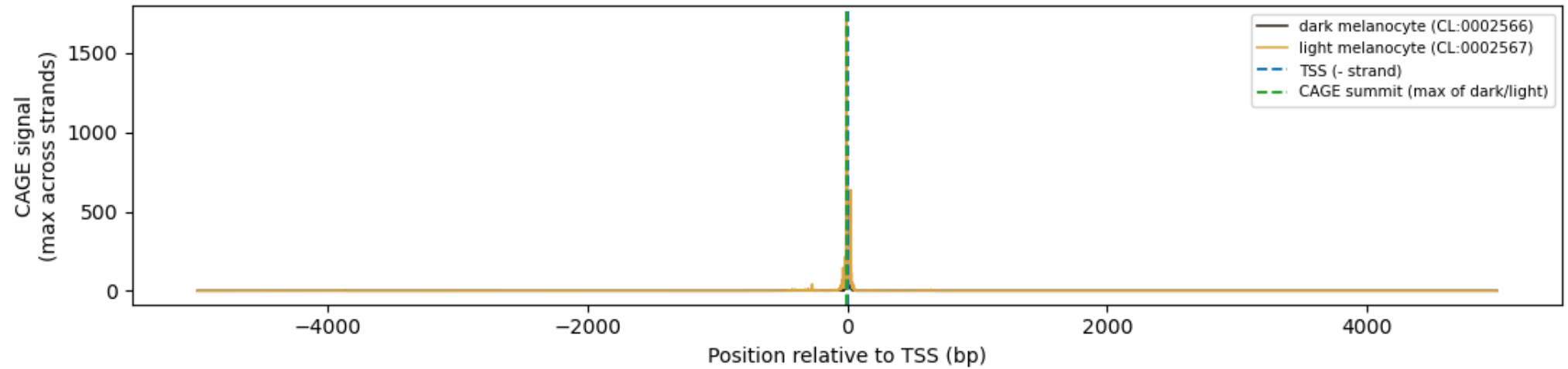
DDB1 -- dark vs. light melanocyte CAGE signal



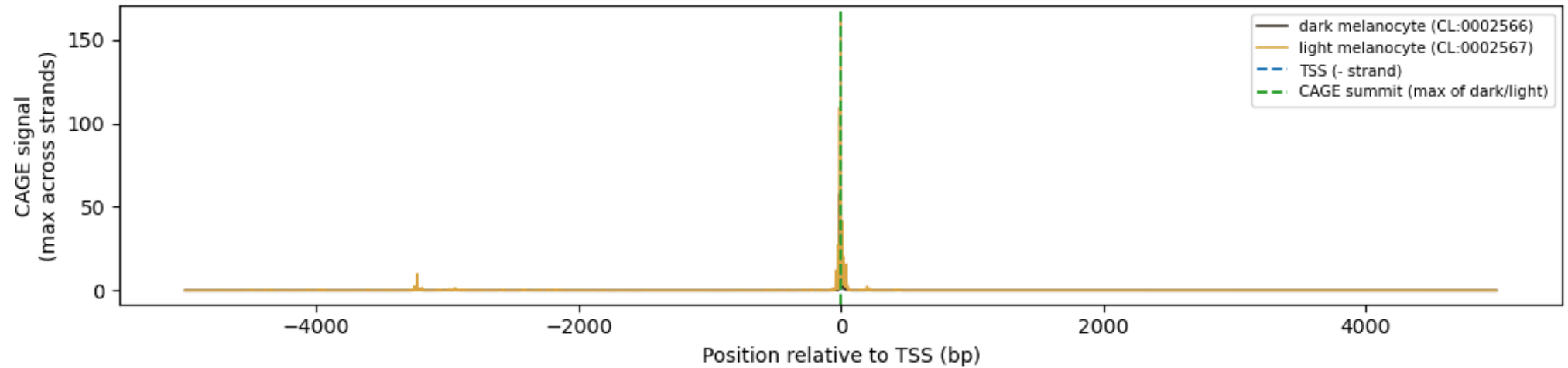
EDAR -- dark vs. light melanocyte CAGE signal



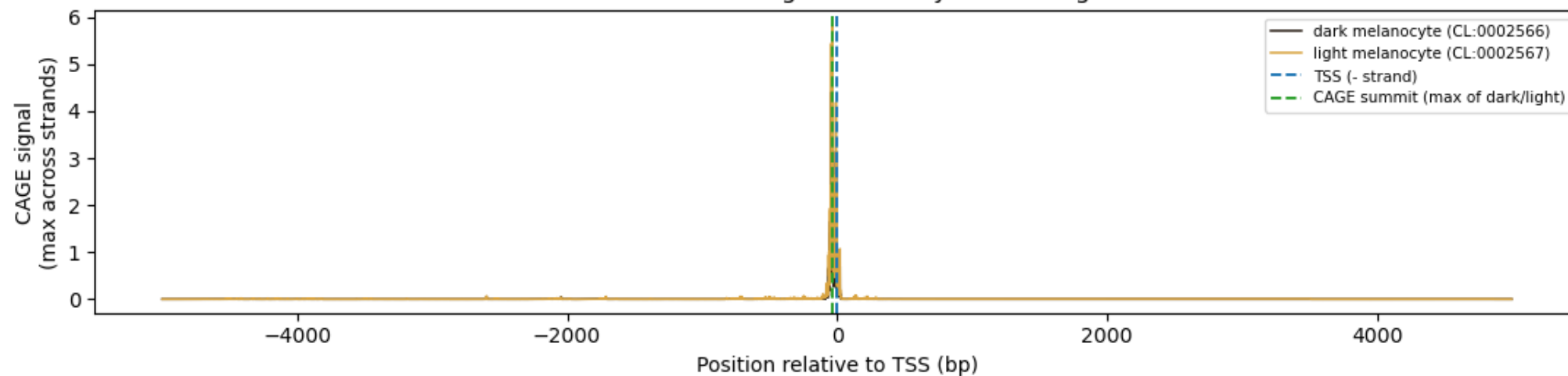
MFSD12 -- dark vs. light melanocyte CAGE signal



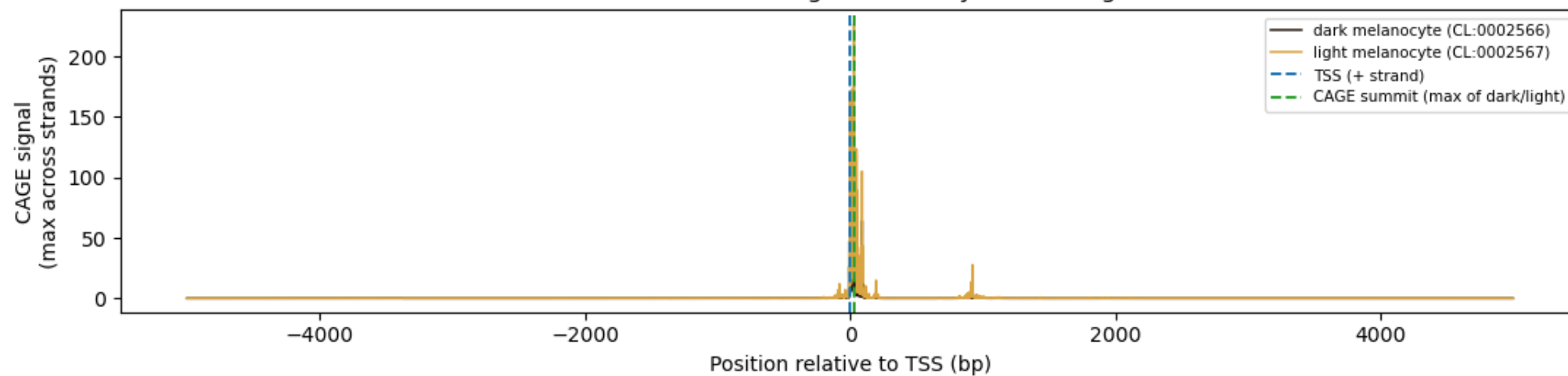
OCA2 -- dark vs. light melanocyte CAGE signal

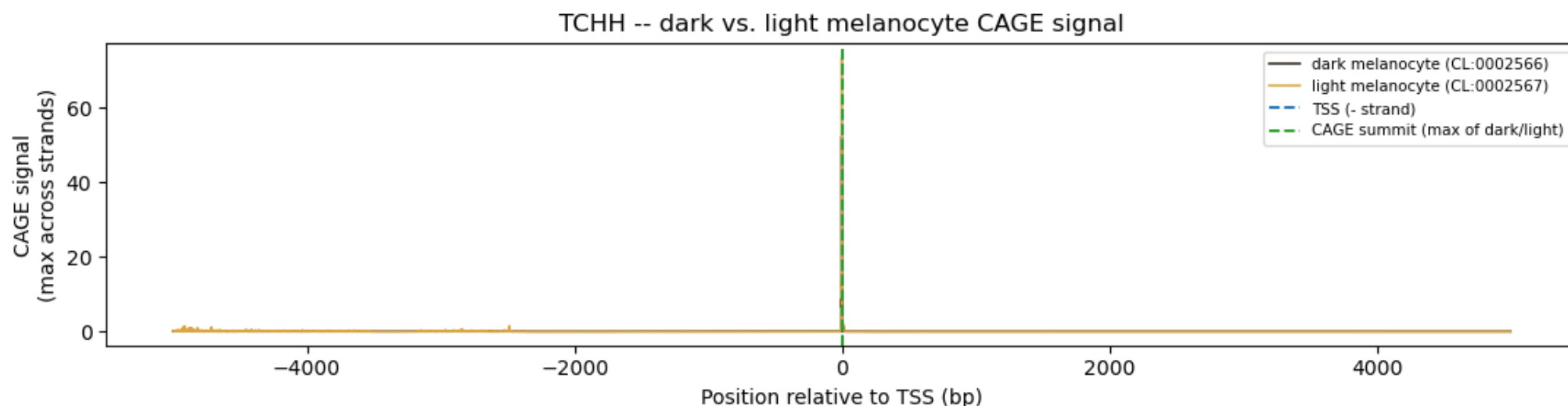


HERC2 -- dark vs. light melanocyte CAGE signal



SLC24A5 -- dark vs. light melanocyte CAGE signal





Sequence / raw-prediction I/O and the window-scramble knockout

A single-base transversion turned out to be too weak a perturbation for most genes (see the discussion above): real promoter/enhancer knockouts remove tens to hundreds of bp, and single-nucleotide sensitivity in AlphaGenome is sparse -- generic TSS/ATAC/CAGE-derived positions usually aren't the one specific base that matters. Instead, **scramble a window_size-bp region** centered on the located position: shuffle (permute, not resample) the existing bases within that window, which destroys any motif/positional structure there (known or unknown -- no need to guess which element matters) while preserving the window's exact base composition (same A/C/G/T counts, so any effect isn't just a GC-content artifact). Sequence length stays fixed, as `predict_sequence` requires.

H1/H2 indel-drift correction. `TSS / ATAC summit / CAGE summit` are all computed once in true reference genome coordinates -- but `<sample>.H{1,2}.window.fixed.fa` is built by `bcftools consensus`, which actually inserts/deletes bases at each variant (including structural `<DEL>` calls, resolved via their `SVLEN/END` INFO fields, not literal indels), shifting every downstream position by that variant's length delta. Since H1 and H2 usually carry different indels/SVs upstream of the target position, a single reference-based local index can land at genuinely different biological positions in the two haplotypes' consensus sequences -- for `HG02461 / TYRP1` this drift is **-2,696 bp on H1 vs. +9 bp on H2** by the time you reach the TSS (H1 carries a real, heterozygous 2,797 bp structural deletion upstream of the TSS that H2 doesn't). `haplotype_local_idx` below corrects for this per haplotype (by replaying that haplotype's own indels/SVs from the window VCF) before scrambling or plotting, so "the TSS" means the same actual sequence content in both H1 and H2, not just the same naive offset.

In [22]:

```
import gzip

def load_haplotype_fasta(sample_id: str, gene: str, haplotype: str) -> str:
    path = DATASET_DIR / "individuals" / sample_id / "windows" / gene / f"{sample_id}.{haplotype}.window.fixed.fa"
    lines = path.read_text().splitlines()
    return "".join(line for line in lines if not line.startswith(">"))

def load_raw_prediction(sample_id: str, gene: str, haplotype: str):
    d = DATASET_DIR / "individuals" / sample_id / "windows" / gene / f"predictions_{haplotype}"
    values = np.load(d / "rna_seq.npz")["values"]
    meta = json.loads((d / "rna_seq_metadata.json").read_text())["metadata"]
    return values, meta

_HAPLOTYPE_DRIFT_CACHE = {}

def _haplotype_indel_events(sample_id: str, gene: str, haplotype: str) -> list:
    '''(pos_lbased, length_delta) for every length-changing variant this haplotype carries in the
    gene's window VCF, sorted by position -- used to replay bcftools consensus's position-shifting
    effect. Handles multi-allelic sites (picks the ALT the phased GT actually points to) and
    symbolic structural-variant alleles: the dataset's consensus-ready VCF is filtered with
    `bcftools view -e \"ALT~\"<\" && ALT!=\"<DEL>\" && ALT!=\"<NON_REF>\"`, so <DEL> entries (using
    their SVLEN/END, not a literal string length) DO reach bcftools consensus and must be counted
    -- e.g. HG02461/TYRP1 carries a real 2,797 bp <DEL> on H1 upstream of the TSS, which a naive
    len(alt)-len(ref) on the literal "<DEL>" text would undercount by ~2.8 kb. Every other symbolic
    type (<DUP>, <INS>, <INS:ME:ALU>, ...) is dropped by that same filter and never reaches the
    consensus sequence, so it's skipped here too.'''
    key = (sample_id, gene, haplotype)
    if key in _HAPLOTYPE_DRIFT_CACHE:
        return _HAPLOTYPE_DRIFT_CACHE[key]

    vcf_path = DATASET_DIR / "individuals" / sample_id / "windows" / gene / f"{sample_id}.window.vcf.gz"
    hap_index = 0 if haplotype == "H1" else 1
    events = []
    with gzip.open(vcf_path, "rt") as f:
        for line in f:
            if line.startswith("#"):
                continue
            cols = line.rstrip("\n").split("\t")
            pos = int(cols[1])
            ref = cols[3]
            alt_alleles = cols[4].split(",")
```

```

info = cols[7]
gt = cols[9].split(":")[0]
sep = "|" if "|" in gt else "/"
alleles = gt.split(sep)
if len(alleles) != 2:
    continue
allele_str = alleles[hap_index]
if allele_str in ("0", "."):
    continue
allele_num = int(allele_str)
if not (1 <= allele_num <= len(alt_alleles)):
    continue
alt = alt_alleles[allele_num - 1]

if alt.startswith("<"):
    if alt != "<DEL>":
        continue # filtered out before bcftools consensus -- never shifts the sequence
    delta = None
    for field in info.split(";"):
        if field.startswith("SVLEN="):
            delta = int(field.split("=", 1)[1].split(",")[0])
            break
    if delta is None:
        end = None
        for field in info.split(";"):
            if field.startswith("END="):
                end = int(field.split("=", 1)[1])
                break
        if end is None:
            continue
        delta = -(end - pos)
else:
    delta = len(alt) - len(ref)

if delta != 0:
    events.append((pos, delta))
events.sort()
_HAPLOTYPE_DRIFT_CACHE[key] = events
return events

```

```

def haplotype_local_idx(sample_id: str, gene: str, haplotype: str, start_lbased: int, target_pos_lbased: int) -> int:
    '''Reference position target_pos_lbased -> 0-based local index into this haplotype's own
    fixed FASTA, correcting for this haplotype's own indels between the window start and the
    target (see markdown above).'''
    events = _haplotype_indel_events(sample_id, gene, haplotype)
    drift = sum(delta for pos, delta in events if pos < target_pos_lbased)

```

```

    return (target_pos_lbased - start_lbased) + drift

def apply_scramble(sequence: str, center_idx: int, window_size: int = 100, seed: int = 0):
    '''Shuffles the window_size-bp window centered on center_idx (clamped to stay in-bounds and
    always exactly window_size long). A fixed seed keeps results reproducible across reruns --
    the shuffle *pattern* is the same every call, but the bases it's applied to differ per
    gene/haplotype/method, so the actual scrambled sequence still differs each time.'''
    half = window_size // 2
    start = max(0, center_idx - half)
    end = min(len(sequence), start + window_size)
    if end - start < window_size:
        start = max(0, end - window_size)
    original_segment = list(sequence[start:end])
    rng = np.random.default_rng(seed)
    scrambled = original_segment.copy()
    rng.shuffle(scrambled)
    scrambled_segment = "".join(scrambled)
    modified = sequence[:start] + scrambled_segment + sequence[end:]
    return modified, "".join(original_segment), scrambled_segment, start, end

def reorder_to_canonical(values: np.ndarray, metadata_df, canonical_order: list) -> np.ndarray:
    lookup = {(row["ontology_curie"], row["strand"]): i for i, row in metadata_df.iterrows()}
    col_idx = [lookup[key] for key in canonical_order]
    return values[:, col_idx]

def predict_modified_sequence(cache_key: str, sequence: str, interval):
    cache_path = ATAC_SUMMIT_CACHE_DIR / f"seq_{cache_key}.npz"
    meta_cache_path = ATAC_SUMMIT_CACHE_DIR / f"seq_{cache_key}_meta.json"
    if cache_path.exists() and meta_cache_path.exists():
        return np.load(cache_path)["values"], json.loads(meta_cache_path.read_text())

    output = ag_client.predict_sequence(
        sequence, organism=ORGANISM, requested_outputs=[dna_client.OutputType.RNA_SEQ],
        ontology_terms=KO_ONTOLOGY_TERMS, interval=interval,
    )
    values = output.rna_seq.values
    meta_records = output.rna_seq.metadata[["ontology_curie", "strand"]].to_dict("records")
    np.savez_compressed(cache_path, values=values)
    meta_cache_path.write_text(json.dumps(meta_records))
    return values, meta_records

print("Sequence/knockout helpers ready.")

```

Sequence/knockout helpers ready.

Validation -- does `predict_sequence` on the unmodified consensus FASTA reproduce the cached

on-disk prediction?

Sanity check before trusting any knockout result: re-predicting the individual's own unmodified haplotype sequence should closely match `predictions_H{1,2}/rna_seq.npz` (the array the CNN was actually trained on for this individual).

In [23]:

```
_val_gene = KO_GENES[0]
_val_window = json.loads((DATASET_DIR / "references" / "windows" / _val_gene / "window_metadata.json").read_text())
_val_chrom = _val_window["chromosome"]
_val_start = int(_val_window["start"])
_val_len = int(_val_window["end"]) - _val_start + 1
_val_interval = genome.Interval(chromosome=_val_chrom, start=_val_start - 1, end=_val_start - 1 + _val_len)

_val_seq = load_haplotype_fasta(INDIVIDUAL_ID, _val_gene, "H1")
_val_orig_array, _val_orig_meta = load_raw_prediction(INDIVIDUAL_ID, _val_gene, "H1")
_val_canonical_order = [(m["ontology_curie"], m["strand"]) for m in _val_orig_meta]

_val_output = ag_client.predict_sequence(
    _val_seq, organism=ORGANISM, requested_outputs=[dna_client.OutputType.RNA_SEQ],
    ontology_terms=KO_ONTOLOGY_TERMS, interval=_val_interval,
)
_val_reordered = reorder_to_canonical(_val_output.rna_seq.values, _val_output.rna_seq.metadata, _val_canonical_order)
_val_corr = np.corrcoef(_val_orig_array.flatten(), _val_reordered.flatten())[0, 1]
print(f"Validation ({INDIVIDUAL_ID}/{_val_gene}/H1): corr(cached, re-predicted-unmodified) = {_val_corr:.6f}")
assert _val_corr > 0.999, "predict_sequence on the unmodified consensus FASTA should closely match the cached prediction"
```

Validation (HG02461/MC1R/H1): corr(cached, re-predicted-unmodified) = 0.999973

Tensor rebuild + CNN scoring, and the main knockout-experiment function

In [24]:

```
def build_individual_tensor(sample_id: str, overrides: dict = None) -> torch.Tensor:
    '''overrides: {(gene, haplotype): (raw_array, track_meta)} substituted in place of the
    on-disk cached prediction for that gene/haplotype; every other gene/haplotype uses the
    individual's real, unmodified prediction.'''
    overrides = overrides or {}
    h1_rows, h2_rows = [], []
    for gene in KO_GENES:
        for haplotype, rows in (("H1", h1_rows), ("H2", h2_rows)):
            if (gene, haplotype) in overrides:
                array, meta = overrides[(gene, haplotype)]
            else:
                array, meta = load_raw_prediction(sample_id, gene, haplotype)
            result = ko_full_ds._process_window_haplotype_channels(
                sample_id, gene, haplotype, {"rna_seq": array}, {"rna_seq": meta},
            )
            if result is None:
                raise RuntimeError(f"could not build tensor for {sample_id}/{gene}/{haplotype}")
            signals, _masks = result
            rows.append(signals)
    stacked = np.stack([np.concatenate(h1_rows, axis=0), np.concatenate(h2_rows, axis=0)], axis=0)
    return apply_normalization(torch.from_numpy(stacked), ko_normalization_params)

def run_cnn_logits(tensor: torch.Tensor) -> torch.Tensor:
    with torch.no_grad():
        return KO_MODEL(tensor.unsqueeze(0).float().to(device))[0]

def knockout_gene_experiment(sample_id: str, gene: str, method: str, scramble_window: int = 100) -> dict:
    window_meta = json.loads((DATASET_DIR / "references" / "windows" / gene / "window_metadata.json").read_text())
    chrom = window_meta["chromosome"]
    start_lbased = int(window_meta["start"])
    full_ref_length = int(window_meta["end"]) - start_lbased + 1
    interval = genome.Interval(chromosome=chrom, start=start_lbased - 1, end=start_lbased - 1 + full_ref_length)

    # All three candidate positions are always computed in *reference* coordinates (regardless of
    # which one this experiment edits) so the plot can show them on one shared window -- the
    # methods can land up to `search_radius` (5 kb) apart, so zooming independently around each
    # method's own site would show non-overlapping genomic windows and make them look spuriously
    # different.
    _, tss_pos_0based, _ = get_gene_tss(gene)
    tss_local_idx = tss_pos_0based - (start_lbased - 1)
    atac_summit_local_idx = find_atac_summit(gene)["summit_local_idx"]
    cage_summit_local_idx = find_cage_promoter(gene)["summit_local_idx"]
    ref_local_idx_by_method = {
```

```

    "biology_tss": tss_local_idx,
    "alphagenome_atac": atac_summit_local_idx,
    "cage_melanocyte": cage_summit_local_idx,
}
genomic_pos_by_method = {m: start_lbased + idx for m, idx in ref_local_idx_by_method.items()}

location_labels = {
    "biology_tss": "TSS (GENCODE MANE Select)",
    "alphagenome_atac": "ATAC accessibility summit (unfiltered ontology)",
    "cage_melanocyte": "CAGE summit (dark/light melanocyte)",
}
if method not in location_labels:
    raise ValueError(f"unknown method: {method!r}")
location_label = location_labels[method]
genomic_pos = genomic_pos_by_method[method]

hap_tracks = {}
for haplotype in ("H1", "H2"):
    # Reference coordinates -> this haplotype's own consensus-local coordinates, correcting
    # for its own indels between the window start and each candidate position (see markdown
    # above -- H1 and H2 usually carry different indels, so a shared reference-based index can
    # land tens to hundreds of bp off target on one haplotype even when it's correct on the
    # other).
    marker_local_idx = {
        m: haplotype_local_idx(sample_id, gene, haplotype, start_lbased, genomic_pos_by_method[m])
        for m in location_labels
    }
    hap_target_local_idx = marker_local_idx[method]

    seq = load_haplotype_fasta(sample_id, gene, haplotype)
    modified_seq, orig_segment, scrambled_segment, seg_start, seg_end = apply_scramble(
        seq, hap_target_local_idx, window_size=scramble_window,
    )
    orig_array, orig_meta = load_raw_prediction(sample_id, gene, haplotype)
    canonical_order = [(m["ontology_curie"], m["strand"]) for m in orig_meta]

    cache_key = f"{sample_id}_{gene}_{haplotype}_{method}_{hap_target_local_idx}_scramble{scramble_window}"
    mod_values, mod_meta_records = predict_modified_sequence(cache_key, modified_seq, interval)
    mod_reordered = reorder_to_canonical(mod_values, pd.DataFrame(mod_meta_records), canonical_order)

    hap_tracks[haplotype] = {
        "orig_array": orig_array, "mod_array": mod_reordered, "meta": orig_meta,
        "scramble_start": seg_start, "scramble_end": seg_end,
        "orig_segment": orig_segment, "scrambled_segment": scrambled_segment,
        "marker_local_idx": marker_local_idx,
    }
}

```

```

baseline_tensor = build_individual_tensor(sample_id)
overrides = {
    (gene, hap): (hap_tracks[hap]["mod_array"], hap_tracks[hap]["meta"]) for hap in ("H1", "H2")
}
perturbed_tensor = build_individual_tensor(sample_id, overrides=overrides)

baseline_logits = run_cnn_logits(baseline_tensor)
perturbed_logits = run_cnn_logits(perturbed_tensor)

return {
    "sample_id": sample_id, "gene": gene, "method": method, "location_label": location_label,
    "chrom": chrom, "genomic_pos": genomic_pos,
    "tss_local_idx": tss_local_idx, "atac_summit_local_idx": atac_summit_local_idx,
    "cage_summit_local_idx": cage_summit_local_idx, "scramble_window": scramble_window,
    "hap_tracks": hap_tracks,
    "baseline_logits": baseline_logits.cpu().numpy(),
    "perturbed_logits": perturbed_logits.cpu().numpy(),
    "delta_strong_logit": (perturbed_logits[ko_strong_idx] - baseline_logits[ko_strong_idx]).item(),
}

print("knockout_gene_experiment ready.")

```

knockout_gene_experiment ready.

Plot: original vs. knockout RNA-seq track, and the impact on the classification

In [25]:

```
from matplotlib.patches import Patch

def cnn_crop_bounds(full_ref_length: int, window_center_size: int):
    # Identical centering formula to ReferenceRealignMapper._center_crop_bounds / Section 8's
    # window_genomic_axis: the actual window_center_size-bp slice the CNN receives as input,
    # taken from the middle of the full 524,288 bp AlphaGenome window.
    size = min(window_center_size, full_ref_length)
    center_offset = full_ref_length // 2
    crop_start = max(0, center_offset - size // 2)
    crop_end = min(full_ref_length, crop_start + size)
    if crop_end - crop_start < size:
        crop_start = max(0, crop_end - size)
    return crop_start, crop_end

def plot_knockout_result(result: dict, marker_margin: int = 1000):
    # Each of the 6 tracks (3 ontology terms x 2 strands) plotted separately, not averaged --
    # they come from different cell types and strands, so a mean across them isn't a meaningful
    # biological quantity (a strong effect on one track can be washed out by four flat ones).
    track_meta = result["hap_tracks"]["H1"]["meta"]
    track_labels = [f"{m['ontology_curie']} ({m['strand']})" for m in track_meta]
    n_tracks = len(track_labels)

    # Base window = the CNN's actual 32,768 bp input crop (so what's plotted matches what the
    # model sees) -- extended only as needed to include the TSS and/or ATAC/CAGE summits (each
    # with a 1 kb margin) when a marker falls outside that crop, which happens for several of the
    # 11 genes (see the discussion above). Computed from the *reference*-coordinate markers just
    # to frame the window -- 1 kb margin comfortably absorbs the haplotype-specific drift
    # (typically tens to a few hundred bp) applied below, so this framing doesn't need per-hap
    # correction itself.
    orig_len = result["hap_tracks"]["H1"]["orig_array"].shape[0]
    crop_lo, crop_hi = cnn_crop_bounds(orig_len, KO_CONFIG.dataset_input.window_center_size)
    ref_markers = [result["tss_local_idx"], result["atac_summit_local_idx"], result["cage_summit_local_idx"]]
    win_lo = min([crop_lo] + [m - marker_margin for m in ref_markers if m < crop_lo])
    win_hi = max([crop_hi] + [m + marker_margin for m in ref_markers if m >= crop_hi])
    win_span = win_hi - win_lo

    # sharey="row": H1 vs H2 for the *same* track must be on the same y-scale to be visually
    # comparable -- without it matplotlib auto-scales each subplot independently, so a flat H2
    # track can look deceptively different from an active H1 track just because of axis scaling.
    fig, axes = plt.subplots(n_tracks, 2, figsize=(13, 1.9 * n_tracks), sharex=True, sharey="row")
    for col, hap in enumerate(["H1", "H2"]):
        track = result["hap_tracks"][hap]
        orig, mod = track["orig_array"], track["mod_array"]
```

```

# Anchor x=0 at *this haplotype's own* indel-corrected TSS (not a shared reference-based
# index) -- H1 and H2 can carry different indels upstream of the target, so "the TSS" can
# sit tens to hundreds of bp apart between the two haplotypes' actual consensus sequences.
hap_tss_idx = track["marker_local_idx"]["biology_tss"]
lo, hi = max(0, hap_tss_idx + (win_lo - result["tss_local_idx"])), \
min(orig.shape[0], hap_tss_idx + (win_lo - result["tss_local_idx"]) + win_span)
x = np.arange(lo, hi) - hap_tss_idx
for row in range(n_tracks):
    ax = axes[row, col]
    # Shaded band = the CNN's actual 32 kb input crop, so it's visually obvious how much
    # of the plotted window (if any extension was needed) the model never directly sees.
    ax.axvspan(crop_lo - hap_tss_idx, crop_hi - hap_tss_idx, color="gray", alpha=0.10, zorder=0)
    ax.plot(x, orig[lo:hi, row], color="tab:blue", linewidth=0.6, alpha=0.65, label="original")
    ax.plot(x, mod[lo:hi, row], color="tab:red", linewidth=0.6, alpha=0.65, label="knockout")
    # All three candidate locations, on every panel, at *this haplotype's* own
    # indel-corrected position -- solid+thicker for whichever one this particular
    # experiment actually edited (plus a shaded band showing the *actual* scrambled
    # window for that method), thin+dotted for the other methods' sites.
    marker_colors = {
        "biology_tss": "#1f77b4", "alphagenome_atac": "#ff7f0e", "cage_melanocyte": "#2ca02c",
    }
    for marker_method, marker_color in marker_colors.items():
        marker_idx = track["marker_local_idx"][marker_method]
        active = marker_method == result["method"]
        if active:
            ax.axvspan(track["scramble_start"] - hap_tss_idx, track["scramble_end"] - hap_tss_idx,
                color=marker_color, alpha=0.20, zorder=1)
            ax.axvline(marker_idx - hap_tss_idx, color=marker_color, linewidth=1.4, alpha=0.9)
        else:
            ax.axvline(marker_idx - hap_tss_idx, color=marker_color, linewidth=0.9, linestyle=":", alpha=0.55)
    if col == 0:
        ax.set_ylabel(track_labels[row], fontsize=7.5)
    if row == 0:
        ax.set_title(
            f"{hap} (scrambled {track['scramble_start'] - hap_tss_idx}..{track['scramble_end'] - hap_tss_idx})"
        )
    handles = [
        plt.Line2D([], [], color="tab:blue", linewidth=0.6, alpha=0.65, label="original"),
        plt.Line2D([], [], color="tab:red", linewidth=0.6, alpha=0.65, label="knockout"),
        plt.Line2D([], [], color="#1f77b4", linewidth=1.6, label="TSS"),
        plt.Line2D([], [], color="#ff7f0e", linewidth=1.6, label="ATAC summit"),
        plt.Line2D([], [], color="#2ca02c", linewidth=1.6, label="CAGE summit"),
        Patch(facecolor="gray", alpha=0.10, label="CNN 32 kb input crop"),
        Patch(facecolor="gray", alpha=0.20, label="scrambled window (active method)"),
    ]
    ax.legend(handles=handles, fontsize=5.5, ncol=2)
    if row == n_tracks - 1:

```

```
ax.set_xlabel("Position relative to this haplotype's own TSS (bp)")

strong_name = class_names[ko_strong_idx]
fig.suptitle(
    f"{result['gene']} -- {result['sample_id']} -- edited: {result['location_label']} @ "
    f"{result['chrom']}:{result['genomic_pos']:,} -- "
    f"delta({strong_name} logit) = {result['delta_strong_logit']:+.4f}"
)
fig.tight_layout()
out_path = KNOCKOUT_FIG_DIR / f"knockout_{result['gene']}_{result['method']}.png"
fig.savefig(out_path, dpi=140)
plt.show()
plt.close(fig)
```

Run the knockout experiment for every gene, all three location methods, for `INDIVIDUAL_ID`

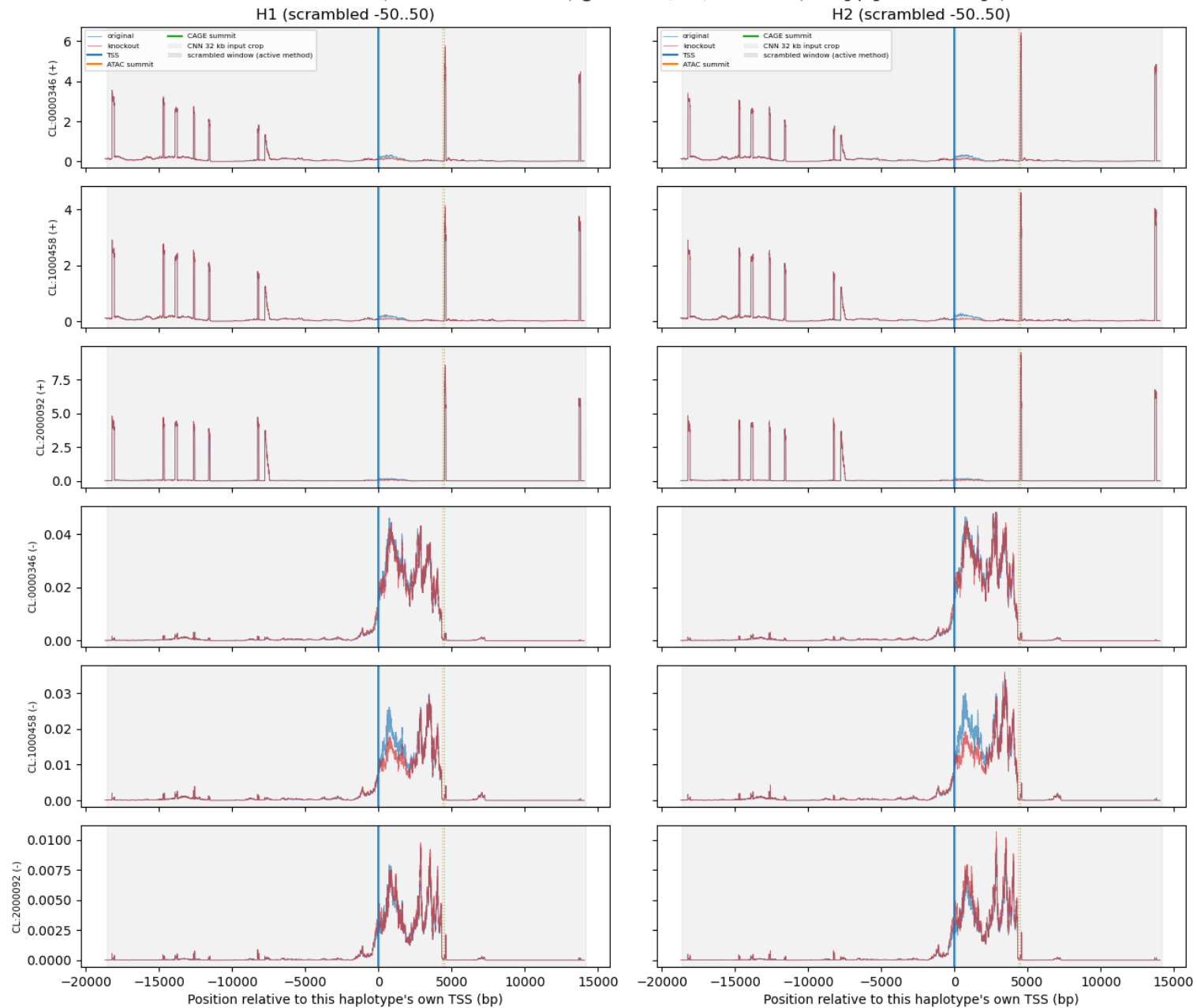
In [26]:

```
KO_METHODS = ["biology_tss", "alphagenome_atac", "cage_melanocyte"]

knockout_results = []
for gene in KO_GENES:
    for method in KO_METHODS:
        print(f"--- {gene} / {method} ---")
        result = knockout_gene_experiment(INDIVIDUAL_ID, gene, method)
        knockout_results.append(result)
        plot_knockout_result(result)
        print(f"baseline logits: {result['baseline_logits']}, perturbed logits: {result['perturbed_logits']}, "
              f"delta({class_names[ko_strong_idx]})={result['delta_strong_logit']:+.4f}")
```

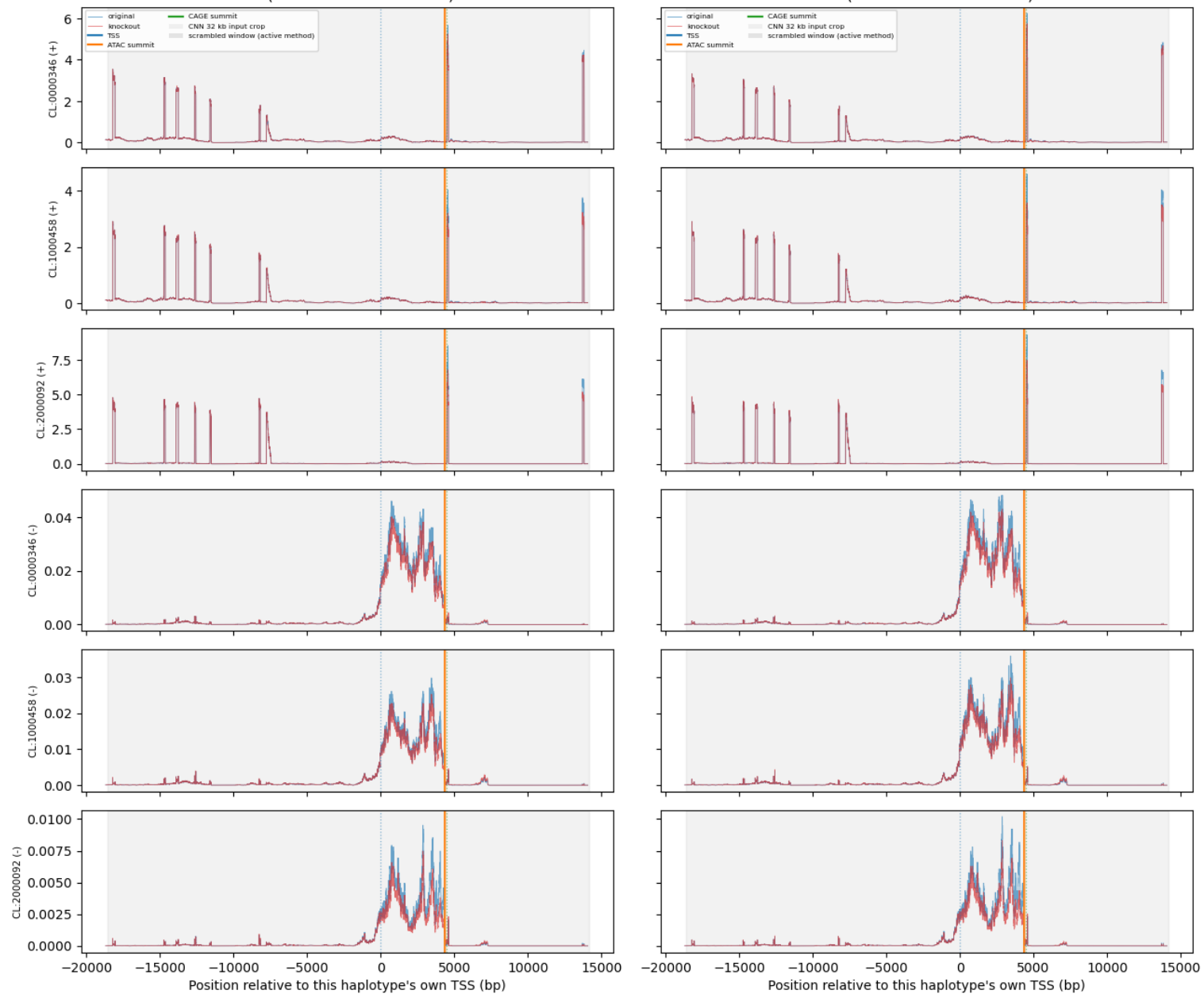
--- MC1R / biology_tss ---

MC1R -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr16:89,918,862 -- delta(strong pigmentation logit) = -0.0714



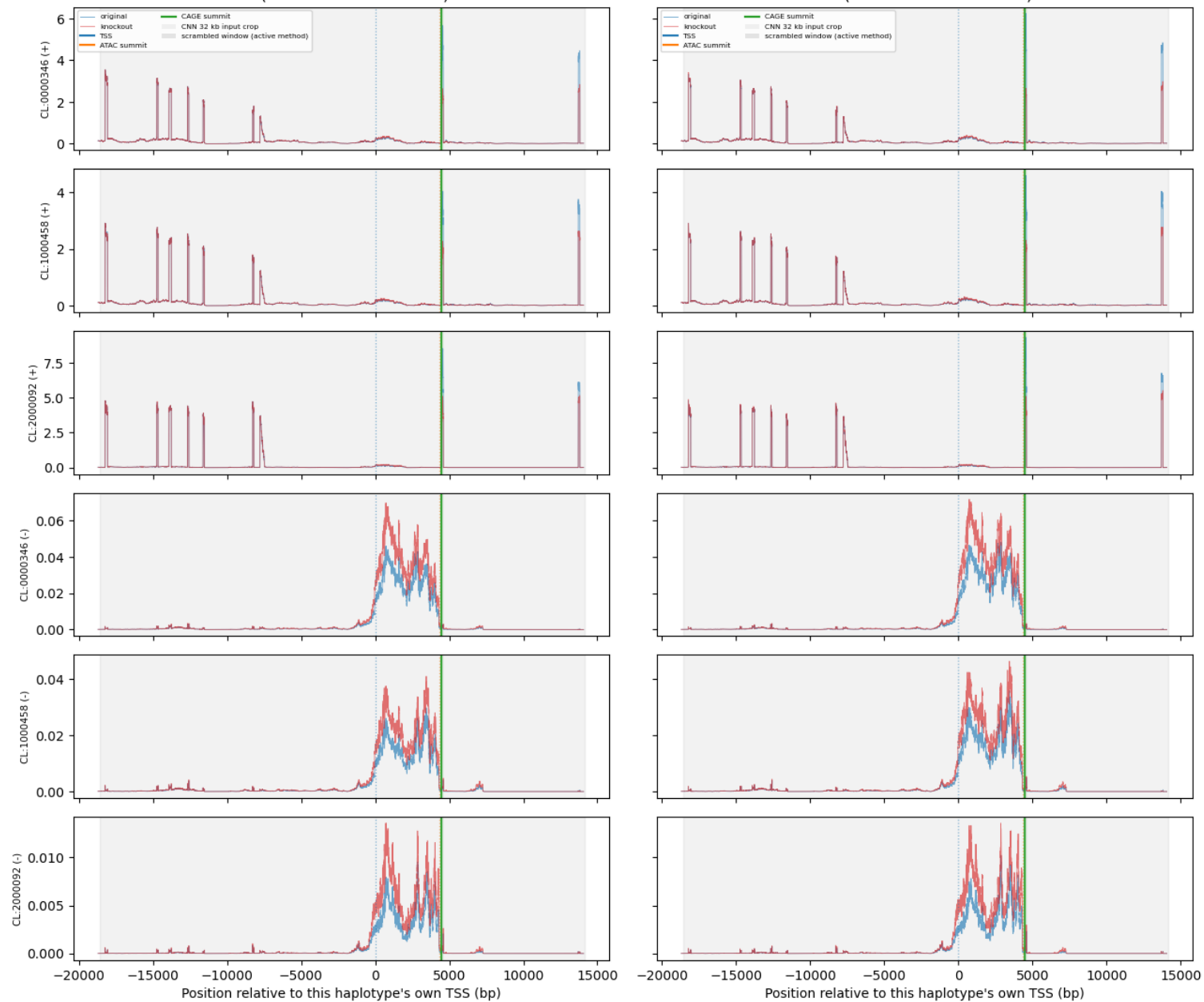
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.7538929 -1.8668519], delta(strong pigmentation)=-0.0714
--- MC1R / alphagenome_atac ---
```

MC1R -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr16:89,923,221 -- delta(strong pigmentation logit) = -0.0299
H1 (scrambled 4309..4409) H2 (scrambled 4309..4409)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.7953846 -1.9711194], delta(strong pigmentation)=-0.0299
--- MC1R / cage_melanocyte ---
```

MC1R -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr16:89,923,342 -- delta(strong pigmentation logit) = +0.0338
H1 (scrambled 4430..4530) H2 (scrambled 4430..4530)

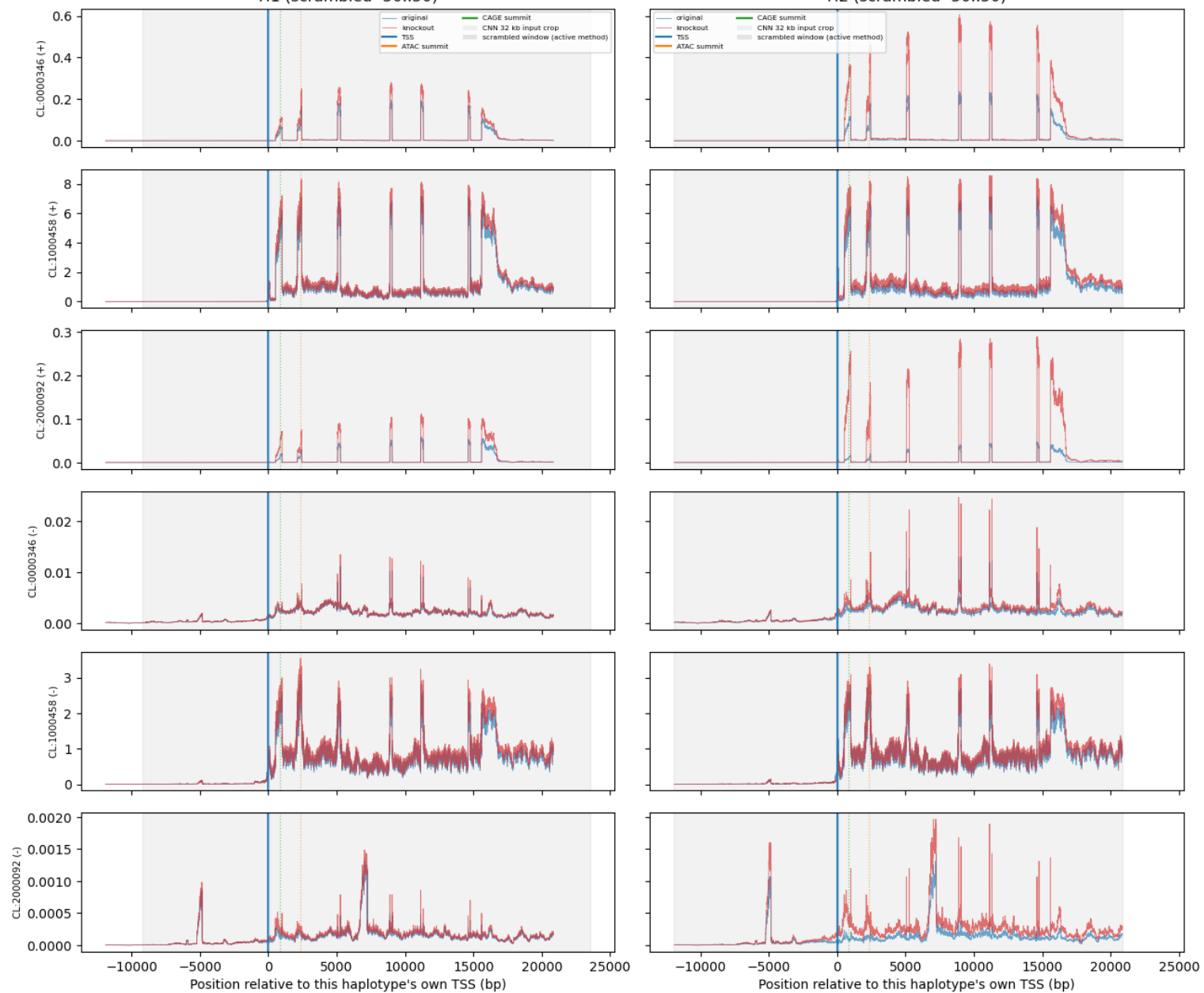


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.85908955 -2.1305037 ], delta(strong pigmentation)=+0.0338
--- TYRP1 / biology_tss ---
```

TYRP1 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr9:12,693,385 -- delta(strong pigmentation logit) = -0.3410

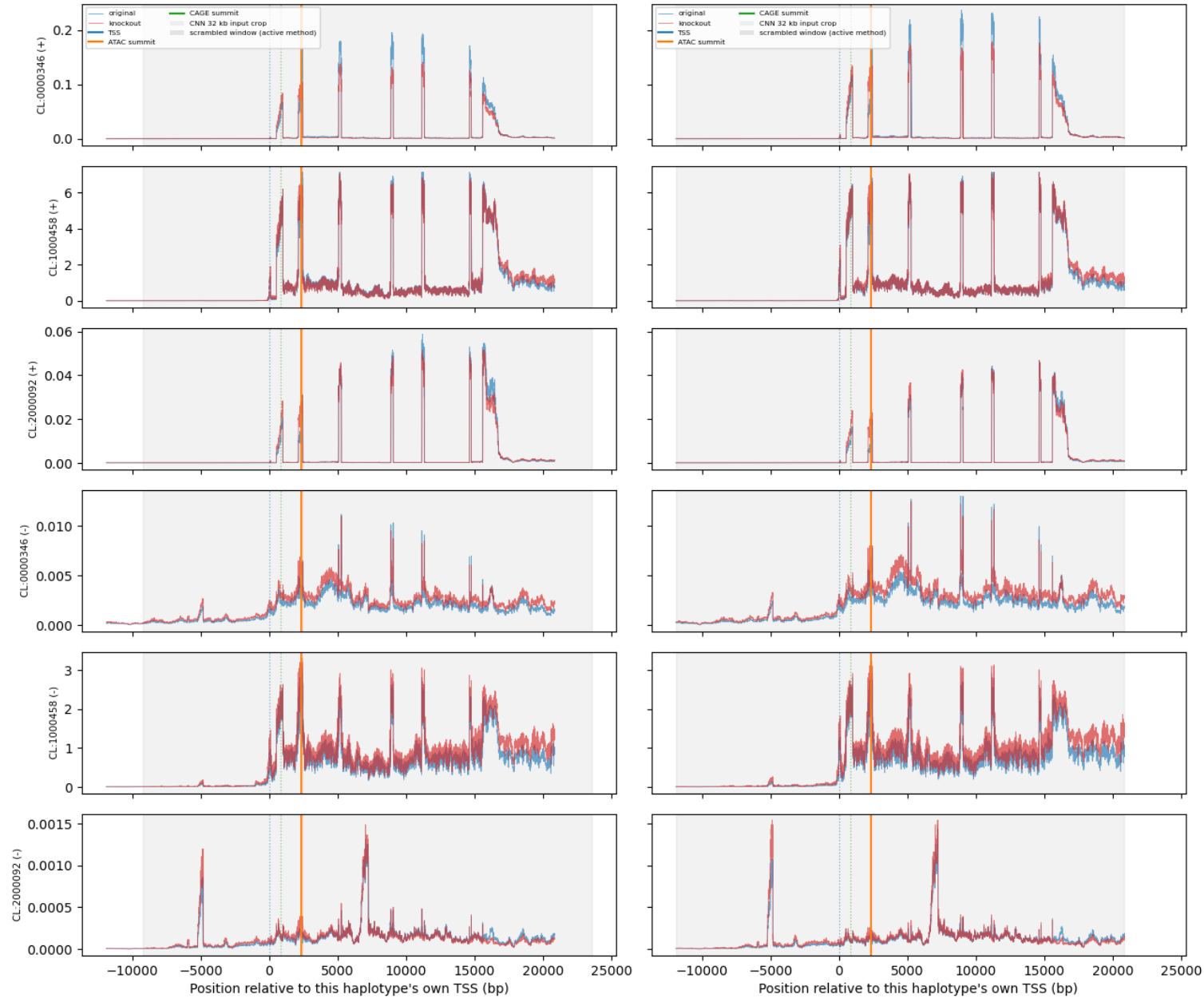
H1 (scrambled -50..50)

H2 (scrambled -50..50)



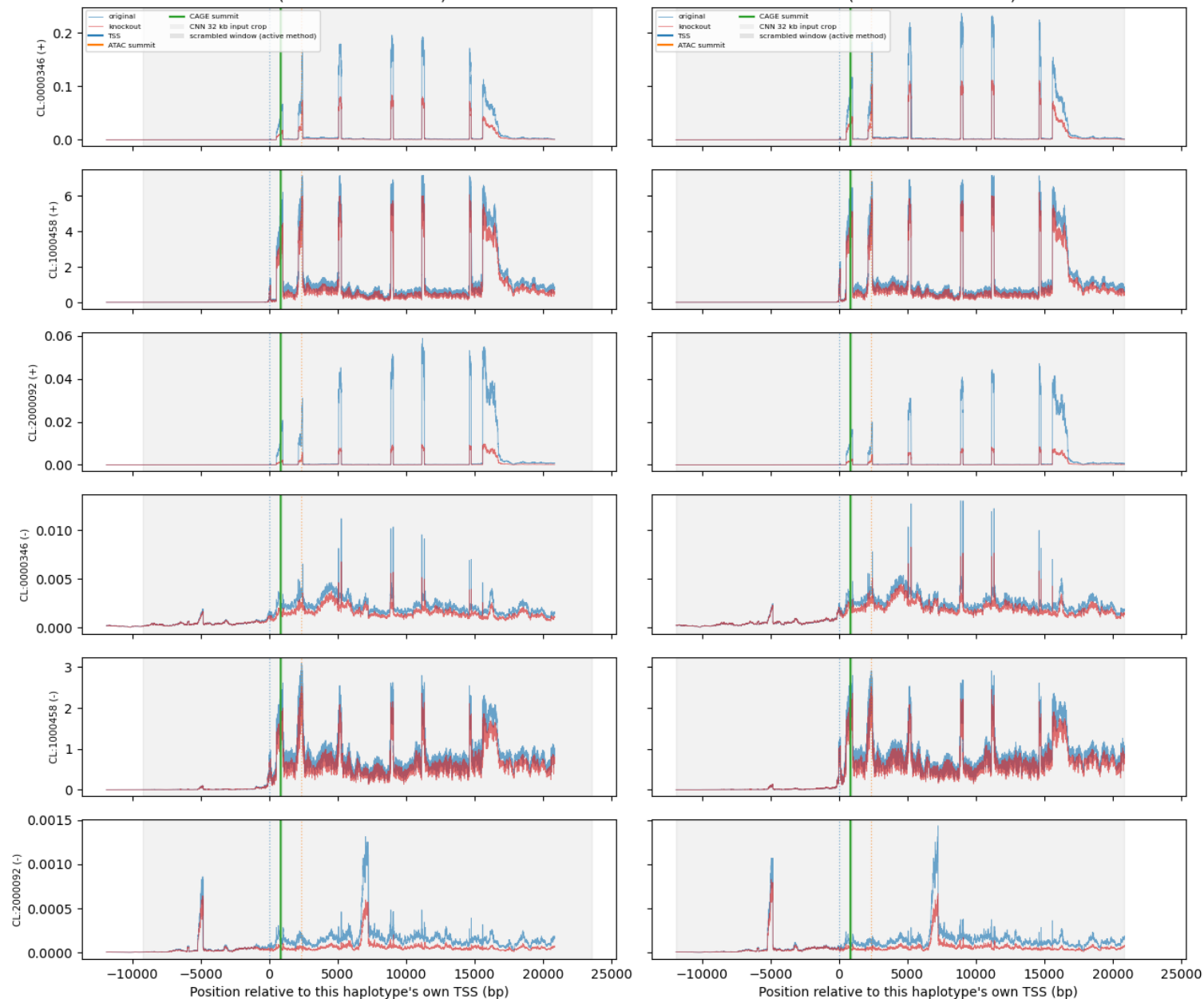
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.48422357 -1.2164335 ], delta(strong pigmentation)=-0.3410  
--- TYRP1 / alphagenome_atac ---
```

TYRP1 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr9:12,695,707 -- delta(strong pigmentation logit) = +0.0717
H1 (scrambled 2272..2372) H2 (scrambled 2272..2372)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.89699006 -2.1929562 ], delta(strong pigmentation)=+0.0717  
--- TYRP1 / cage_melanocyte ---
```

TYRP1 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr9:12,694,229 -- delta(strong pigmentation logit) = +0.2666
H1 (scrambled 794..894) H2 (scrambled 794..894)

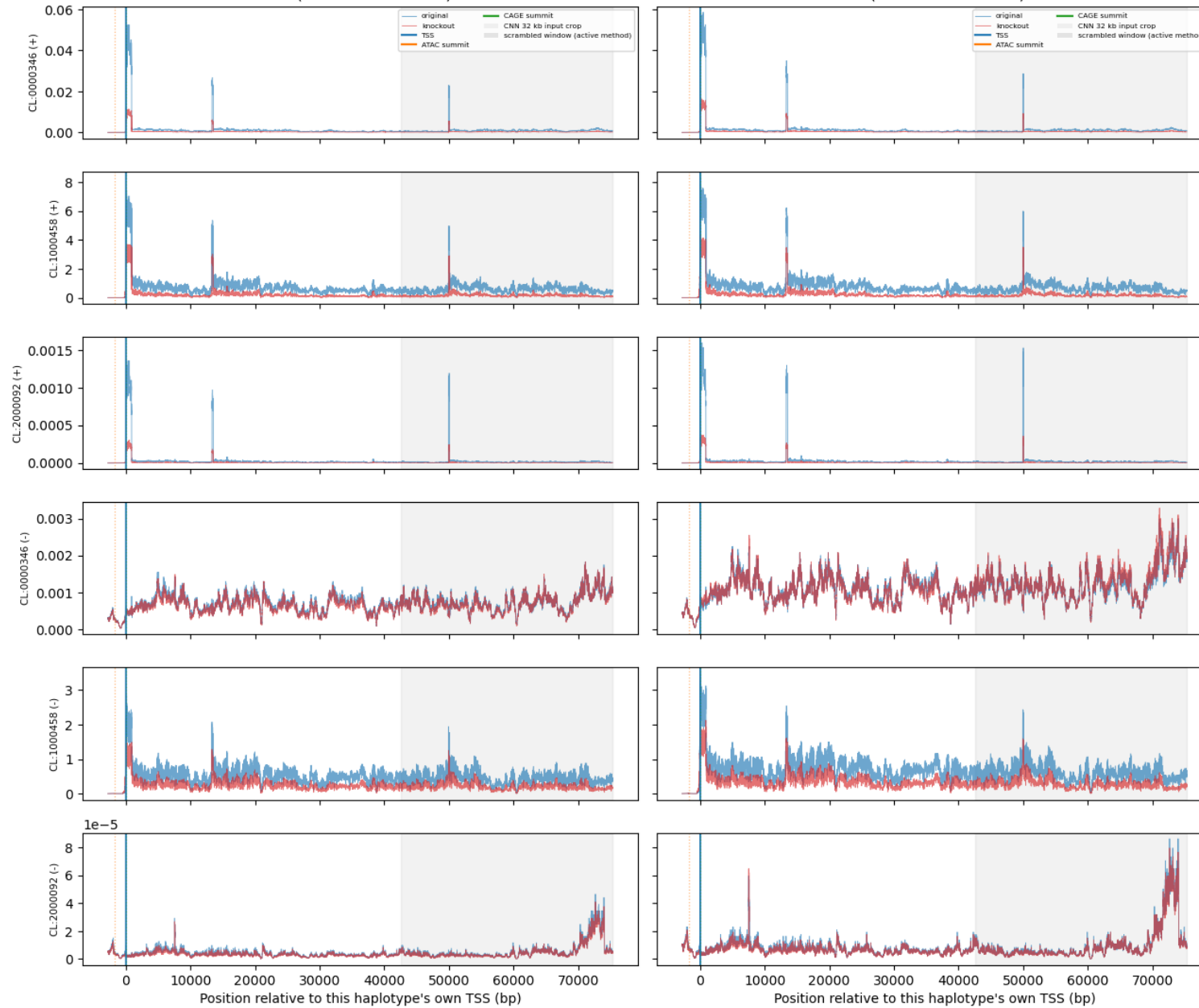


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 1.0918958 -2.7149403], delta(strong pigmentation)=+0.2666  
--- TYR / biology_tss ---
```

TYR -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr11:89,177,875 -- delta(strong pigmentation logit) = +2.6132

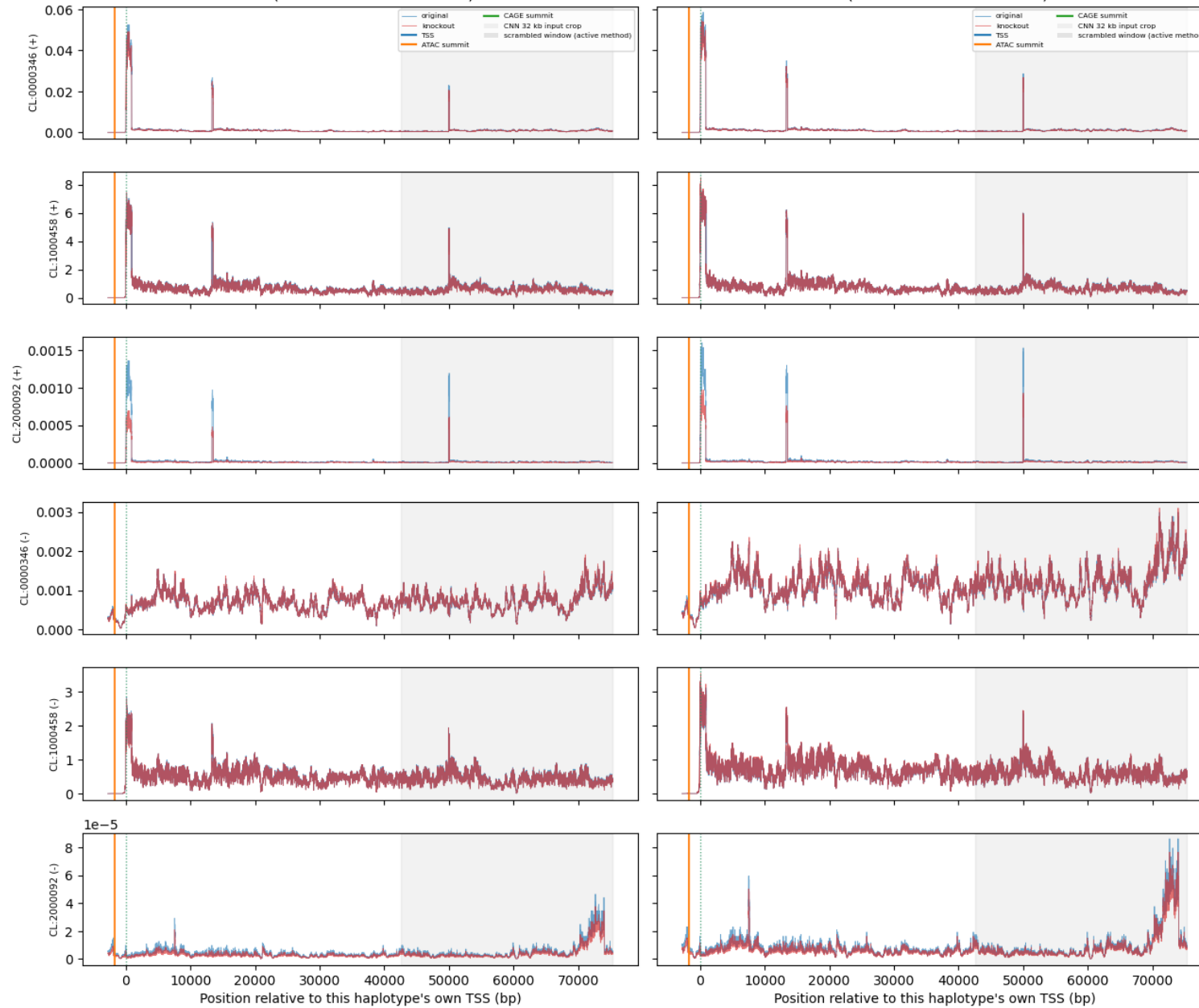
H1 (scrambled -50..50)

H2 (scrambled -50..50)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 3.4384658 -4.7458377], delta(strong pigmentation)=+2.6132
--- TYR / alphagenome_atac ---
```

TYR -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr11:89,176,088 -- delta(strong pigmentation logit) = -0.0104
H1 (scrambled -1838..-1738) H2 (scrambled -1838..-1738)

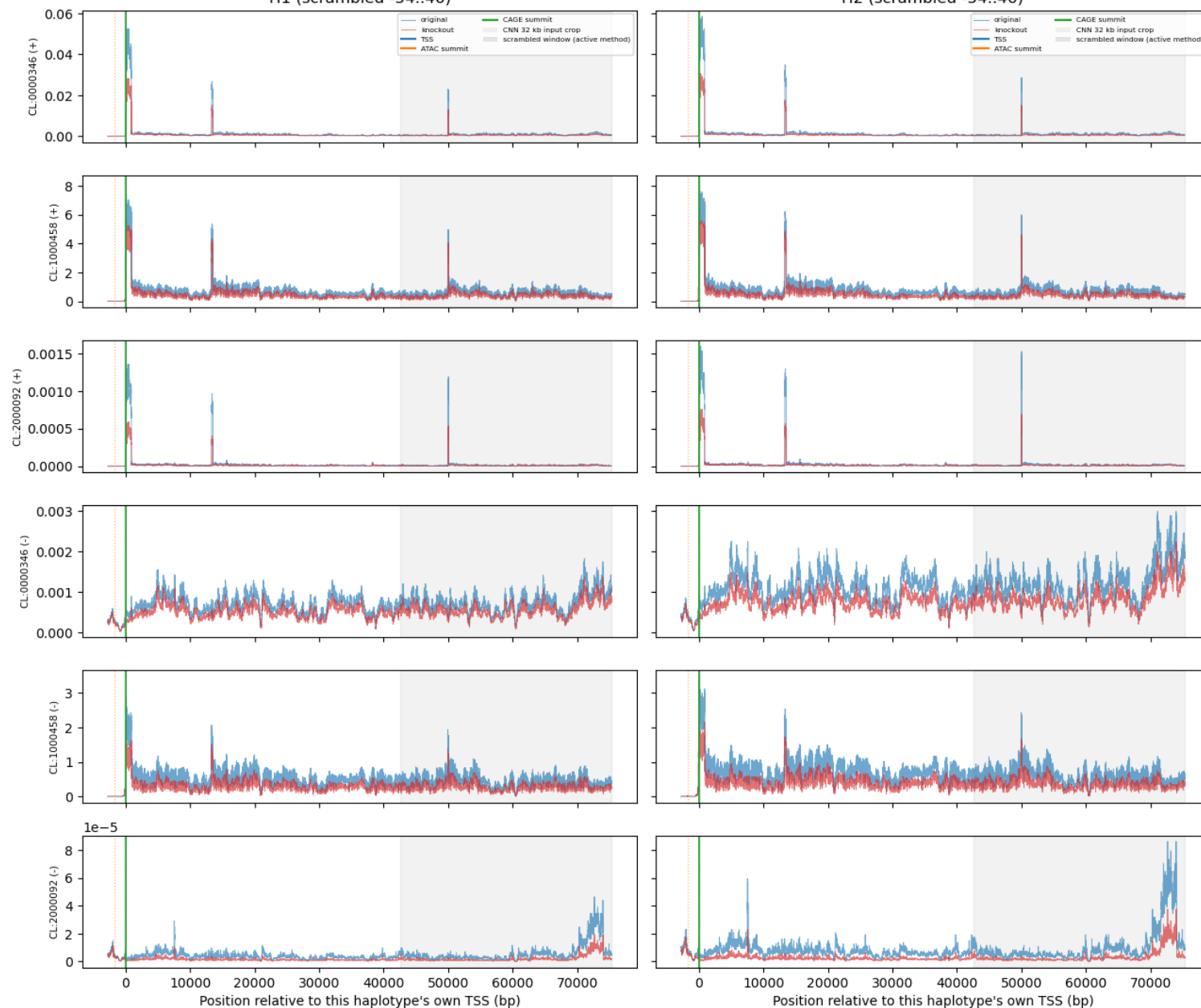


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8148576 -2.0316656], delta(strong pigmentation)=-0.0104  
--- TYR / cage_melanocyte ---
```

TYR -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr11:89,177,871 -- delta(strong pigmentation logit) = +0.7391

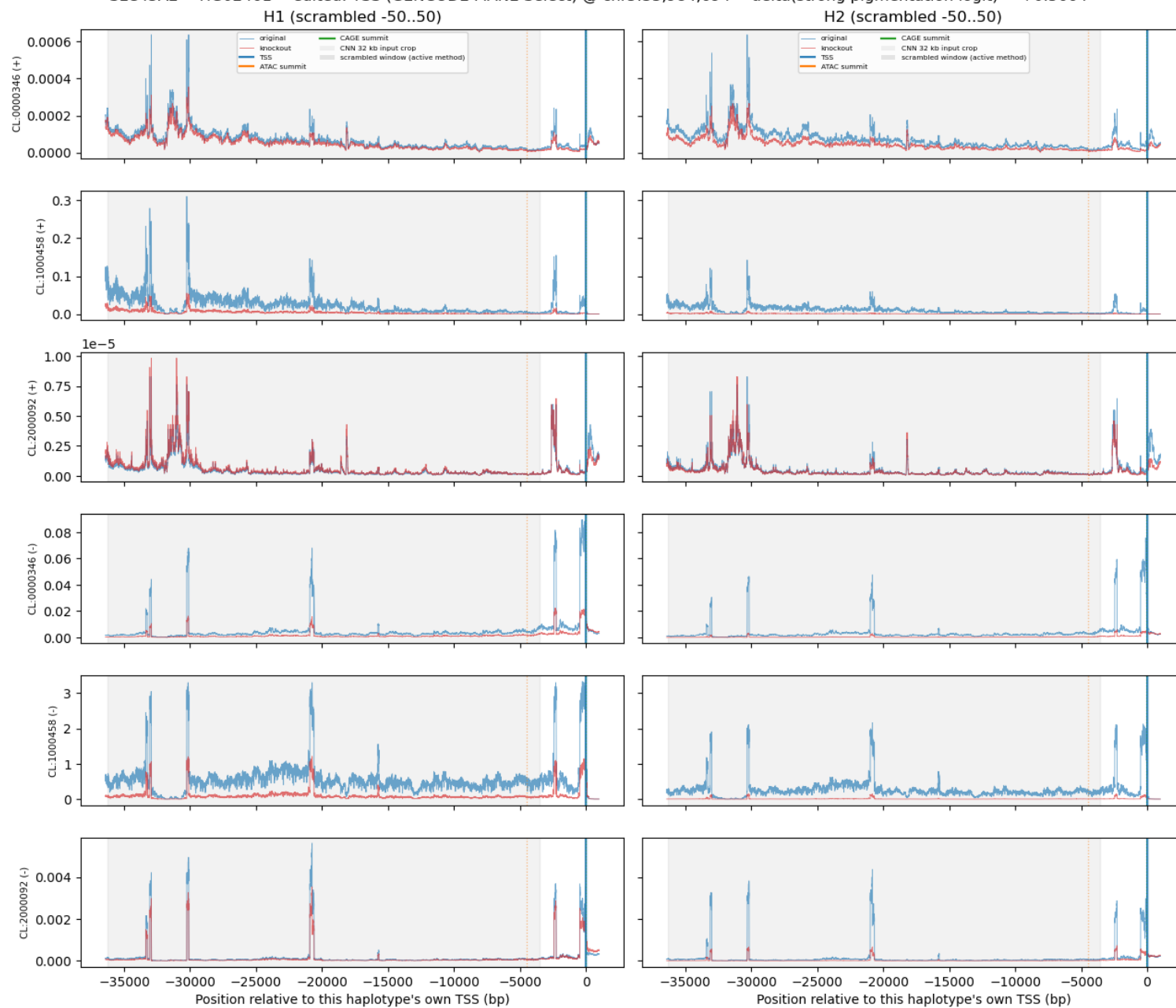
H1 (scrambled -54..46)

H2 (scrambled -54..46)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 1.5643463 -3.6779745], delta(strong pigmentation)=+0.7391
--- SLC45A2 / biology_tss ---
```

SLC45A2 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr5:33,984,694 -- delta(strong pigmentation logit) = +0.5664

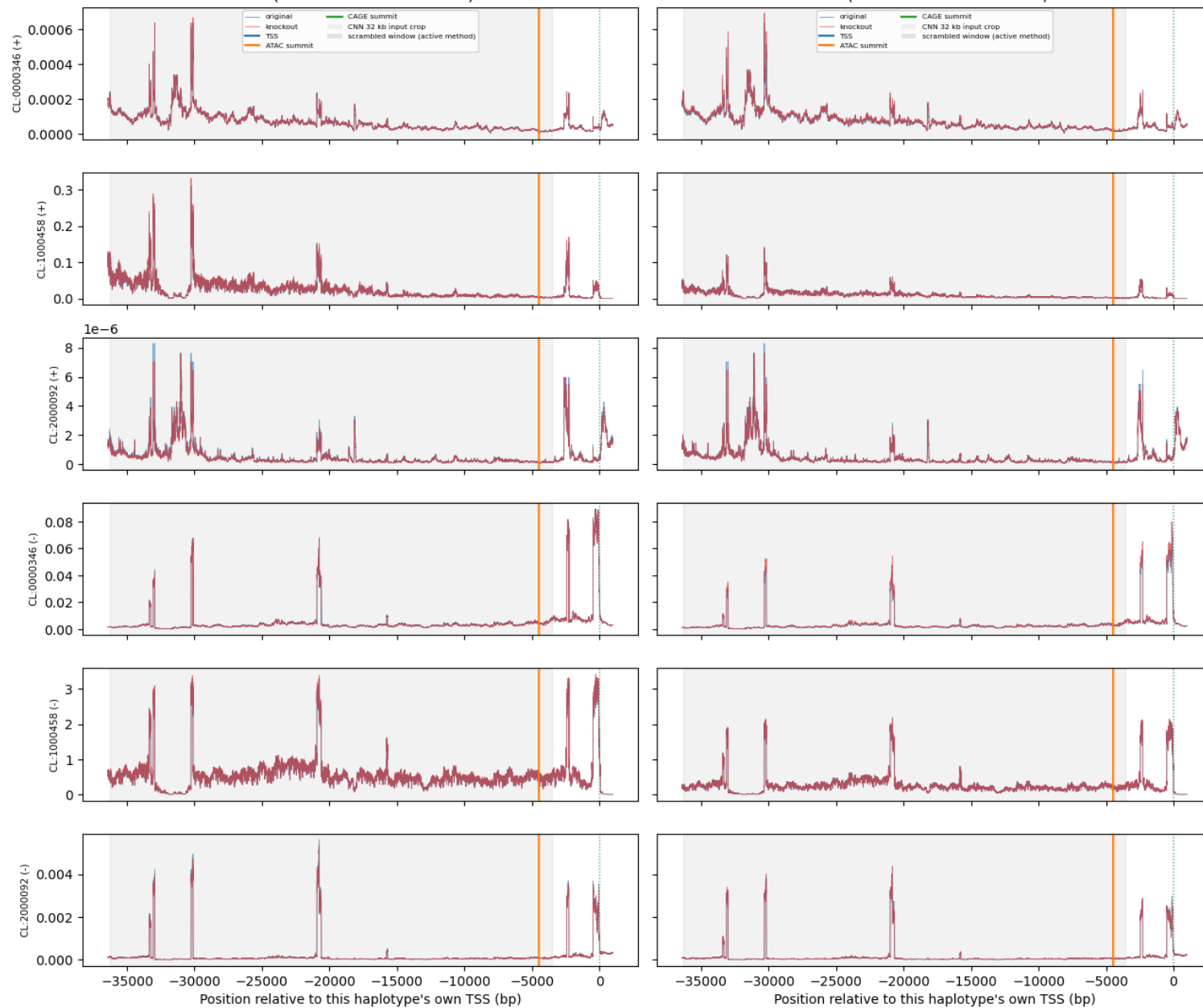


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 1.3916576 -3.5994892], delta(strong pigmentation)=+0.5664  
--- SLC45A2 / alphagenome_atac ---
```

SLC45A2 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr5:33,980,218 -- delta(strong pigmentation logit) = -0.0107

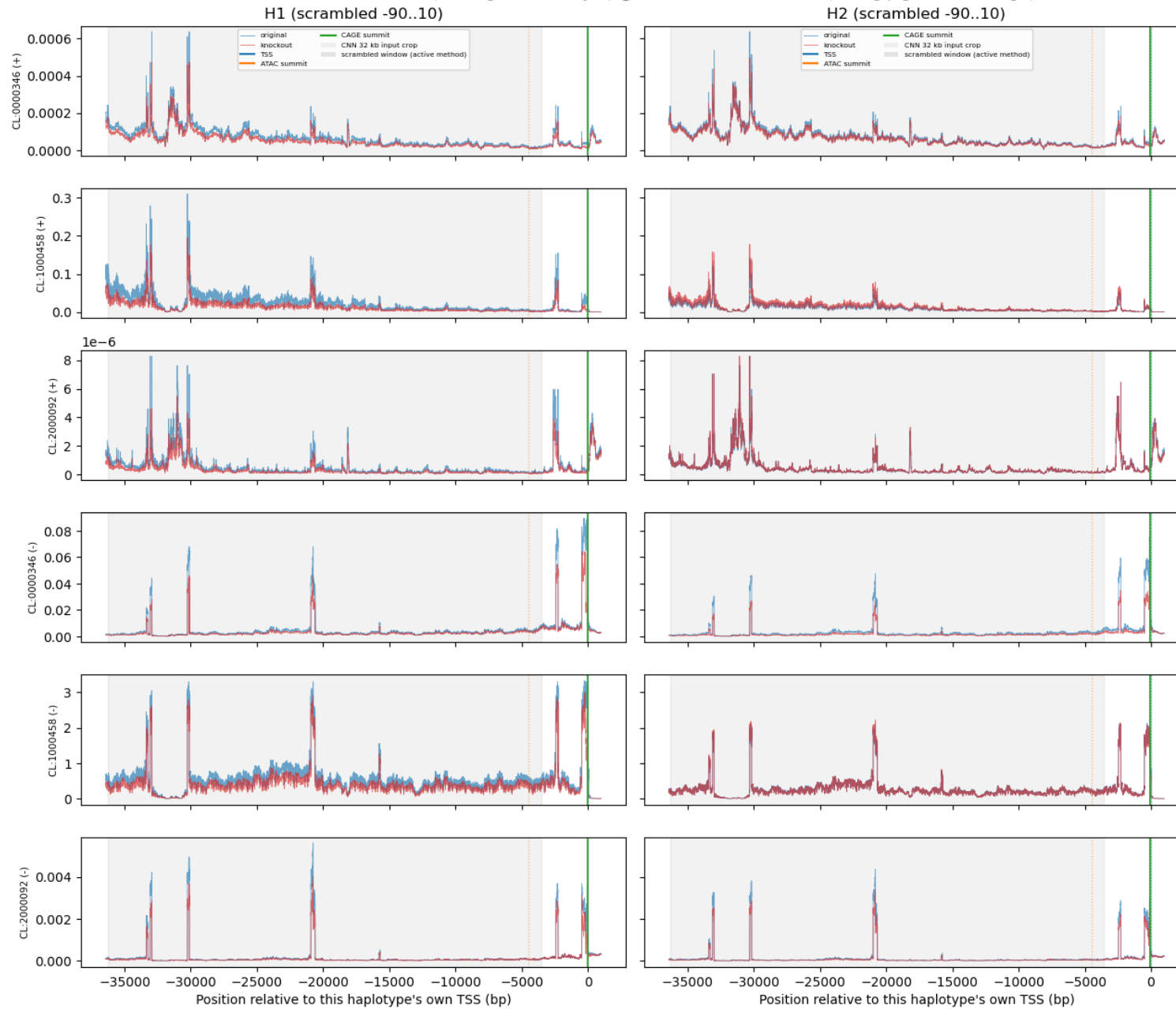
H1 (scrambled -4525..-4425)

H2 (scrambled -4526..-4426)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8145631 -2.0170124], delta(strong pigmentation)=-0.0107
--- SLC45A2 / cage_melanocyte ---
```

SLC45A2 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr5:33,984,654 -- delta(strong pigmentation logit) = +0.1151

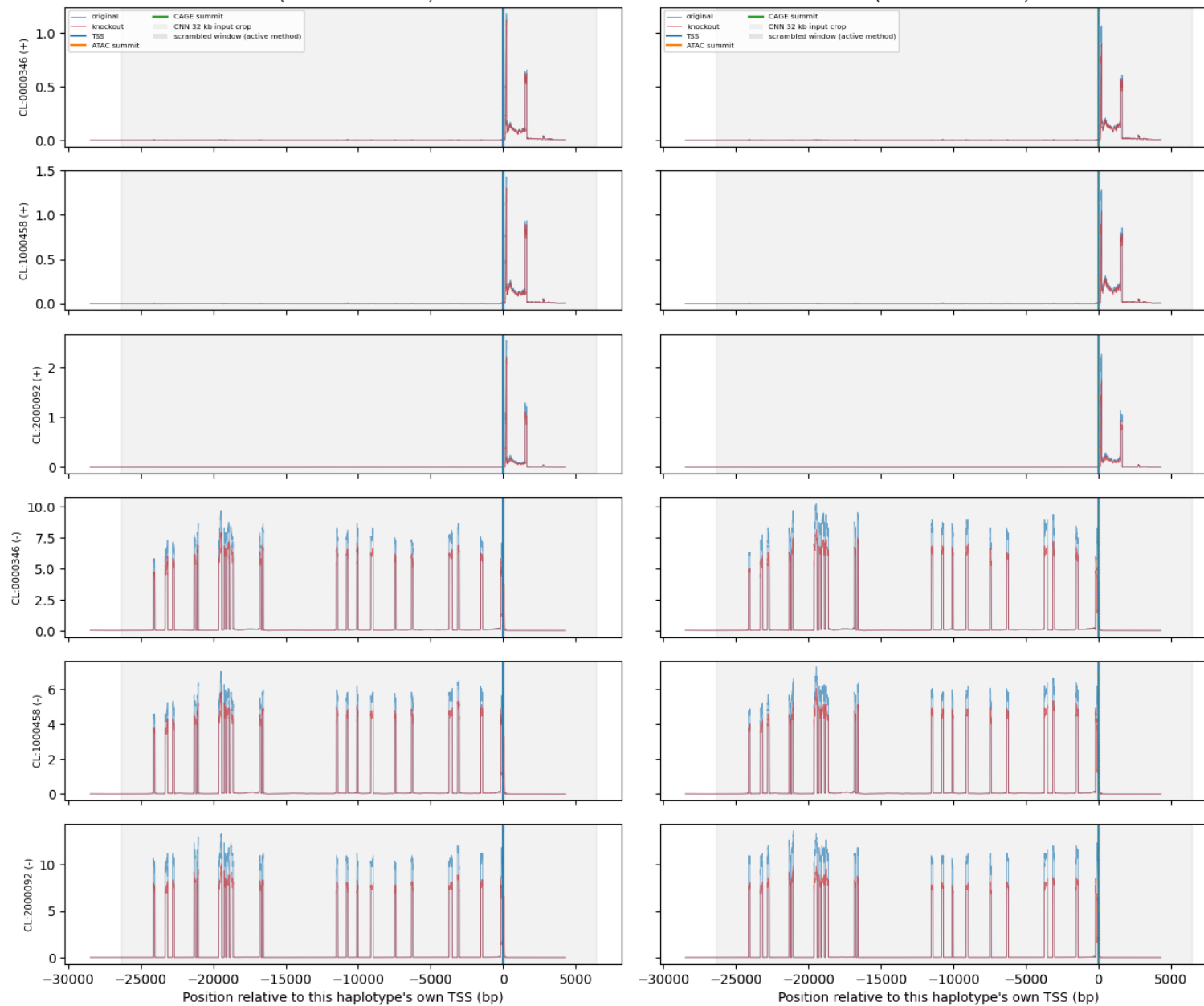


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.94032747 -2.356228 ], delta(strong pigmentation)=+0.1151  
--- DDB1 / biology_tss ---
```

DDB1 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr11:61,333,106 -- delta(strong pigmentation logit) = -0.2836

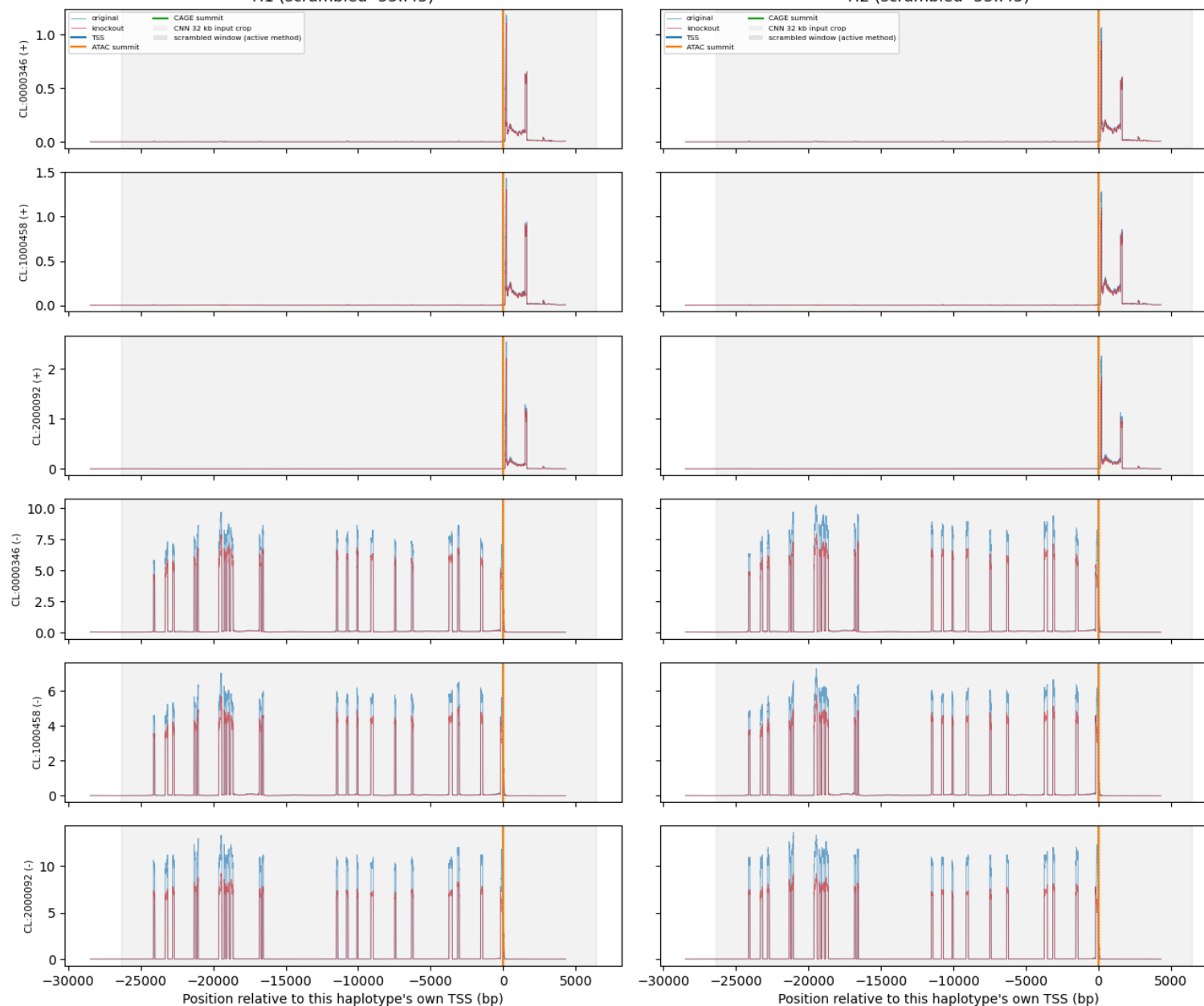
H1 (scrambled -50..50)

H2 (scrambled -50..50)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.54163575 -1.2951831 ], delta(strong pigmentation)=-0.2836
--- DDB1 / alphagenome_atac ---
```

DDB1 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr11:61,333,101 -- delta(strong pigmentation logit) = -0.3861
H1 (scrambled -55..45) H2 (scrambled -55..45)

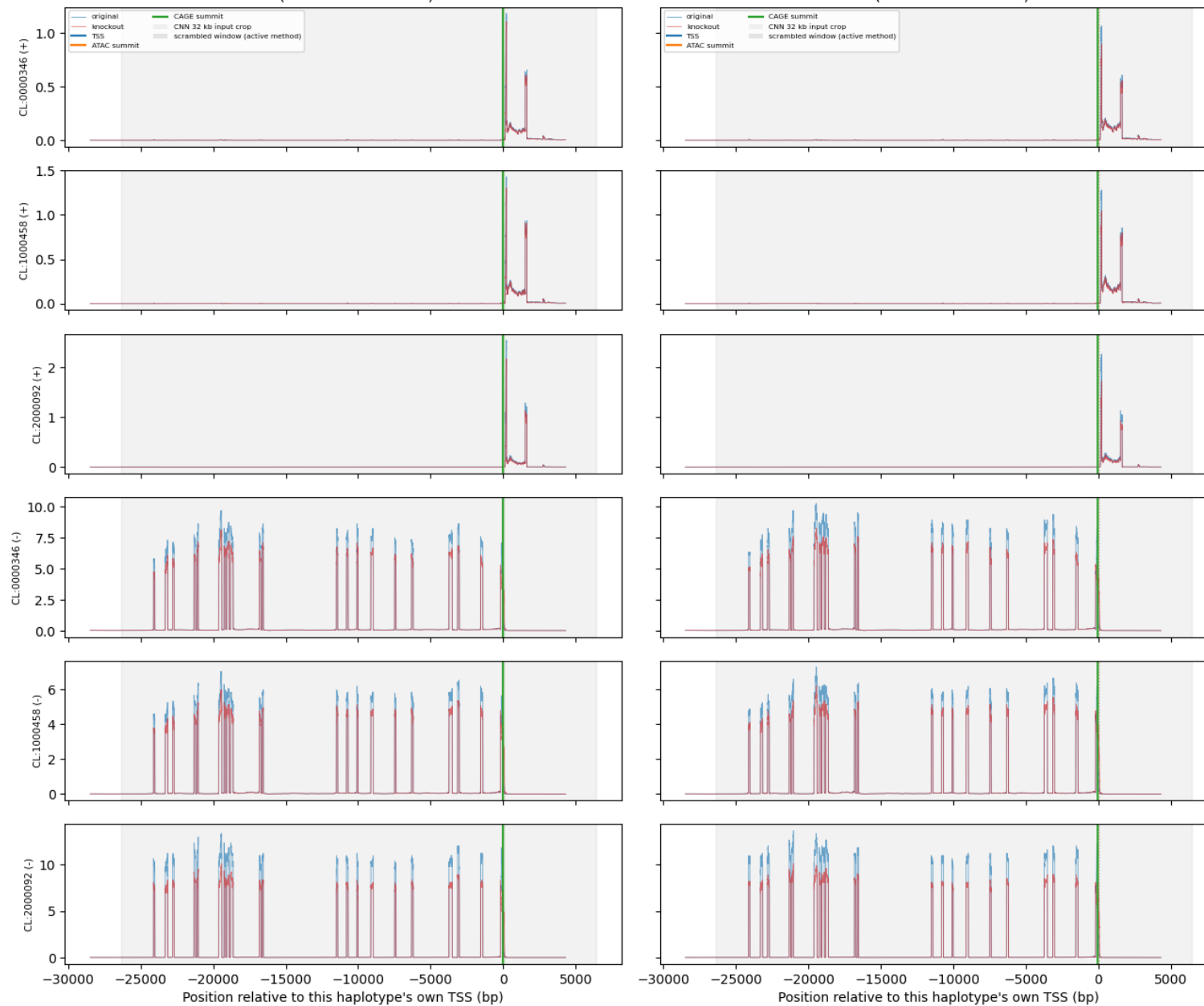


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.43914562 -1.0900538 ], delta(strong pigmentation)=-0.3861
--- DDB1 / cage_melanocyte ---
```

DDB1 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr11:61,333,070 -- delta(strong pigmentation logit) = -0.2163

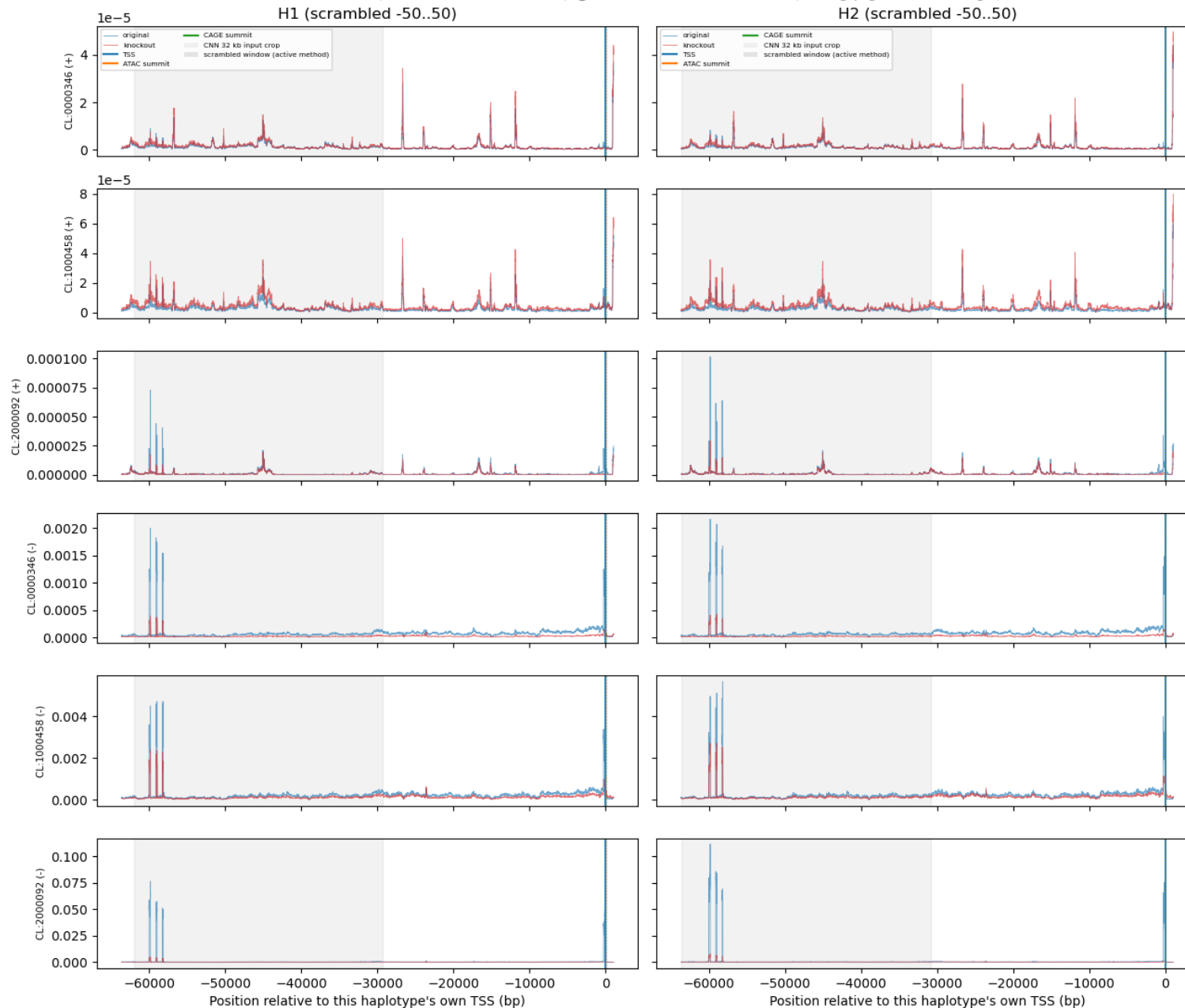
H1 (scrambled -86..14)

H2 (scrambled -86..14)



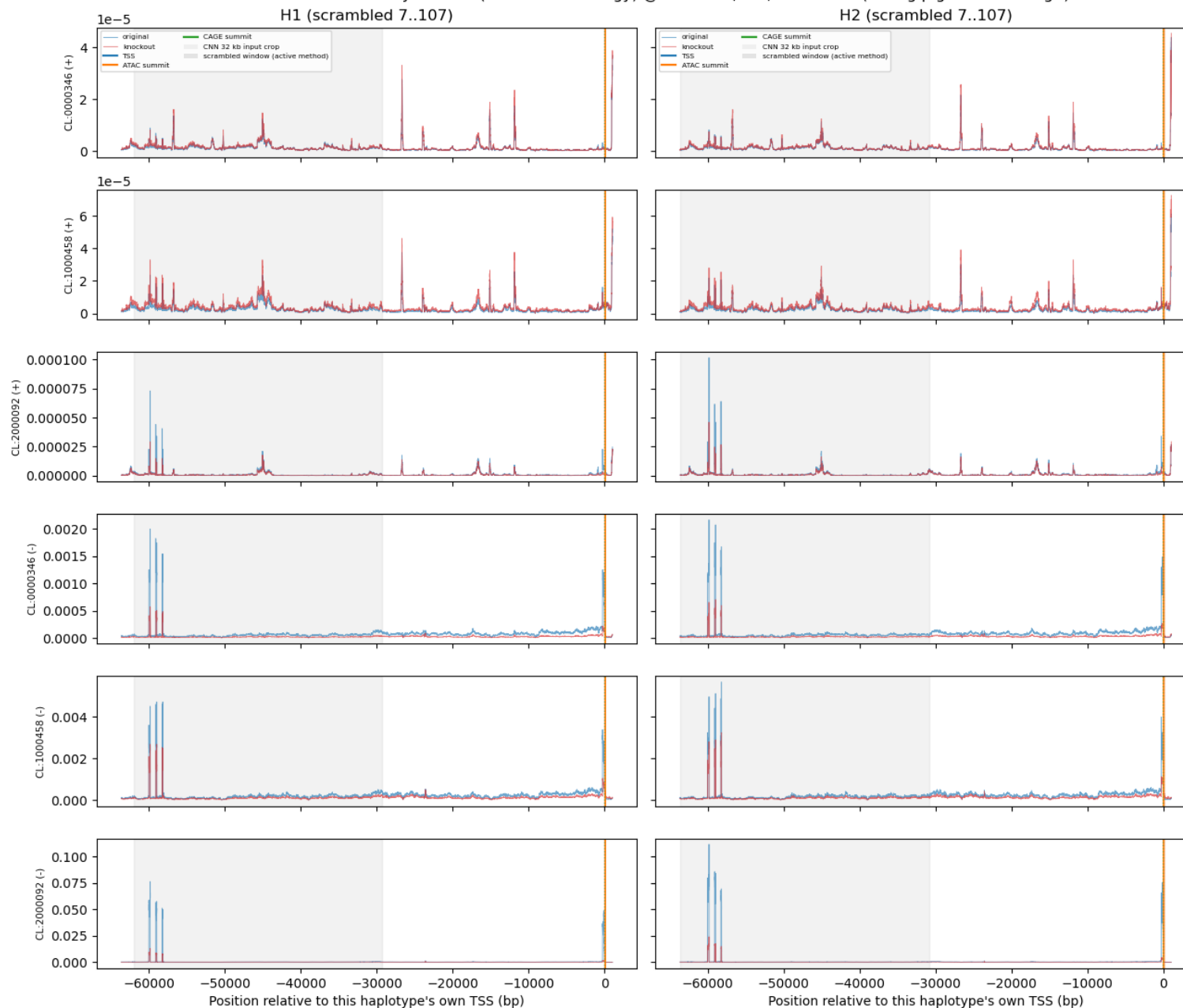
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.60895026 -1.460574 ], delta(strong pigmentation)=-0.2163
--- EDAR / biology_tss ---
```

EDAR -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr2:108,989,221 -- delta(strong pigmentation logit) = +0.0027



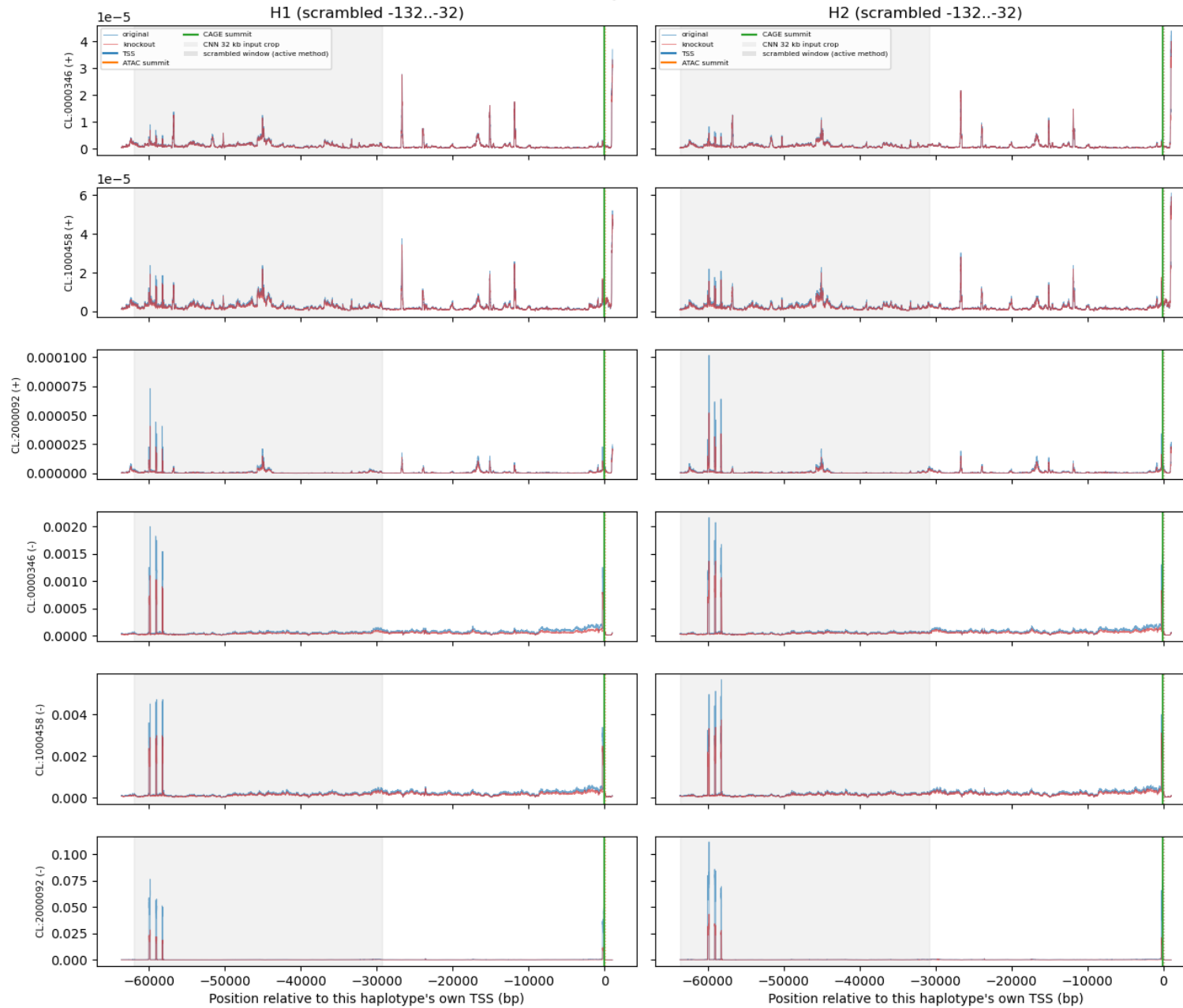
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8279921 -2.0525432], delta(strong pigmentation)=+0.0027
--- EDAR / alphagenome_atac ---
```

EDAR -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr2:108,989,278 -- delta(strong pigmentation logit) = +0.0026



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8278597 -2.052236 ], delta(strong pigmentation)=+0.0026
--- EDAR / cage_melanocyte ---
```

EDAR -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr2:108,989,139 -- delta(strong pigmentation logit) = +0.0021

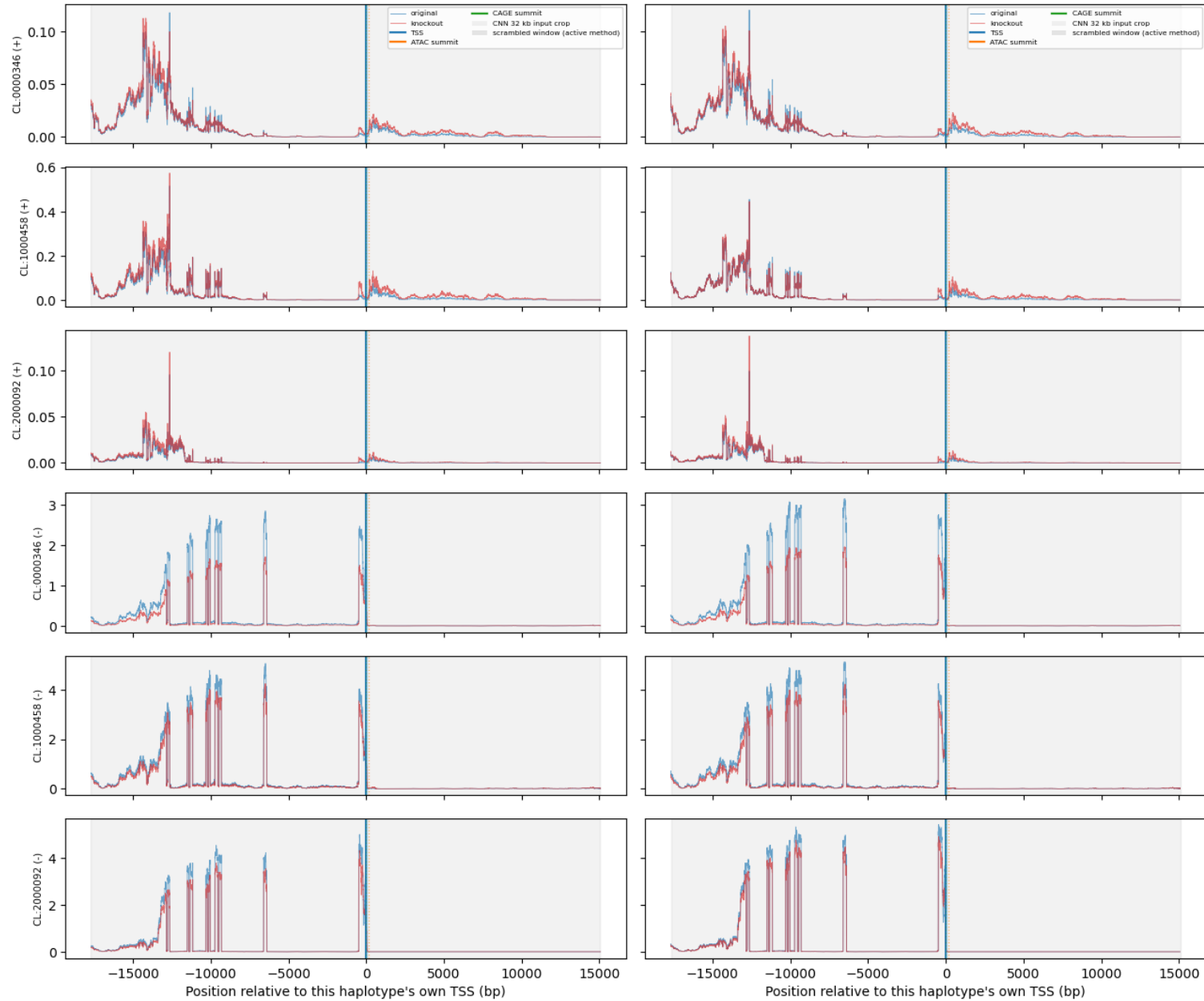


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8273356 -2.050997 ], delta(strong pigmentation)=+0.0021
--- MFSD12 / biology_tss ---
```

MFSD12 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr19:3,557,587 -- delta(strong pigmentation logit) = +0.4492

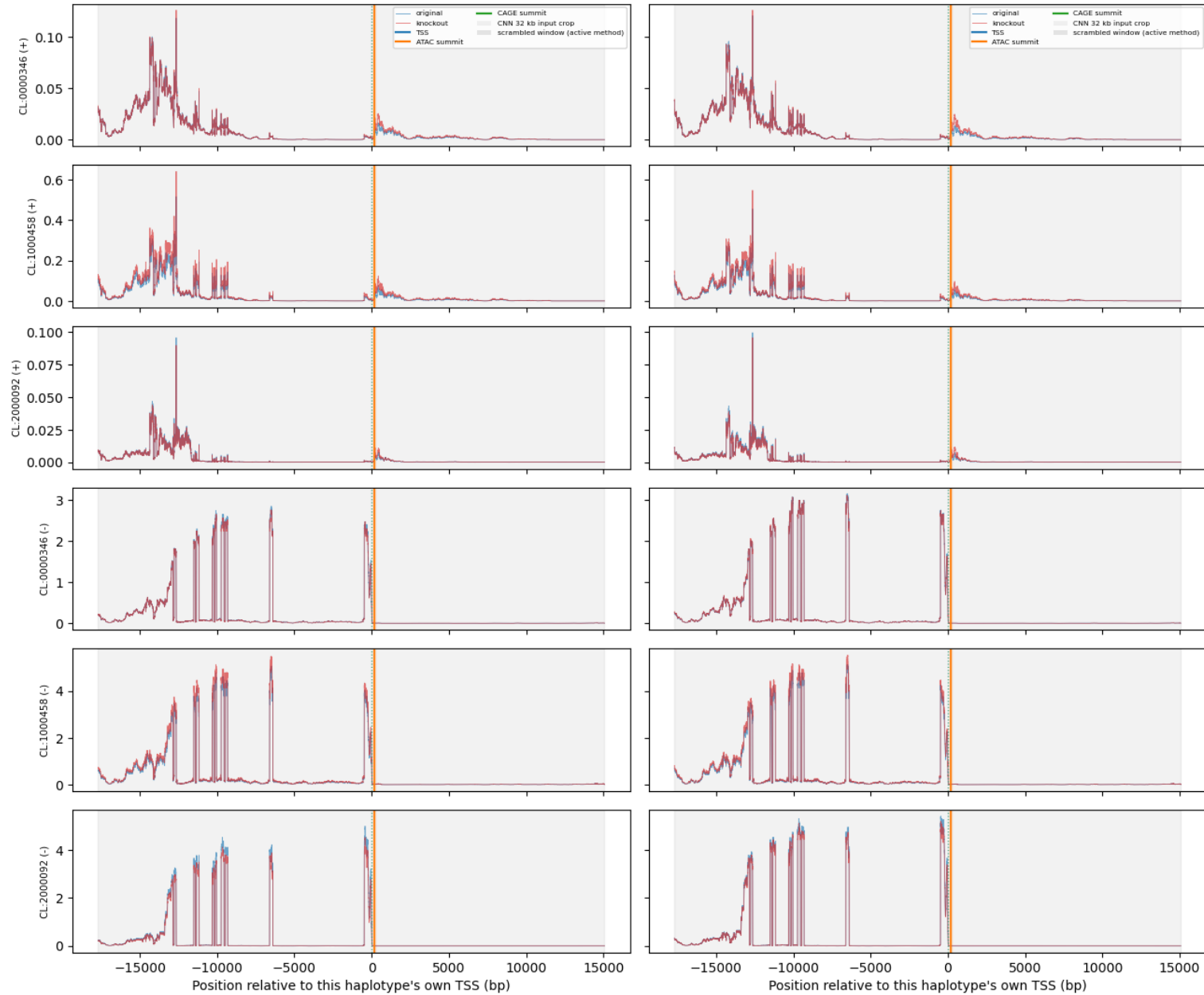
H1 (scrambled -50..50)

H2 (scrambled -50..50)



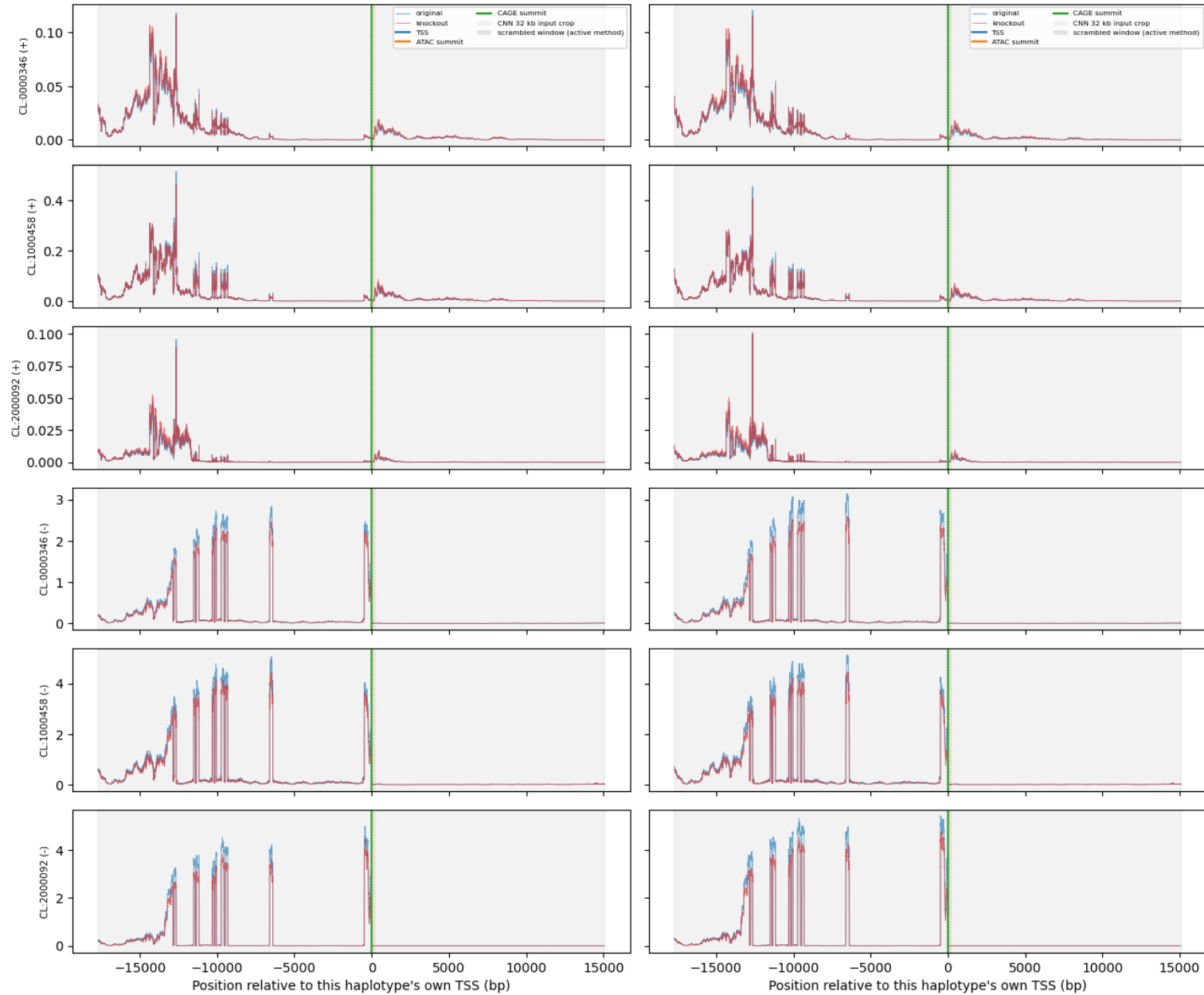
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 1.2744968 -3.1285183], delta(strong pigmentation)=+0.4492
--- MFSD12 / alphagenome_atac ---
```

MFSD12 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr19:3,557,762 -- delta(strong pigmentation logit) = -0.1331
H1 (scrambled 125..225) H2 (scrambled 125..225)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.6921647 -1.7254426], delta(strong pigmentation)=-0.1331
--- MFSD12 / cage_melanocyte ---
```

MFSD12 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr19:3,557,584 -- delta(strong pigmentation logit) = +0.2804
H1 (scrambled -53..47) H2 (scrambled -53..47)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 1.1056217 -2.7191036], delta(strong pigmentation)=+0.2804  
--- OCA2 / biology_tss ---
```

OCA2 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr15:28,099,316 -- delta(strong pigmentation logit) = +0.0022

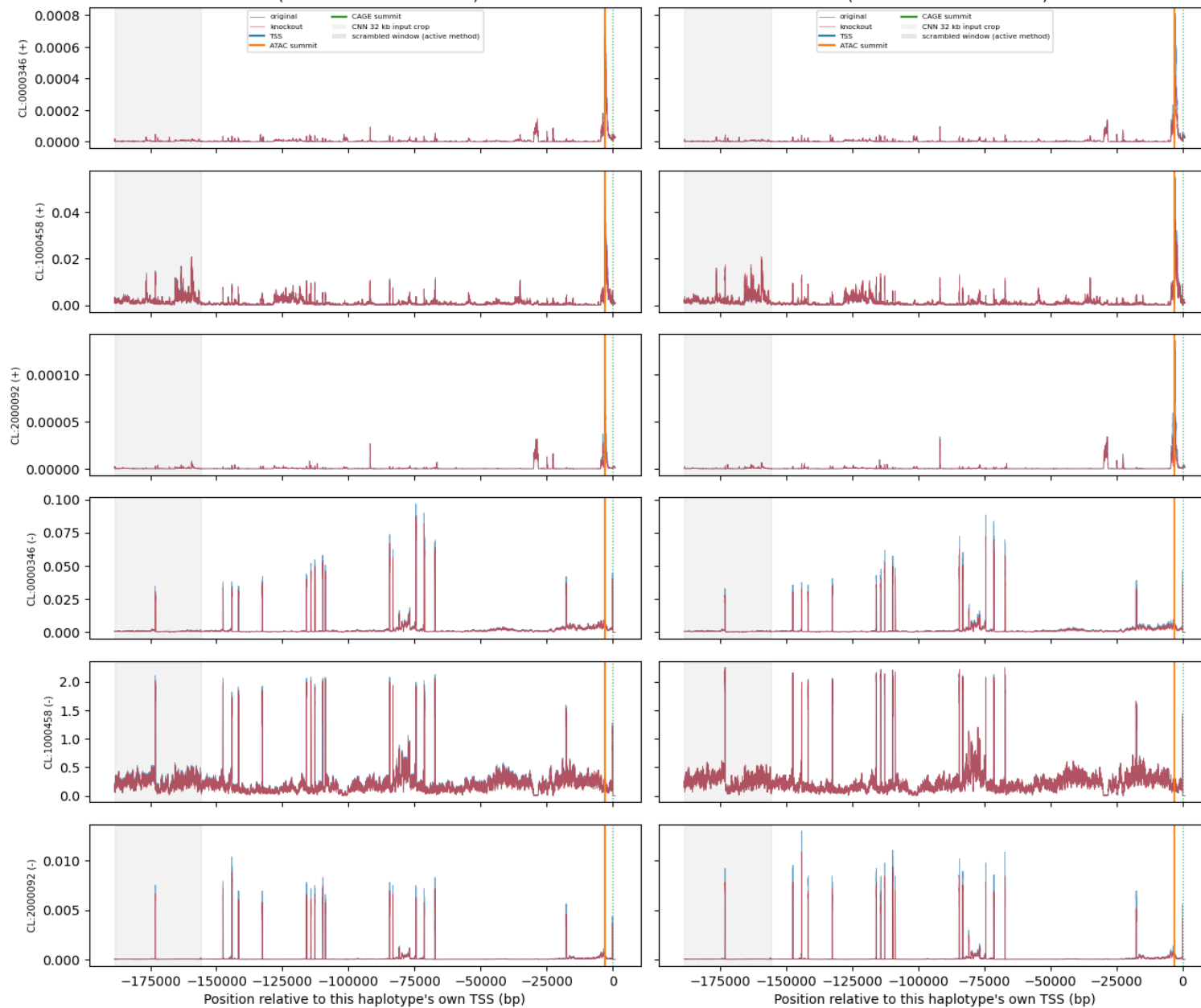


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.82745 -2.0570312], delta(strong pigmentation)=+0.0022
--- OCA2 / alphagenome_atac ---
```

OCA2 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr15:28,096,239 -- delta(strong pigmentation logit) = +0.0029

H1 (scrambled -3127...-3027)

H2 (scrambled -3127...-3027)

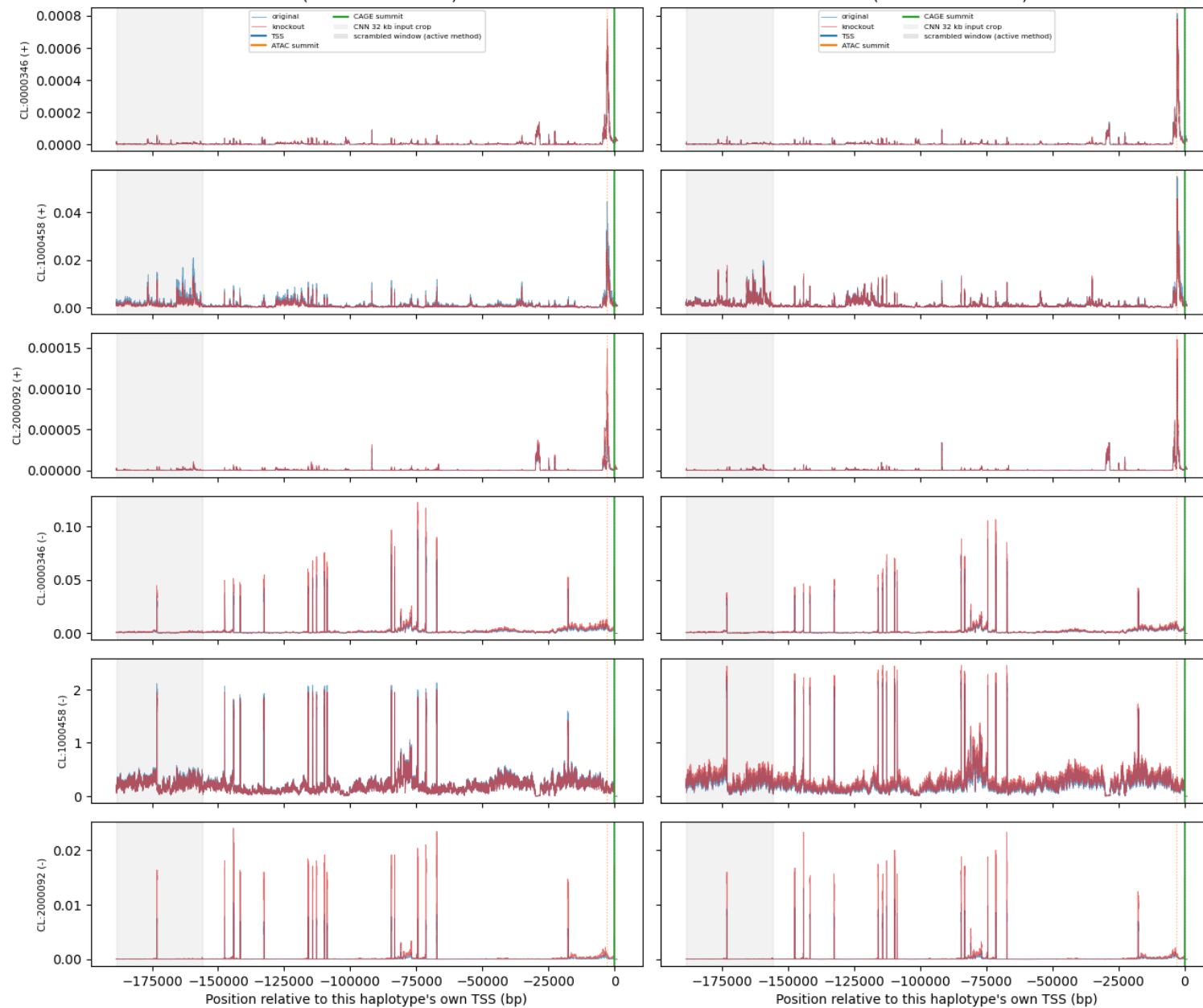


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.82815266 -2.0601437 ], delta(strong pigmentation)=+0.0029
--- OCA2 / cage_melanocyte ---
```

OCA2 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr15:28,099,317 -- delta(strong pigmentation logit) = +0.0033

H1 (scrambled -49..51)

H2 (scrambled -49..51)

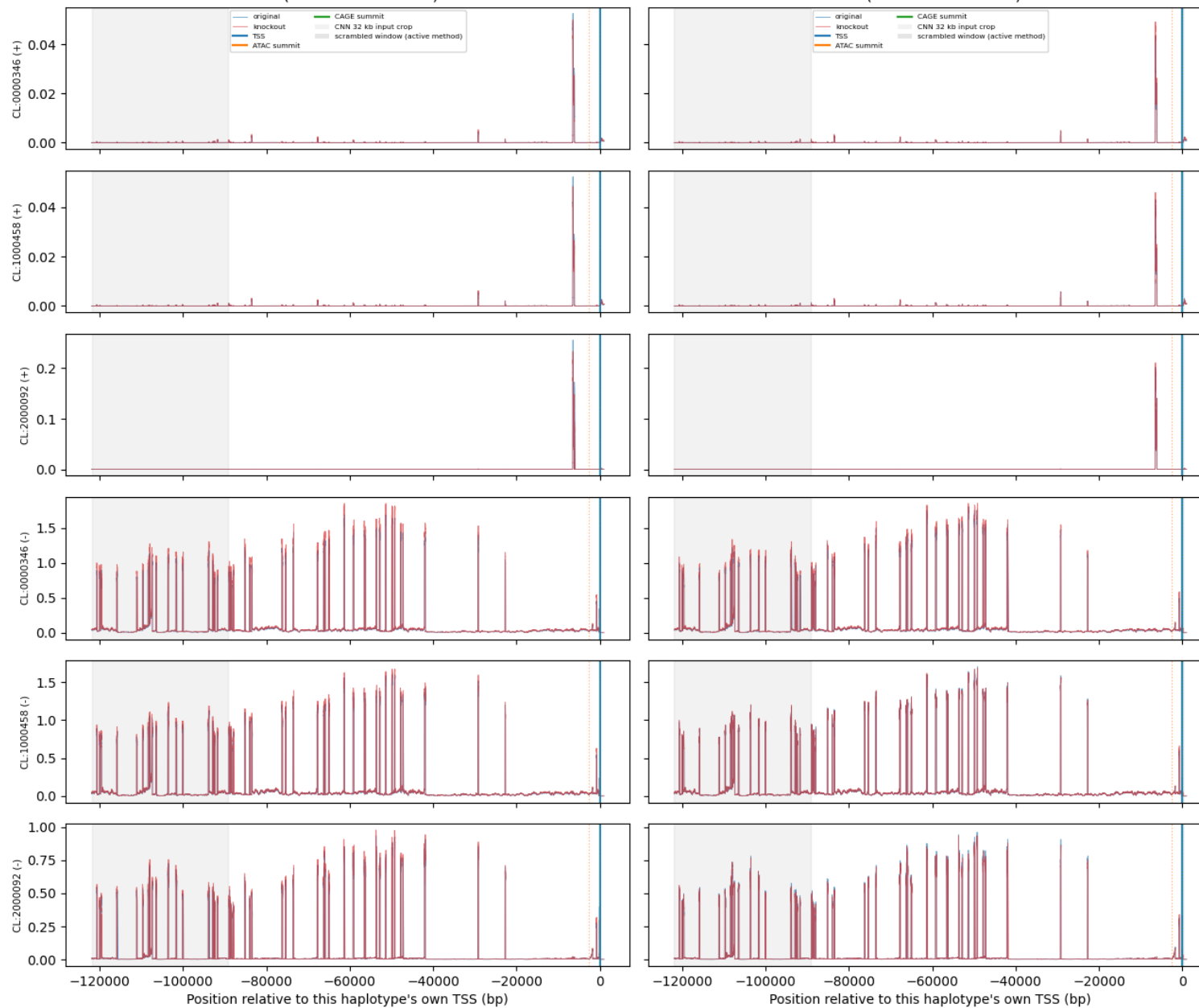


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.82860553 -2.0714202 ], delta(strong pigmentation)=+0.0033
--- HERC2 / biology_tss ---
```

HERC2 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr15:28,322,180 -- delta(strong pigmentation logit) = +0.0356

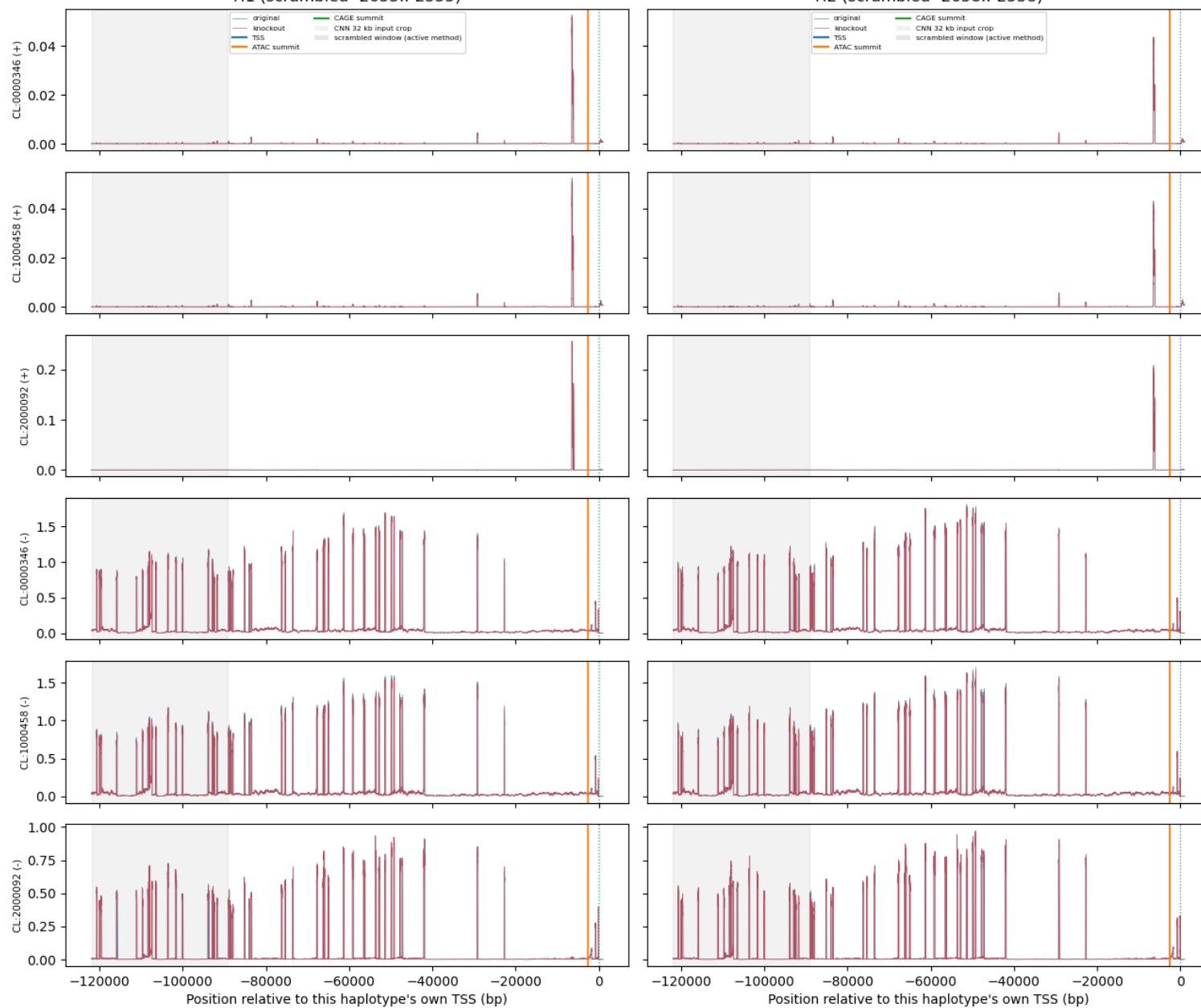
H1 (scrambled -50..50)

H2 (scrambled -50..50)



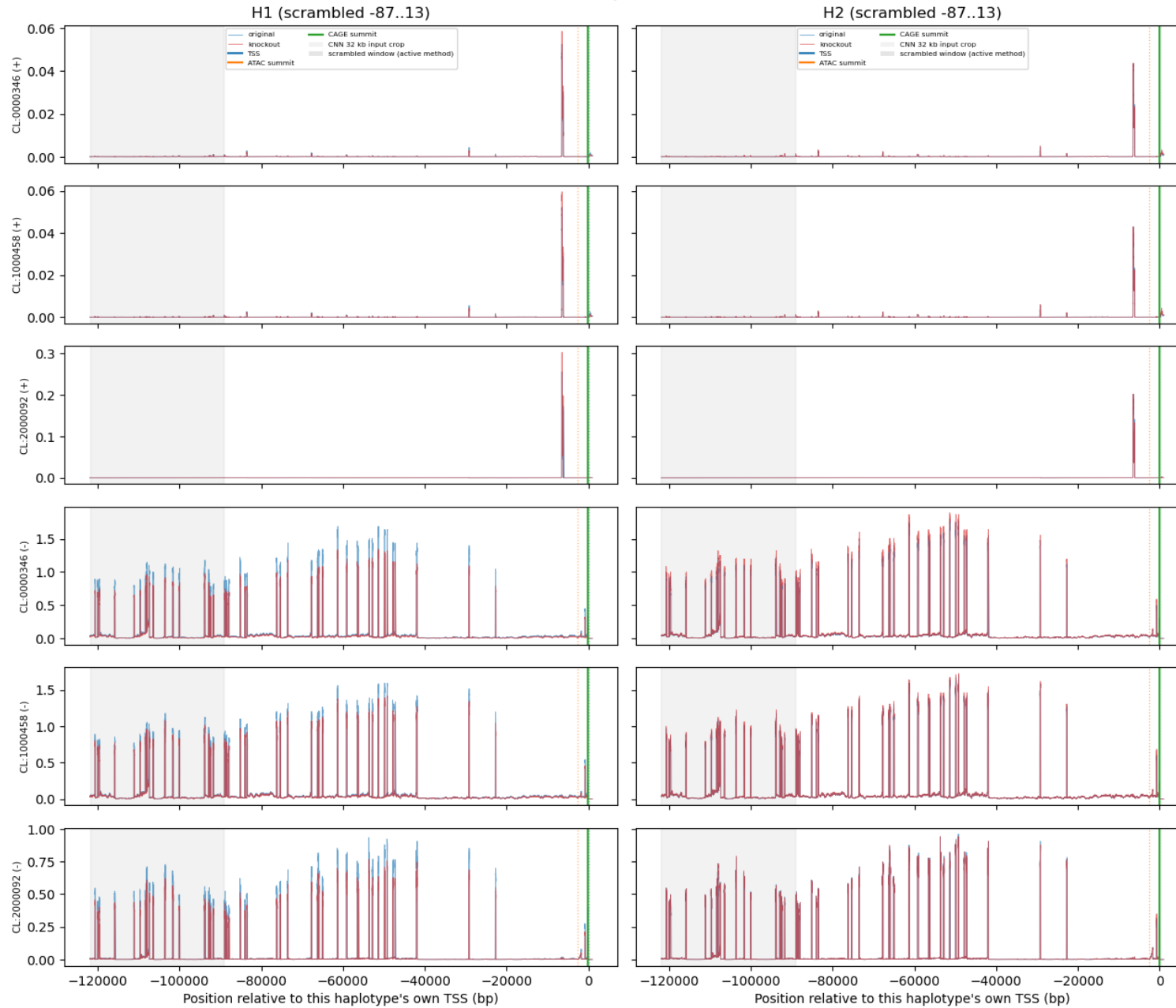
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8608676 -2.1410556], delta(strong pigmentation)=+0.0356
--- HERC2 / alphagenome_atac ---
```

HERC2 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr15:28,319,572 -- delta(strong pigmentation logit) = -0.0114
H1 (scrambled -2655..-2555) H2 (scrambled -2658..-2558)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.813892 -2.0154276], delta(strong pigmentation)=-0.0114
--- HERC2 / cage_melanocyte ---
```

HERC2 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr15:28,322,143 -- delta(strong pigmentation logit) = -0.0381

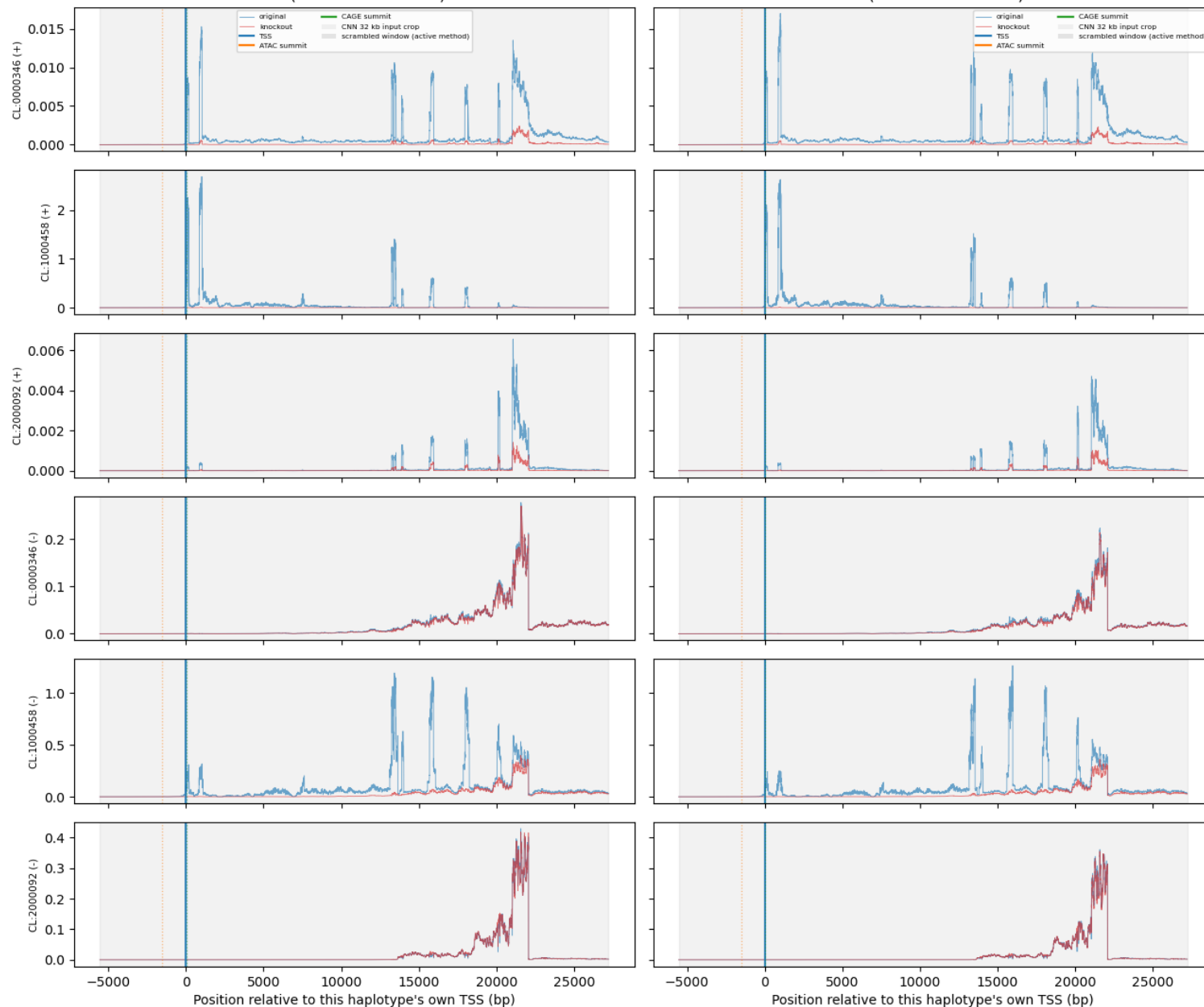


```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.7871533 -1.9505371], delta(strong pigmentation)=-0.0381
--- SLC24A5 / biology_tss ---
```

SLC24A5 -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr15:48,120,990 -- delta(strong pigmentation logit) = -1.6093

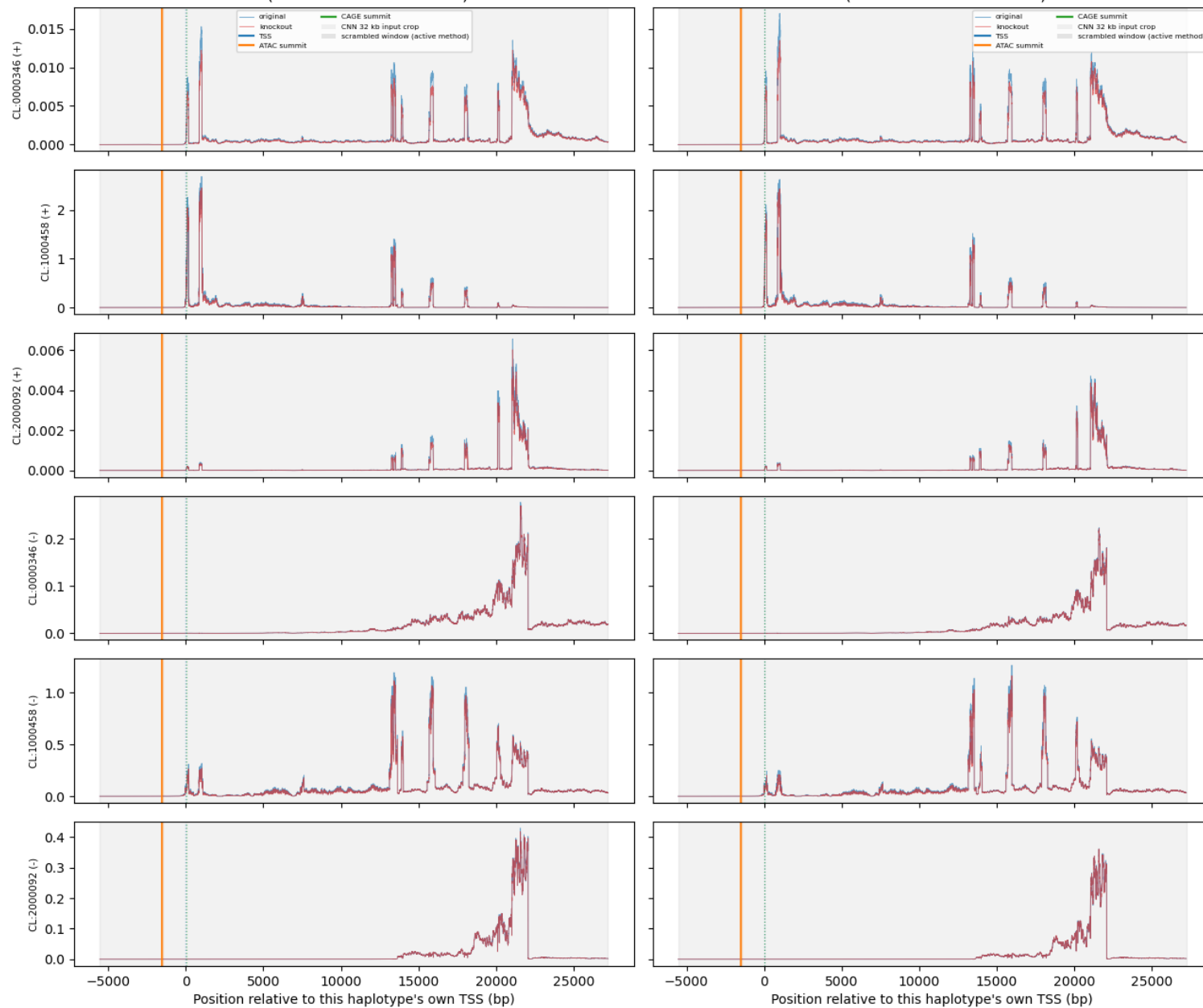
H1 (scrambled -50..50)

H2 (scrambled -50..50)



baseline logits: [0.8252655 -2.046092], perturbed logits: [-0.7840563 0.79424477], delta(strong pigmentation)=-1.6093
--- SLC24A5 / alphagenome_atac ---

SLC24A5 -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr15:48,119,479 -- delta(strong pigmentation logit) = -0.1389

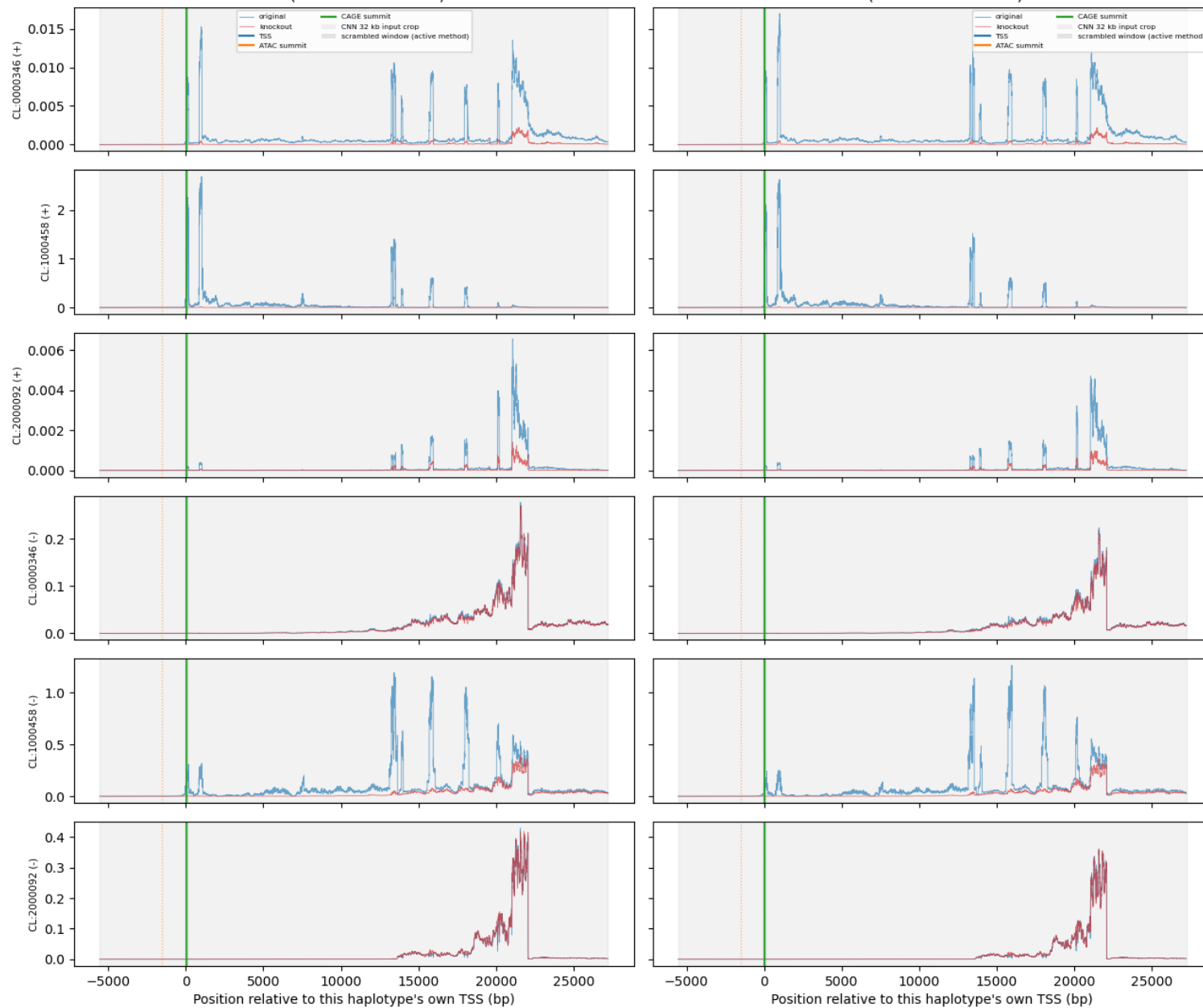


baseline logits: [0.8252655 -2.046092], perturbed logits: [0.68638974 -1.6835274], delta(strong pigmentation)=-0.1389
--- SLC24A5 / cage_melanocyte ---

SLC24A5 -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr15:48,121,018 -- delta(strong pigmentation logit) = -1.5998

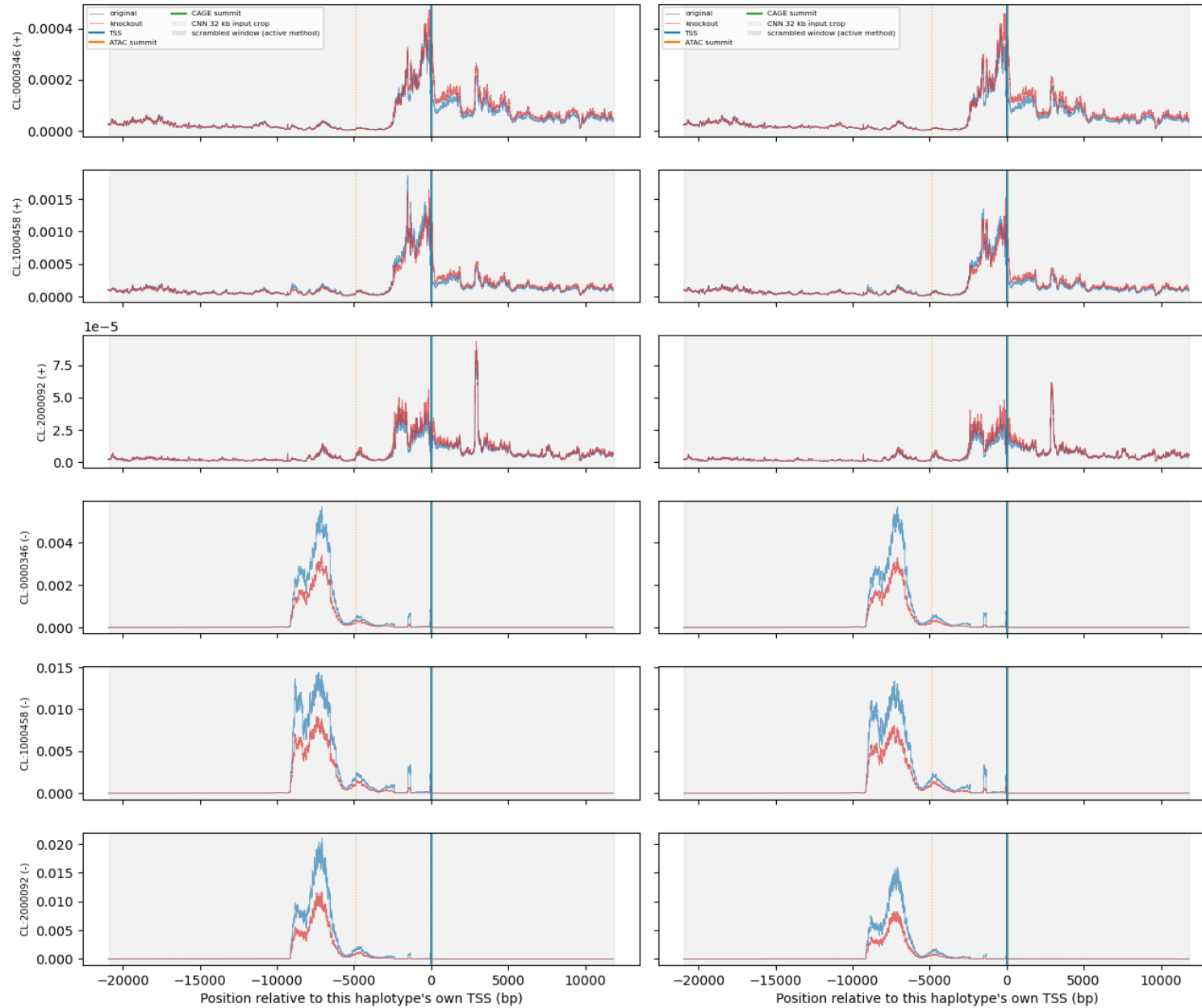
H1 (scrambled -22..78)

H2 (scrambled -22..78)



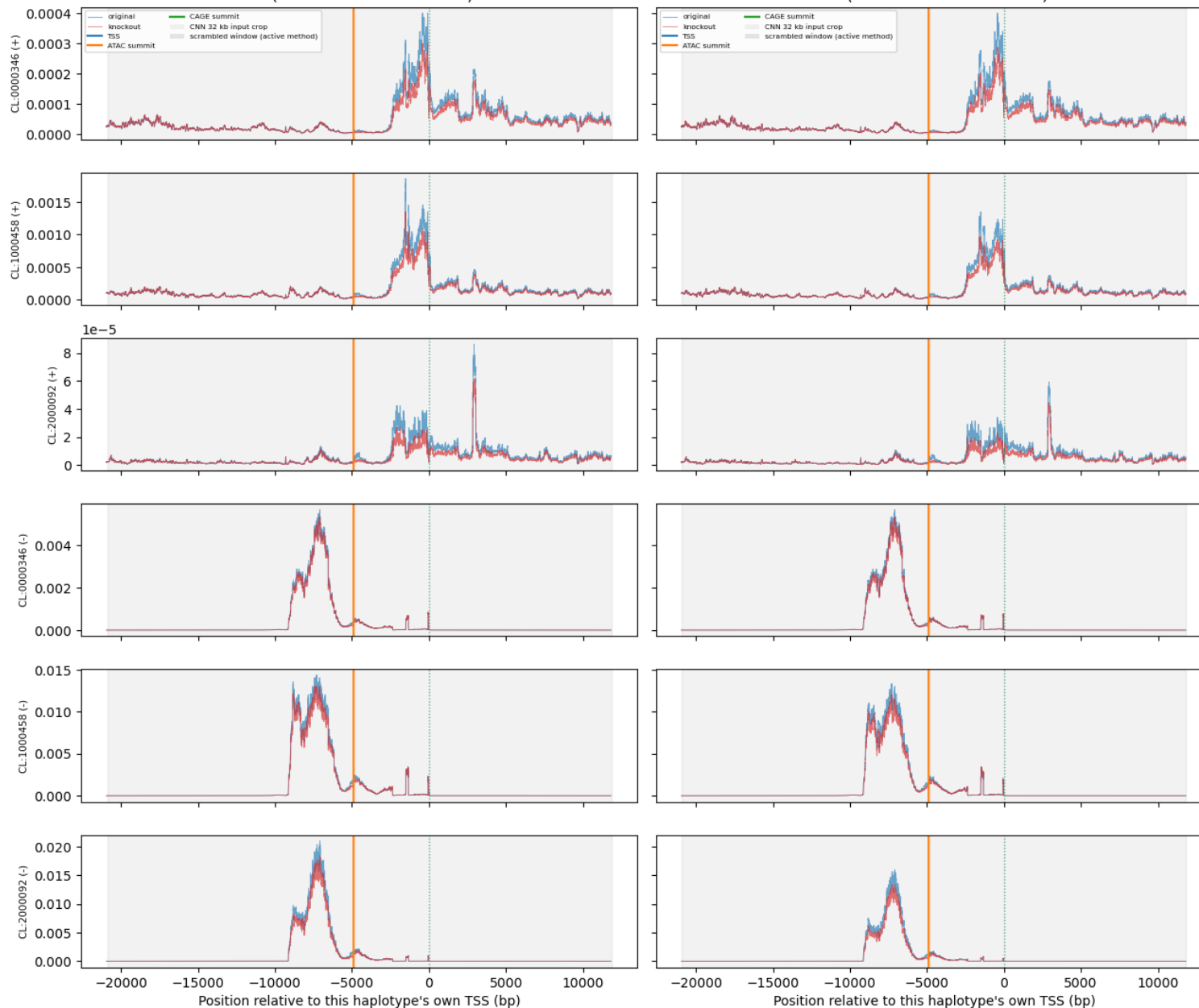
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [-0.7745795  0.7840071], delta(strong pigmentation)=-1.5998
--- TCHH / biology_tss ---
```

TCHH -- HG02461 -- edited: TSS (GENCODE MANE Select) @ chr1:152,115,445 -- delta(strong pigmentation logit) = +0.0001
H1 (scrambled -50..50) H2 (scrambled -50..50)



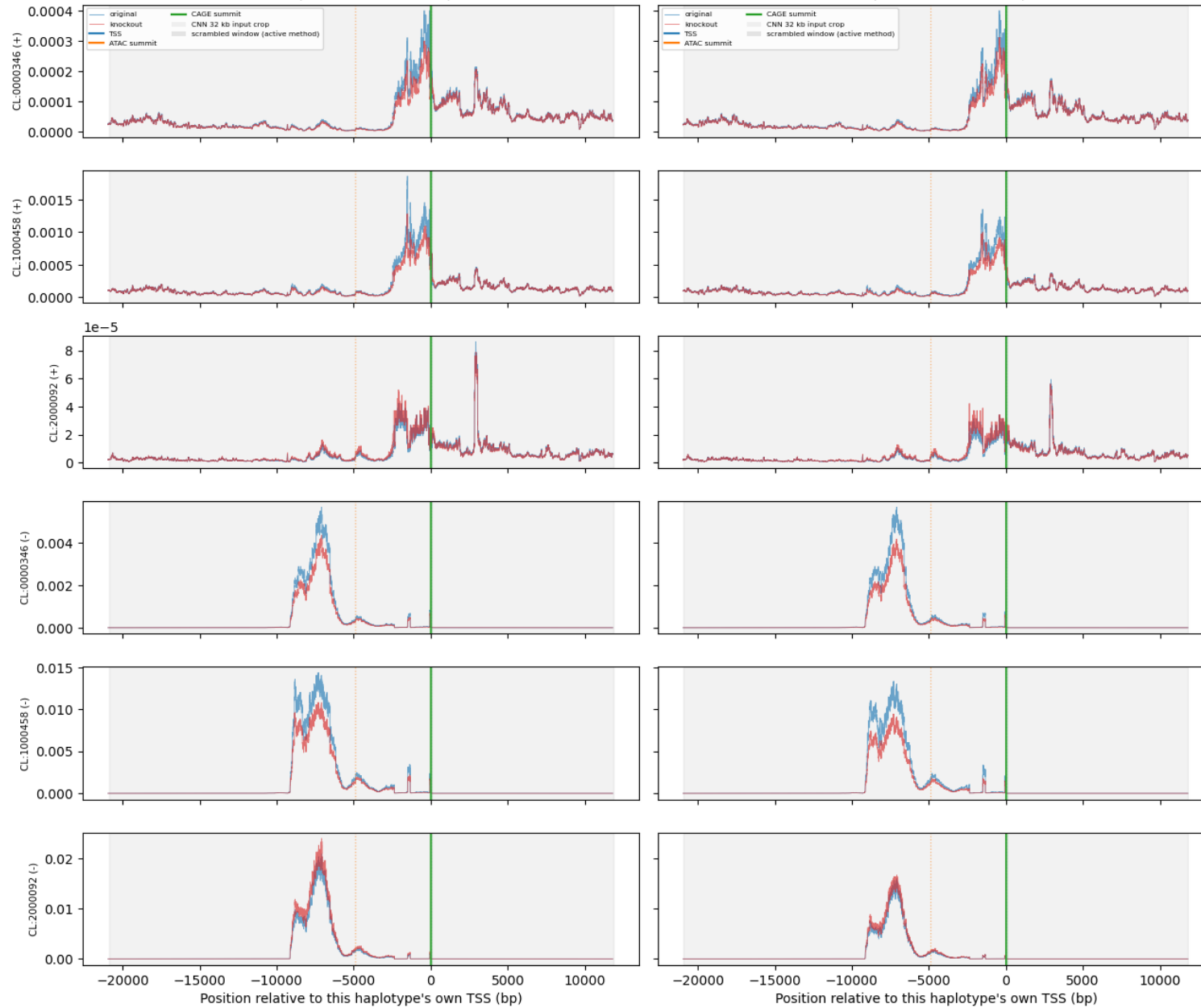
```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.82534647 -2.046229 ], delta(strong pigmentation)=+0.0001
--- TCHH / alphagenome_atac ---
```

TCHH -- HG02461 -- edited: ATAC accessibility summit (unfiltered ontology) @ chr1:152,110,548 -- delta(strong pigmentation logit) = +0.0000
H1 (scrambled -4947..-4847) H2 (scrambled -4947..-4847)



```
baseline logits: [ 0.8252655 -2.046092 ], perturbed logits: [ 0.8252902 -2.0461285], delta(strong pigmentation)=+0.0000  
--- TCHH / cage_melanocyte ---
```

TCHH -- HG02461 -- edited: CAGE summit (dark/light melanocyte) @ chr1:152,115,444 -- delta(strong pigmentation logit) = +0.0000
H1 (scrambled -51..49) H2 (scrambled -51..49)



baseline logits: [0.8252655 -2.046092], perturbed logits: [0.82527137 -2.0461347], delta(strong pigmentation)=+0.0000

Summary across all genes and all three location methods

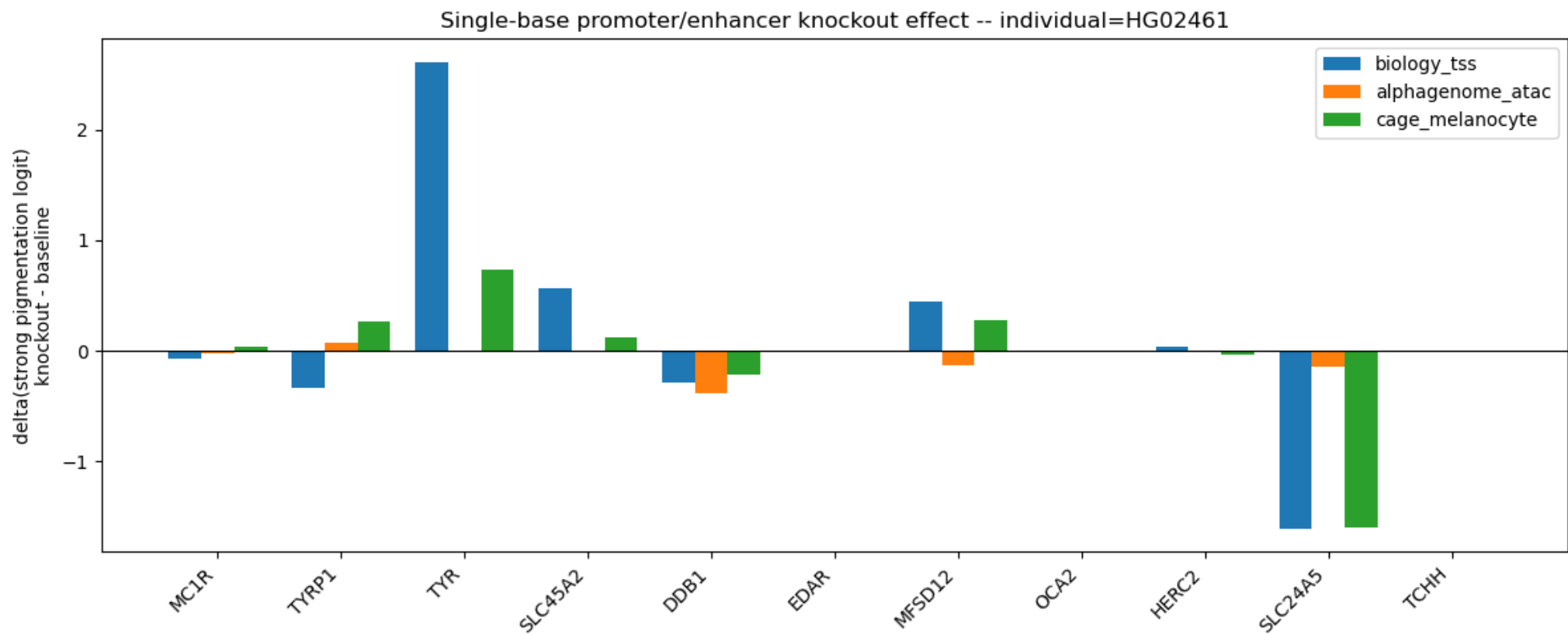
In [27]:

```
knockout_summary_df = pd.DataFrame([
    {
        "gene": r["gene"], "method": r["method"], "chrom": r["chrom"], "genomic_pos": r["genomic_pos"],
        "scramble_window": r["scramble_window"],
        "delta_strong_logit": r["delta_strong_logit"],
    }
    for r in knockout_results
])
display(knockout_summary_df)

fig, ax = plt.subplots(figsize=(12, 5))
genes_order = KO_GENES
x = np.arange(len(genes_order))
n_methods = len(KO_METHODS)
width = 0.8 / n_methods
for i, method in enumerate(KO_METHODS):
    sub = knockout_summary_df[knockout_summary_df["method"] == method].set_index("gene").reindex(genes_order)
    offset = (i - (n_methods - 1) / 2) * width
    ax.bar(x + offset, sub["delta_strong_logit"], width, label=method)
ax.axhline(0, color="black", linewidth=0.8)
ax.set_xticks(x)
ax.set_xticklabels(genes_order, rotation=45, ha="right")
ax.set_ylabel(f"delta({class_names[ko_strong_idx]} logit)\nknockout - baseline")
ax.set_title(f"Single-base promoter/enhancer knockout effect -- individual={INDIVIDUAL_ID}")
ax.legend()
fig.tight_layout()
plt.show()
```

	gene	method	chrom	genomic_pos	scramble_window	delta_strong_logit
0	MC1R	biology_tss	chr16	89918862	100	-0.071373
1	MC1R	alphagenome_atac	chr16	89923221	100	-0.029881
2	MC1R	cage_melanocyte	chr16	89923342	100	0.033824
3	TYRP1	biology_tss	chr9	12693385	100	-0.341042
4	TYRP1	alphagenome_atac	chr9	12695707	100	0.071725
5	TYRP1	cage_melanocyte	chr9	12694229	100	0.266630
6	TYR	biology_tss	chr11	89177875	100	2.613200
7	TYR	alphagenome_atac	chr11	89176088	100	-0.010408
8	TYR	cage_melanocyte	chr11	89177871	100	0.739081
9	SLC45A2	biology_tss	chr5	33984694	100	0.566392
10	SLC45A2	alphagenome_atac	chr5	33980218	100	-0.010702
11	SLC45A2	cage_melanocyte	chr5	33984654	100	0.115062
12	DDB1	biology_tss	chr11	61333106	100	-0.283630
13	DDB1	alphagenome_atac	chr11	61333101	100	-0.386120
14	DDB1	cage_melanocyte	chr11	61333070	100	-0.216315
15	EDAR	biology_tss	chr2	108989221	100	0.002727
16	EDAR	alphagenome_atac	chr2	108989278	100	0.002594
17	EDAR	cage_melanocyte	chr2	108989139	100	0.002070
18	MFSD12	biology_tss	chr19	3557587	100	0.449231
19	MFSD12	alphagenome_atac	chr19	3557762	100	-0.133101
20	MFSD12	cage_melanocyte	chr19	3557584	100	0.280356
21	OCA2	biology_tss	chr15	28099316	100	0.002184
22	OCA2	alphagenome_atac	chr15	28096239	100	0.002887
23	OCA2	cage_melanocyte	chr15	28099317	100	0.003340
24	HERC2	biology_tss	chr15	28322180	100	0.035602
25	HERC2	alphagenome_atac	chr15	28319572	100	-0.011374
26	HERC2	cage_melanocyte	chr15	28322143	100	-0.038112
27	SLC24A5	biology_tss	chr15	48120990	100	-1.609322
28	SLC24A5	alphagenome_atac	chr15	48119479	100	-0.138876
29	SLC24A5	cage_melanocyte	chr15	48121018	100	-1.599845
30	TCHH	biology_tss	chr1	152115445	100	0.000081

	gene	method	chrom	genomic_pos	scramble_window	delta_strong_logit
31	TCHH	alphagenome_atac	chr1	152110548	100	0.000025
32	TCHH	cage_melanocyte	chr1	152115444	100	0.000006



10. Bulk knockout results -- comparing the three location methods across the whole test split

Section 9 walked through the knockout mechanics for a single individual. Separately, the exact same `knockout_gene_experiment` logic was run as a standalone script (`bulk_knockout.py` , not part of this notebook) over **all 162 individuals in the test split x all 11 genes x all 3 location methods** (`biology_tss` , `alphagenome_atac` , `cage_melanocyte`), scrambling a 100 bp window at each candidate position and recording the baseline and perturbed logits. Results are cached at `results/genotype_based_predictor/knockout_bulk/pigmentation_test_split_knockout.csv` (5,346 rows, 0 failures). This section loads that CSV and compares the three methods against each other: which genes and methods produce the strongest, most consistent, most agreed-upon knockout effect.

In [28]:

```
BULK_RESULTS_PATH = REPO_ROOT / "results" / "genotype_based_predictor" / "knockout_bulk" / "pigmentation_test_split_knockout.csv"
bulk_df = pd.read_csv(BULK_RESULTS_PATH)
print(f"Loaded {len(bulk_df)} rows: {bulk_df['sample_id'].nunique()} individuals x "
      f"{bulk_df['gene'].nunique()} genes x {bulk_df['method'].nunique()} methods")

METHOD_COLORS = {"biology_tss": "#1f77b4", "alphagenome_atac": "#ff7f0e", "cage_melanocyte": "#2ca02c"}
METHOD_LABELS = {"biology_tss": "TSS (biology)", "alphagenome_atac": "ATAC summit", "cage_melanocyte": "CAGE summit"}
BULK_GENES = sorted(bulk_df["gene"].unique(), key=lambda g: KO_GENES.index(g) if g in KO_GENES else 999)
BULK_METHODS = ["biology_tss", "alphagenome_atac", "cage_melanocyte"]

gene_method_stats = (
    bulk_df.groupby(["gene", "method"])
    .agg(
        mean_baseline=("baseline_strong_logit", "mean"),
        mean_perturbed=("perturbed_strong_logit", "mean"),
        mean_delta=("delta_strong_logit", "mean"),
        std_delta=("delta_strong_logit", "std"),
        flip_rate=("flipped", "mean"),
        n=("flipped", "size"),
    )
    .reset_index()
)
gene_method_stats["flip_rate_pct"] = gene_method_stats["flip_rate"] * 100
display(gene_method_stats.sort_values("flip_rate", ascending=False).head(15))
```

Loaded 5346 rows: 162 individuals x 11 genes x 3 methods

	gene	method	mean_baseline	mean_perturbed	mean_delta	std_delta	flip_rate	n	flip_rate_pct
19	SLC24A5	biology_tss	0.125281	-1.291005	-1.416285	0.461311	0.685185	162	68.518519
20	SLC24A5	cage_melanocyte	0.125281	-1.282255	-1.407536	0.460260	0.679012	162	67.901235
0	DDB1	alphagenome_atac	0.125281	-0.514329	-0.639610	0.270685	0.345679	162	34.567901
31	TYRP1	biology_tss	0.125281	-0.435350	-0.560630	0.284311	0.327160	162	32.716049
1	DDB1	biology_tss	0.125281	-0.419403	-0.544683	0.235176	0.308642	162	30.864198
29	TYR	cage_melanocyte	0.125281	1.279924	1.154644	0.672153	0.259259	162	25.925926
28	TYR	biology_tss	0.125281	2.508581	2.383300	0.894233	0.259259	162	25.925926
22	SLC45A2	biology_tss	0.125281	0.789538	0.664258	0.207033	0.222222	162	22.222222
13	MFSD12	biology_tss	0.125281	0.737346	0.612065	0.239844	0.216049	162	21.604938
2	DDB1	cage_melanocyte	0.125281	-0.265968	-0.391248	0.182861	0.209877	162	20.987654
14	MFSD12	cage_melanocyte	0.125281	0.544715	0.419434	0.161636	0.172840	162	17.283951
12	MFSD12	alphagenome_atac	0.125281	-0.111655	-0.236935	0.094857	0.123457	162	12.345679
18	SLC24A5	alphagenome_atac	0.125281	-0.077743	-0.203023	0.092654	0.117284	162	11.728395
30	TYRP1	alphagenome_atac	0.125281	0.376935	0.251655	0.177201	0.080247	162	8.024691
32	TYRP1	cage_melanocyte	0.125281	0.418522	0.293242	0.098119	0.080247	162	8.024691

10.1 Population-level arrow plot -- mean baseline -> mean perturbed logit, per gene x method

Each arrow starts at the population-mean **strong pigmentation** logit before knocking out a gene's regulatory region, and points to the mean logit after. Longer arrows pointing left (toward negative, i.e. toward **weak pigmentation**) mean a more effective knockout on average, for that method.

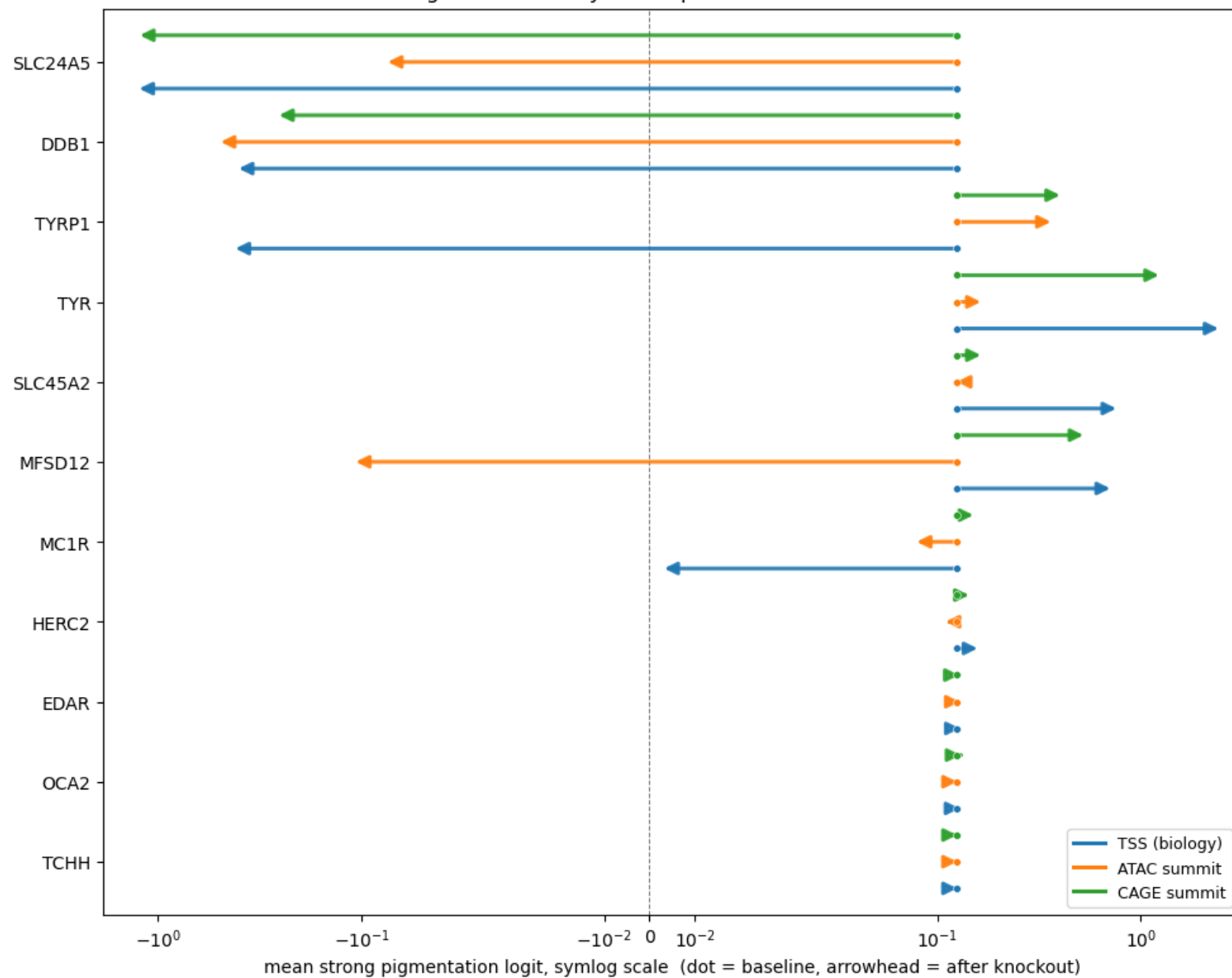
Genes are ranked by **flip rate** (Section 10.2), not raw mean-logit-shift: TYR's logit shift is ~20x larger than any other gene's (its scrambled-window knockout appears to push the model into an out-of-distribution / saturating regime -- flagged as an open caveat in Section 9), which would otherwise dominate the ranking despite not being the most decisive knockout (see the flip-rate chart). The x-axis uses a symlog scale so both TYR's large excursions and the other genes' subtler shifts stay legible in one plot.

In [29]:

```
gene_max_flip_rate = gene_method_stats.groupby("gene")["flip_rate"].max().sort_values(ascending=False)
gene_order_by_effect = gene_max_flip_rate.index.tolist()

fig, ax = plt.subplots(figsize=(10, 0.62 * len(gene_order_by_effect) + 1.5))
n_methods = len(BULK_METHODS)
row_height = 0.62
all_x = []
for gi, gene in enumerate(gene_order_by_effect):
    y_base = (len(gene_order_by_effect) - 1 - gi) * row_height * n_methods
    for mi, method in enumerate(BULK_METHODS):
        row = gene_method_stats[(gene_method_stats["gene"] == gene) & (gene_method_stats["method"] == method)]
        if row.empty:
            continue
        row = row.iloc[0]
        y = y_base + mi * row_height
        color = METHOD_COLORS[method]
        ax.annotate(
            "", xy=(row["mean_perturbed"], y), xytext=(row["mean_baseline"], y),
            arrowprops=dict(arrowstyle="->", color=color, lw=2.2, alpha=0.9, mutation_scale=14),
        )
        ax.scatter([row["mean_baseline"]], [y], color=color, s=18, zorder=3, edgecolor="white", linewidth=0.4)
        all_x.extend([row["mean_baseline"], row["mean_perturbed"]])
ax.axvline(0, color="black", linewidth=0.7, linestyle="--", alpha=0.5)
yticks = [(len(gene_order_by_effect) - 1 - gi) * row_height * n_methods + row_height * (n_methods - 1) / 2 for gi in
range(len(gene_order_by_effect))]
ax.set_yticks(yticks)
ax.set_yticklabels(gene_order_by_effect)
ax.set_xscale("symlog", lincthresh=0.05)
x_lo, x_hi = min(all_x), max(all_x)
x_pad = 0.15 * (x_hi - x_lo)
ax.set_xlim(x_lo - x_pad, x_hi + x_pad)
ax.set_xlabel(f"mean {class_names[ko_strong_idx]} logit, symlog scale (dot = baseline, arrowhead = after knockout)")
ax.set_title(f"Population-mean knockout effect by gene x method (n={bulk_df['sample_id'].nunique()} individuals)\ngenotypes ranked by max flip rate across methods")
handles = [plt.Line2D([], [], color=METHOD_COLORS[m], lw=2.2, label=METHOD_LABELS[m]) for m in BULK_METHODS]
ax.legend(handles=handles, loc="lower right", fontsize=9)
ax.set_ylim(-row_height, len(gene_order_by_effect) * row_height * n_methods)
fig.tight_layout()
fig.savefig(KNOCKOUT_FIG_DIR / "bulk_arrow_population_mean.png", dpi=140)
plt.show()
```

Population-mean knockout effect by gene x method (n=162 individuals)
genes ranked by max flip rate across methods

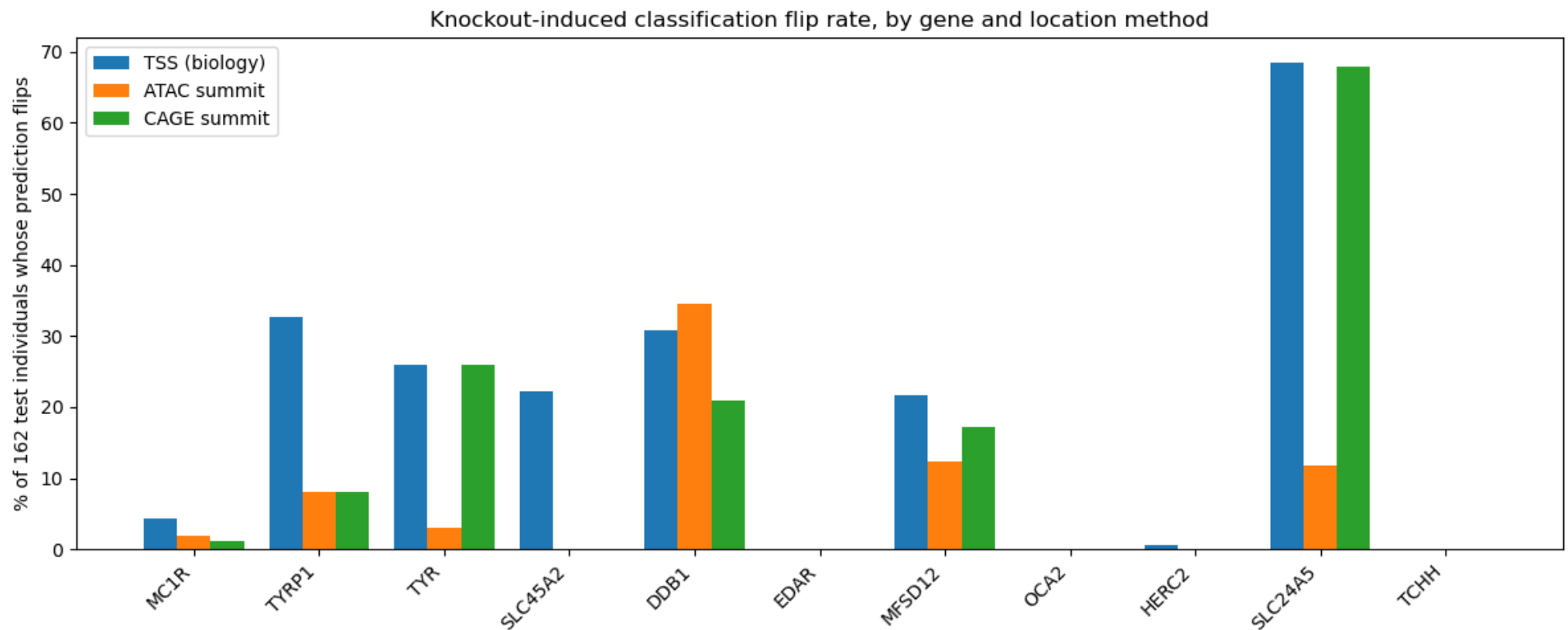


10.2 Flip rate by gene x method, across all 162 test individuals

The fraction of individuals whose predicted class (strong pigmentation vs weak pigmentation) actually flips after the knockout -- the clearest, most interpretable measure of knockout effectiveness.

In [30]:

```
fig, ax = plt.subplots(figsize=(12, 5))
x = np.arange(len(BULK_GENES))
width = 0.8 / len(BULK_METHODS)
for i, method in enumerate(BULK_METHODS):
    sub = gene_method_stats[gene_method_stats["method"] == method].set_index("gene").reindex(BULK_GENES)
    offset = (i - (len(BULK_METHODS) - 1) / 2) * width
    ax.bar(x + offset, sub["flip_rate_pct"], width, label=METHOD_LABELS[method], color=METHOD_COLORS[method])
ax.set_xticks(x)
ax.set_xticklabels(BULK_GENES, rotation=45, ha="right")
ax.set_ylabel("% of 162 test individuals whose prediction flips")
ax.set_title("Knockout-induced classification flip rate, by gene and location method")
ax.legend()
fig.tight_layout()
fig.savefig(KNOCKOUT_FIG_DIR / "bulk_flip_rate_bars.png", dpi=140)
plt.show()
```

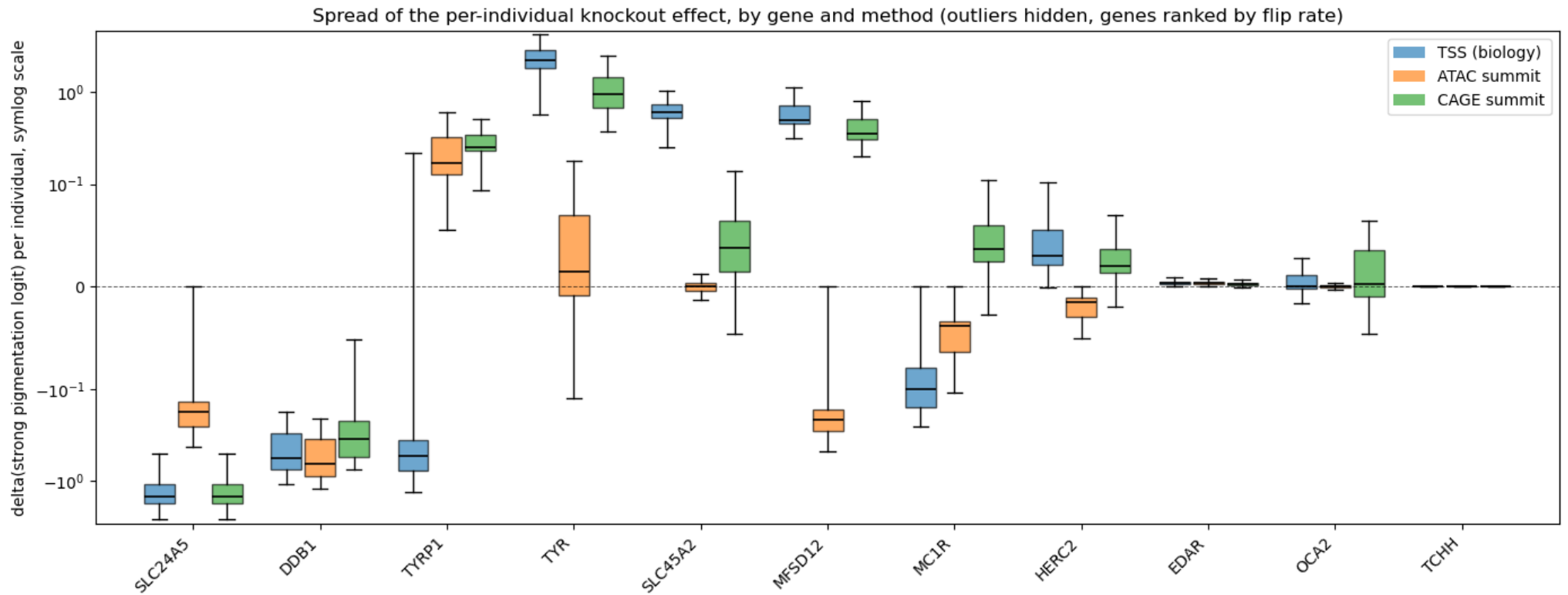


10.3 Distribution of the effect (not just the mean) -- box plots of `delta_strong_logit`

The mean can hide a lot: a method might have a large mean delta driven by a few outlier individuals, or a small but very consistent delta across everyone. Box plots (one box per gene x method) show the spread.

In [31]:

```
fig, ax = plt.subplots(figsize=(14, 5.5))
box_data, box_positions, box_colors = [], [], []
group_width = 0.8
n_methods = len(BULK_METHODS)
for gi, gene in enumerate(gene_order_by_effect):
    for mi, method in enumerate(BULK_METHODS):
        vals = bulk_df[(bulk_df["gene"] == gene) & (bulk_df["method"] == method)]["delta_strong_logit"].values
        box_data.append(vals)
        box_positions.append(gi + (mi - (n_methods - 1) / 2) * (group_width / n_methods))
        box_colors.append(METHOD_COLORS[method])
bp = ax.boxplot(
    box_data, positions=box_positions, widths=group_width / n_methods * 0.9, patch_artist=True,
    showfliers=False, medianprops=dict(color="black", linewidth=1.2),
)
for patch, color in zip(bp["boxes"], box_colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.65)
ax.axhline(0, color="black", linewidth=0.7, linestyle="--", alpha=0.6)
ax.set_xticks(range(len(gene_order_by_effect)))
ax.set_xticklabels(gene_order_by_effect, rotation=45, ha="right")
ax.set_yscale("symlog", linthresh=0.1)
ax.set_ylabel(f"delta({class_names[ko_strong_idx]} logit) per individual, symlog scale")
ax.set_title("Spread of the per-individual knockout effect, by gene and method (outliers hidden, genes ranked by flip rate)")
handles = [Patch(facecolor=METHOD_COLORS[m], alpha=0.65, label=METHOD_LABELS[m]) for m in BULK_METHODS]
ax.legend(handles=handles)
fig.tight_layout()
fig.savefig(KNOCKOUT_FIG_DIR / "bulk_delta_boxplots.png", dpi=140)
plt.show()
```



10.4 Individual-level arrow plots for the top genes -- every one of the 162 individuals

For the genes with the strongest population-level effect, this shows every individual's own baseline $\rightarrow$ perturbed arrow (sorted by baseline logit), one column per method, so patterns like "only strong-pigmentation individuals flip" or "this method barely moves anyone" are visible directly, not just in aggregate.

In [32]:

```
TOP_N_GENES_DETAIL = 4
top_genes_detail = gene_order_by_effect[:TOP_N_GENES_DETAIL]
print(f"Top {TOP_N_GENES_DETAIL} genes by max flip rate: {top_genes_detail}")

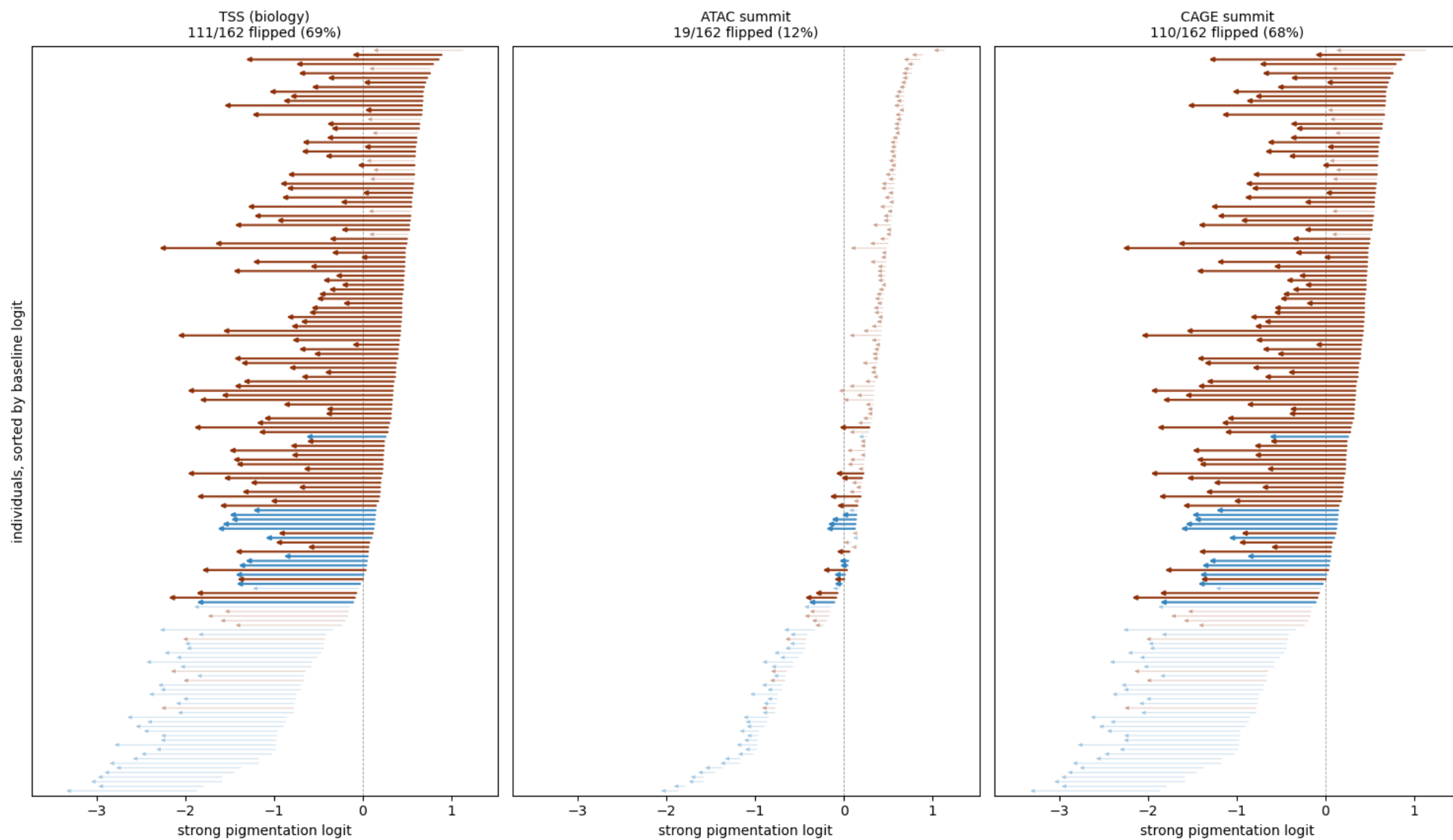
TRUE_LABEL_COLORS = {"strong pigmentation": "#8c2d04", "weak pigmentation": "#3182bd"}

for gene in top_genes_detail:
    fig, axes = plt.subplots(1, len(BULK_METHODS), figsize=(15, 9), sharey=True)
    gene_df = bulk_df[bulk_df["gene"] == gene]
    gene_x_lo = gene_df[["baseline_strong_logit", "perturbed_strong_logit"]].min().min()
    gene_x_hi = gene_df[["baseline_strong_logit", "perturbed_strong_logit"]].max().max()
    gene_x_pad = 0.08 * (gene_x_hi - gene_x_lo + 1e-6)
    for ax, method in zip(axes, BULK_METHODS):
        sub = gene_df[gene_df["method"] == method].sort_values("baseline_strong_logit").reset_index(drop=True)
        for row_i, row in sub.iterrows():
            color = TRUE_LABEL_COLORS.get(row["true_label"], "gray")
            alpha = 0.9 if row["flipped"] else 0.25
            lw = 1.6 if row["flipped"] else 0.7
            ax.annotate(
                "", xy=(row["perturbed_strong_logit"], row_i), xytext=(row["baseline_strong_logit"], row_i),
                arrowprops=dict(arrowstyle="->", color=color, lw=lw, alpha=alpha, mutation_scale=6),
            )
        ax.set_xlim(gene_x_lo - gene_x_pad, gene_x_hi + gene_x_pad)
        ax.set_ylim(-1, len(sub))
        ax.axvline(0, color="black", linewidth=0.6, linestyle="--", alpha=0.4)
        n_flipped = int(sub["flipped"].sum())
        ax.set_title(f"{METHOD_LABELS[method]}\n{n_flipped}/{len(sub)} flipped ({100 * n_flipped / len(sub):.0f}%)", fontsize=10)
        ax.set_xlabel(f"{class_names[ko_strong_idx]} logit")
    axes[0].set_ylabel("individuals, sorted by baseline logit")
    for ax in axes:
        ax.set_yticks([])
    handles = [plt.Line2D([], [], color=c, lw=1.6, label=lbl) for lbl, c in TRUE_LABEL_COLORS.items()]
    handles.append(plt.Line2D([], [], color="gray", lw=0.7, alpha=0.25, label="not flipped (faint)"))
    fig.legend(handles=handles, loc="upper center", ncol=3, bbox_to_anchor=(0.5, 1.02), fontsize=9)
    fig.suptitle(f"{gene}: per-individual knockout arrows, baseline -> perturbed (n={sub['sample_id'].nunique()})", y=1.06)
    fig.tight_layout()
    fig.savefig(KNOCKOUT_FIG_DIR / f"bulk_arrows_individual_{gene}.png", dpi=140, bbox_inches="tight")
    plt.show()
```

Top 4 genes by max flip rate: ['SLC24A5', 'DDB1', 'TYRP1', 'TYR']

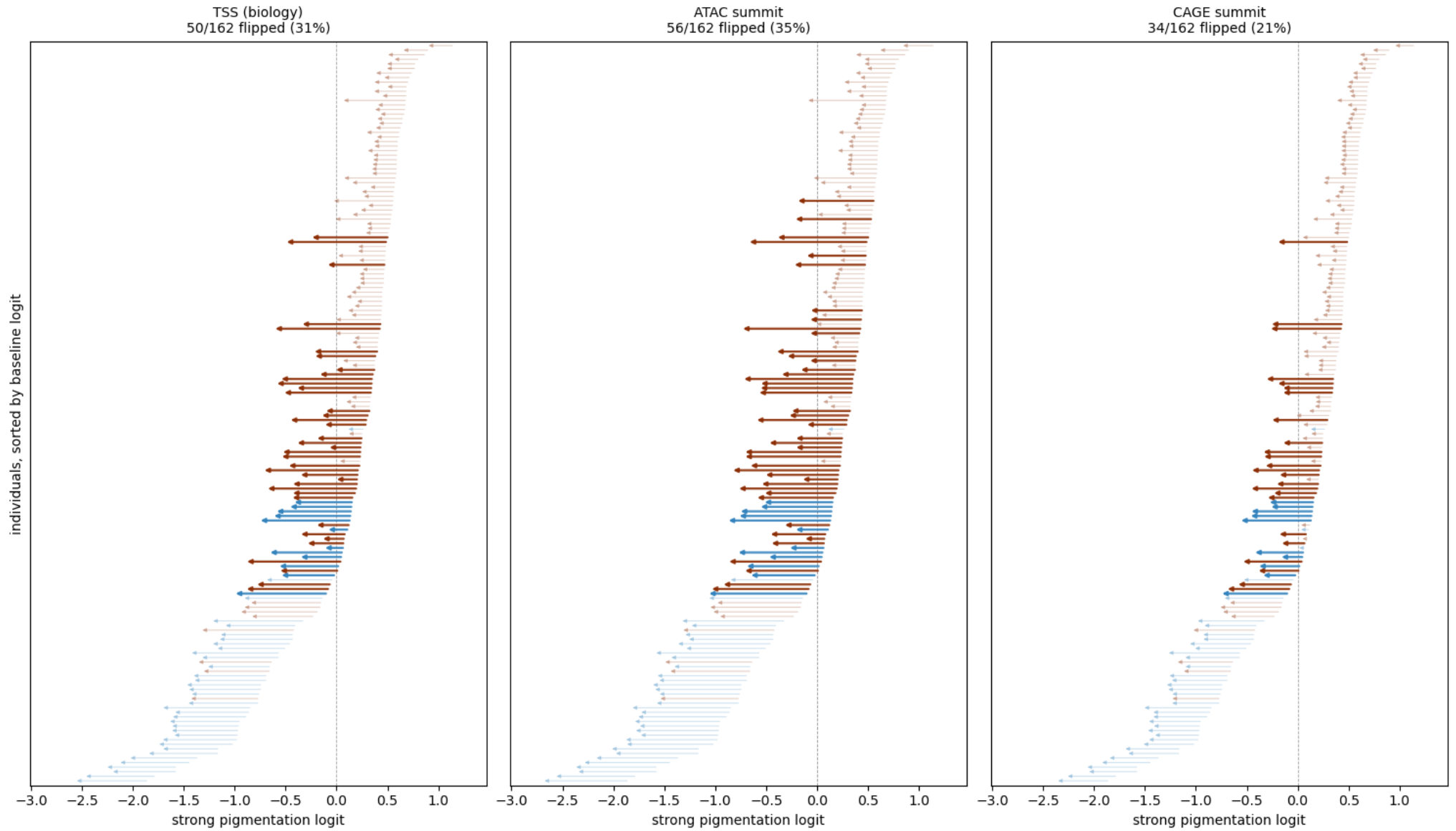
SLC24A5: per-individual knockout arrows, baseline -> perturbed (n=162)

strong pigmentation weak pigmentation not flipped (faint)



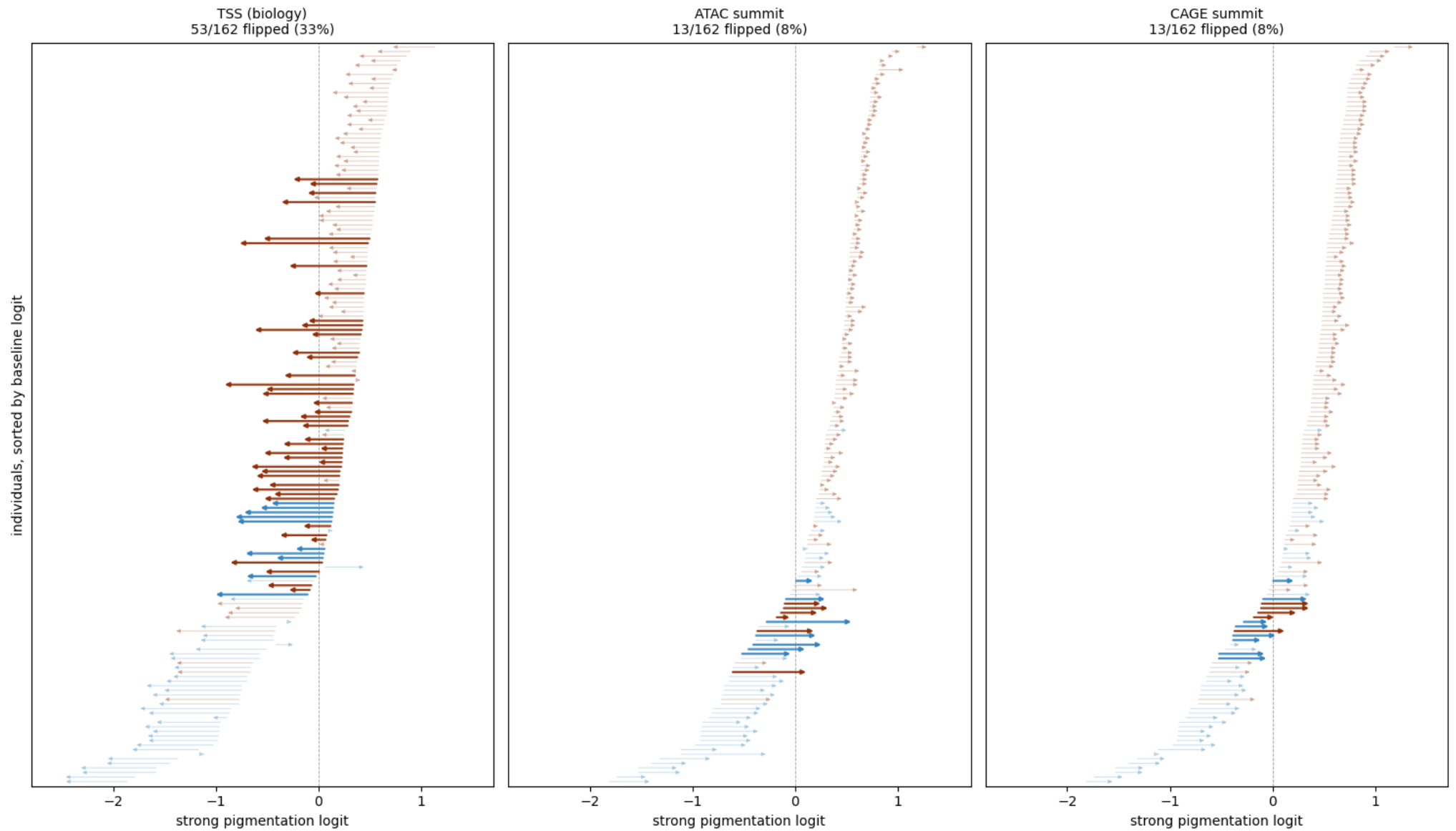
DDB1: per-individual knockout arrows, baseline -> perturbed (n=162)

strong pigmentation weak pigmentation not flipped (faint)



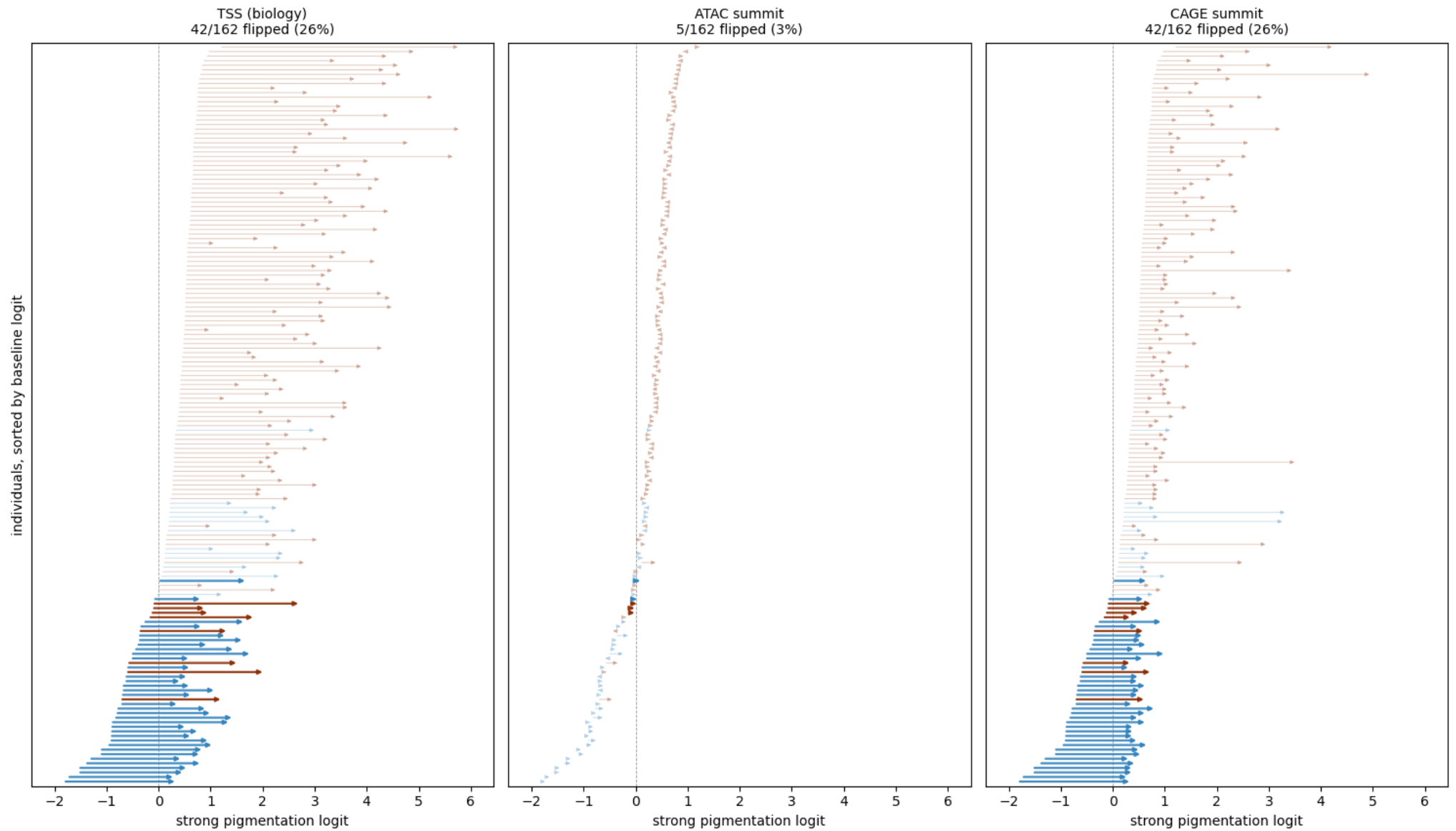
TYRP1: per-individual knockout arrows, baseline -> perturbed (n=162)

strong pigmentation weak pigmentation not flipped (faint)



TYR: per-individual knockout arrows, baseline -> perturbed (n=162)

strong pigmentation weak pigmentation not flipped (faint)



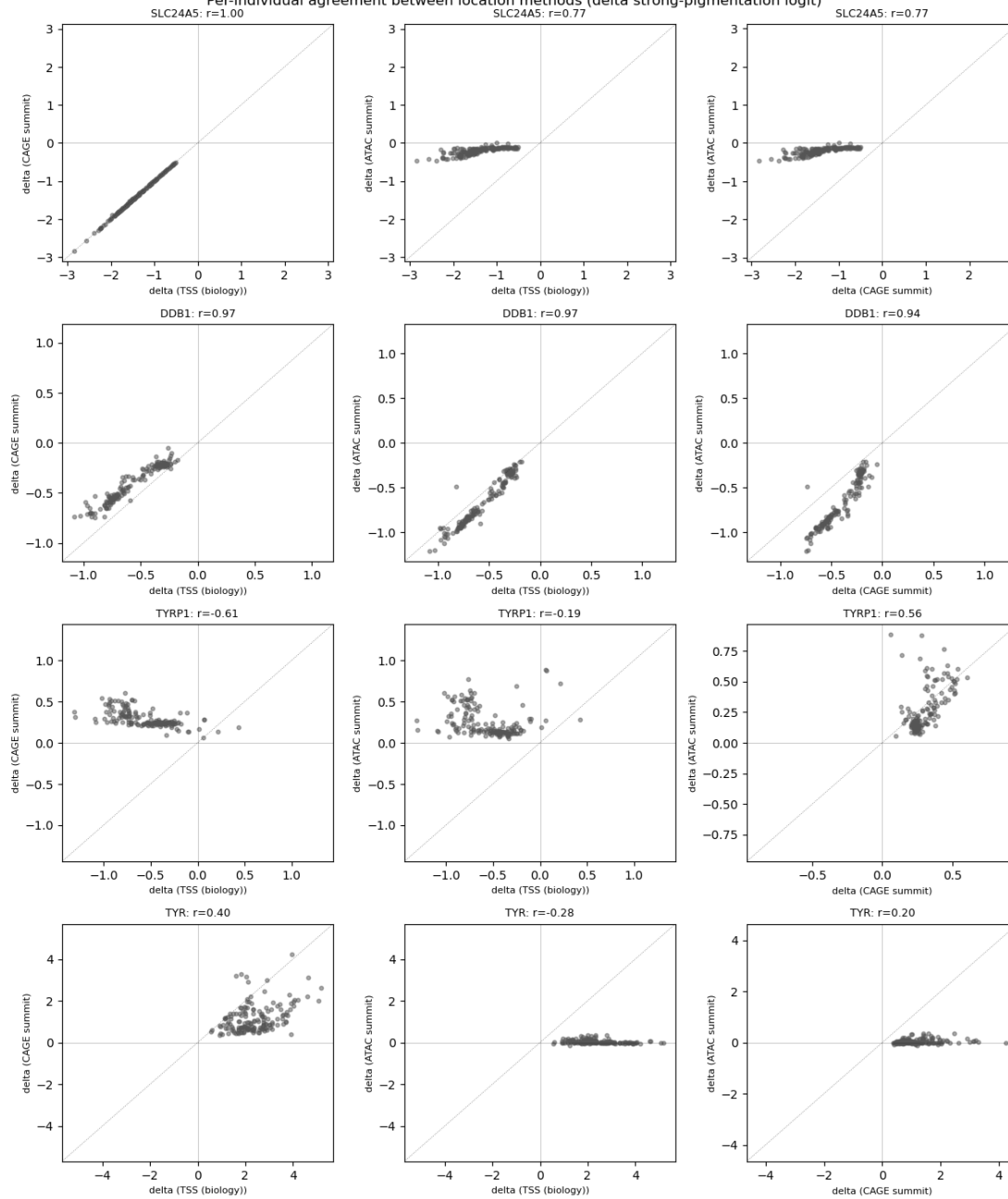
10.5 Do the three methods agree? Per-individual delta correlation

If `biology_tss` and `cage_melanocyte` are both correctly finding the same functional promoter, their per-individual knockout effects on the same person should correlate -- even though the exact scrambled window differs (different candidate position, possibly different haplotype-local coordinates).

In [33]:

```
method_pairs = [("biology_tss", "cage_melanocyte"), ("biology_tss", "alphagenome_atac"), ("cage_melanocyte", "alphagenome_atac")]
fig, axes = plt.subplots(len(top_genes_detail), len(method_pairs), figsize=(4 * len(method_pairs), 3.6 * len(top_genes_detail)))
for gi, gene in enumerate(top_genes_detail):
    gene_df = bulk_df[bulk_df["gene"] == gene]
    for pi, (m1, m2) in enumerate(method_pairs):
        ax = axes[gi, pi] if len(top_genes_detail) > 1 else axes[pi]
        d1 = gene_df[gene_df["method"] == m1].set_index("sample_id")["delta_strong_logit"]
        d2 = gene_df[gene_df["method"] == m2].set_index("sample_id")["delta_strong_logit"]
        common = d1.index.intersection(d2.index)
        x_vals, y_vals = d1.loc[common].values, d2.loc[common].values
        ax.scatter(x_vals, y_vals, s=10, alpha=0.5, color="#555555")
        if len(common) > 1 and x_vals.std() > 0 and y_vals.std() > 0:
            r = np.corrcoef(x_vals, y_vals)[0, 1]
        else:
            r = float("nan")
        lim = max(np.abs(x_vals).max(), np.abs(y_vals).max(), 1e-6) * 1.1
        ax.plot([-lim, lim], [-lim, lim], color="black", linewidth=0.6, linestyle=":", alpha=0.5)
        ax.axhline(0, color="black", linewidth=0.5, alpha=0.3)
        ax.axvline(0, color="black", linewidth=0.5, alpha=0.3)
        ax.set_xlim(-lim, lim)
        ax.set_ylim(-lim, lim)
        ax.set_title(f"{gene}: r={r:.2f}", fontsize=9)
        ax.set_xlabel(f"delta ({METHOD_LABELS[m1]})", fontsize=8)
        ax.set_ylabel(f"delta ({METHOD_LABELS[m2]})", fontsize=8)
fig.suptitle("Per-individual agreement between location methods (delta strong-pigmentation logit)")
fig.tight_layout()
fig.savefig(KNOCKOUT_FIG_DIR / "bulk_method_agreement_scatter.png", dpi=140)
plt.show()
```

Per-individual agreement between location methods (delta strong-pigmentation logit)



10.6 Population breakdown for the single strongest gene

If a knockout is genuinely capturing a functional pigmentation mechanism, its effect should track individual genotype/ancestry (strong pigmentation populations should show larger negative deltas), not be a uniform artifact applied identically regardless of population.

In [34]:

```
top_gene = gene_order_by_effect[0]
pop_order = (
    bulk_df[["population", "superpopulation"]].drop_duplicates().sort_values(["superpopulation", "population"])["population"].tolist()
)
fig, axes = plt.subplots(1, len(BULK_METHODS), figsize=(16, 5), sharey=True)
for ax, method in zip(axes, BULK_METHODS):
    sub = bulk_df[(bulk_df["gene"] == top_gene) & (bulk_df["method"] == method)]
    data = [sub[sub["population"] == pop]["delta_strong_logit"].values for pop in pop_order]
    bp = ax.boxplot(data, tick_labels=pop_order, patch_artist=True, showfliers=True)
    for patch, pop in zip(bp["boxes"], pop_order):
        superpop = bulk_df[bulk_df["population"] == pop]["superpopulation"].iloc[0]
        patch.set_facecolor("#8c2d04" if superpop == "AFR" else "#3182bd")
        patch.set_alpha(0.6)
    ax.axhline(0, color="black", linewidth=0.6, linestyle="--", alpha=0.5)
    ax.set_title(METHOD_LABELS[method], fontsize=10)
    ax.tick_params(axis="x", rotation=45)
axes[0].set_ylabel(f"delta({class_names[ko_strong_idx]} logit)")
fig.suptitle(f"{top_gene}: per-population knockout effect (AFR=strong-pigmentation ancestry in orange, EUR=weak in blue)")
fig.tight_layout()
fig.savefig(KNOCKOUT_FIG_DIR / f"bulk_population_breakdown_{top_gene}.png", dpi=140)
plt.show()
```

SLC24A5: per-population knockout effect (AFR=strong-pigmentation ancestry in orange, EUR=weak in blue)

